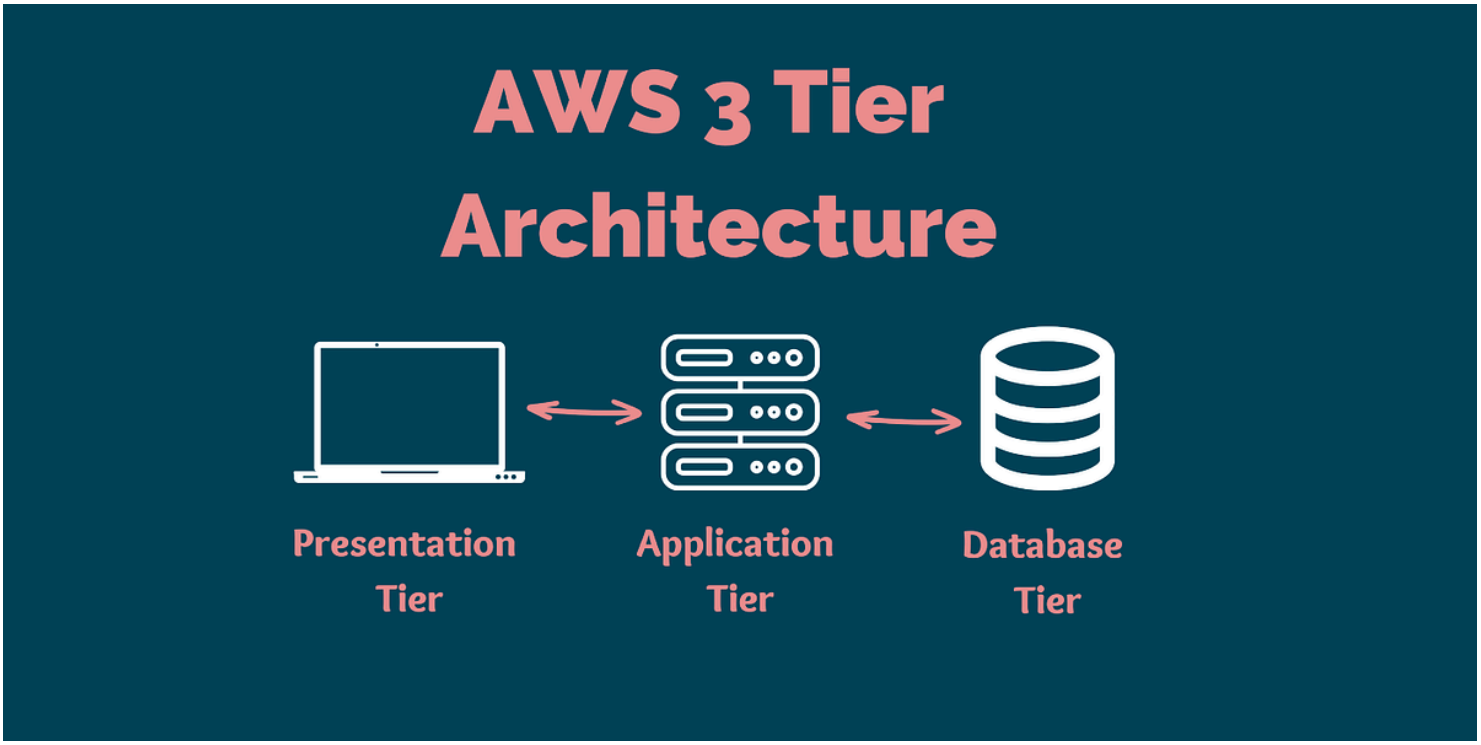
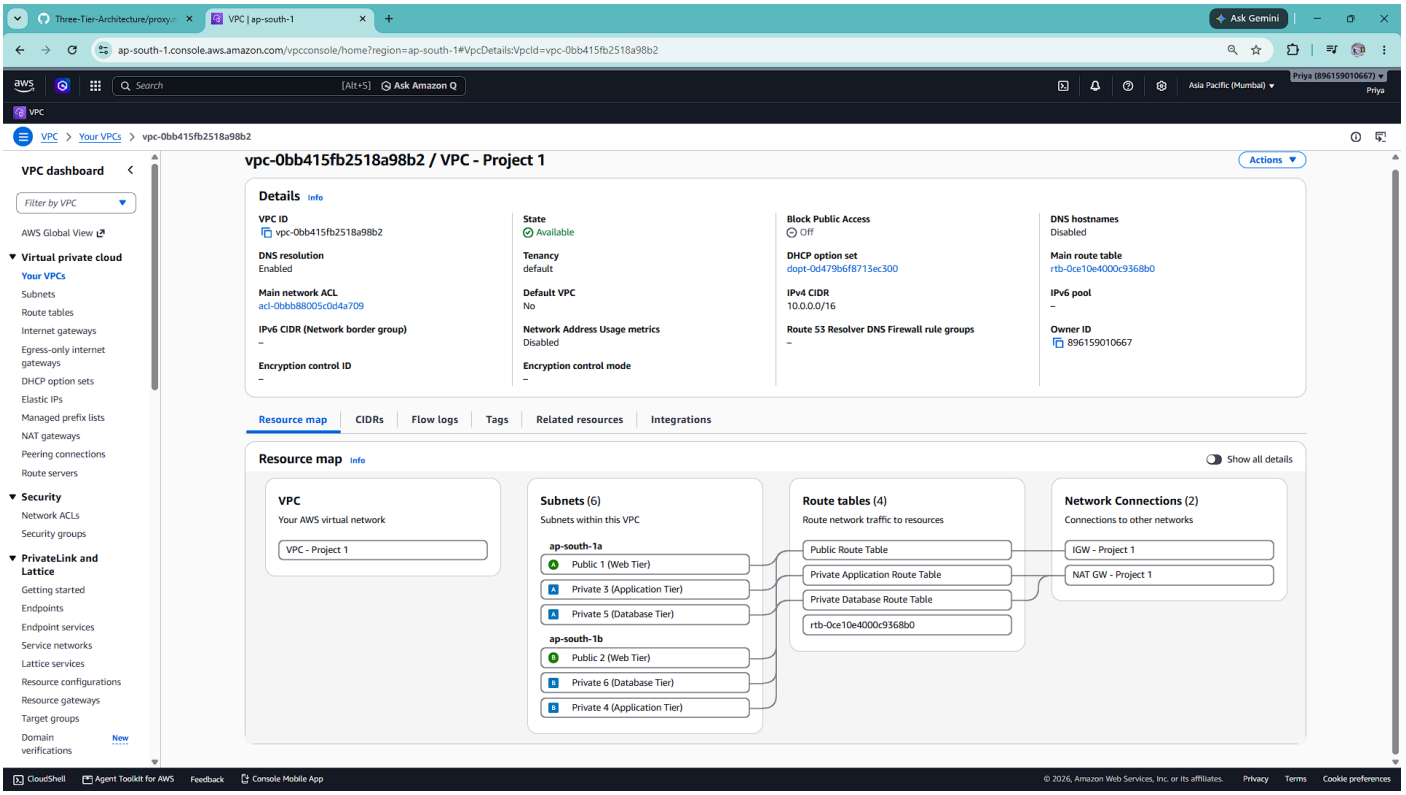
PROJECT - 1

Building Scalable, Secure, and High-Performance Web Applications with AWS 3-Tier Architecture

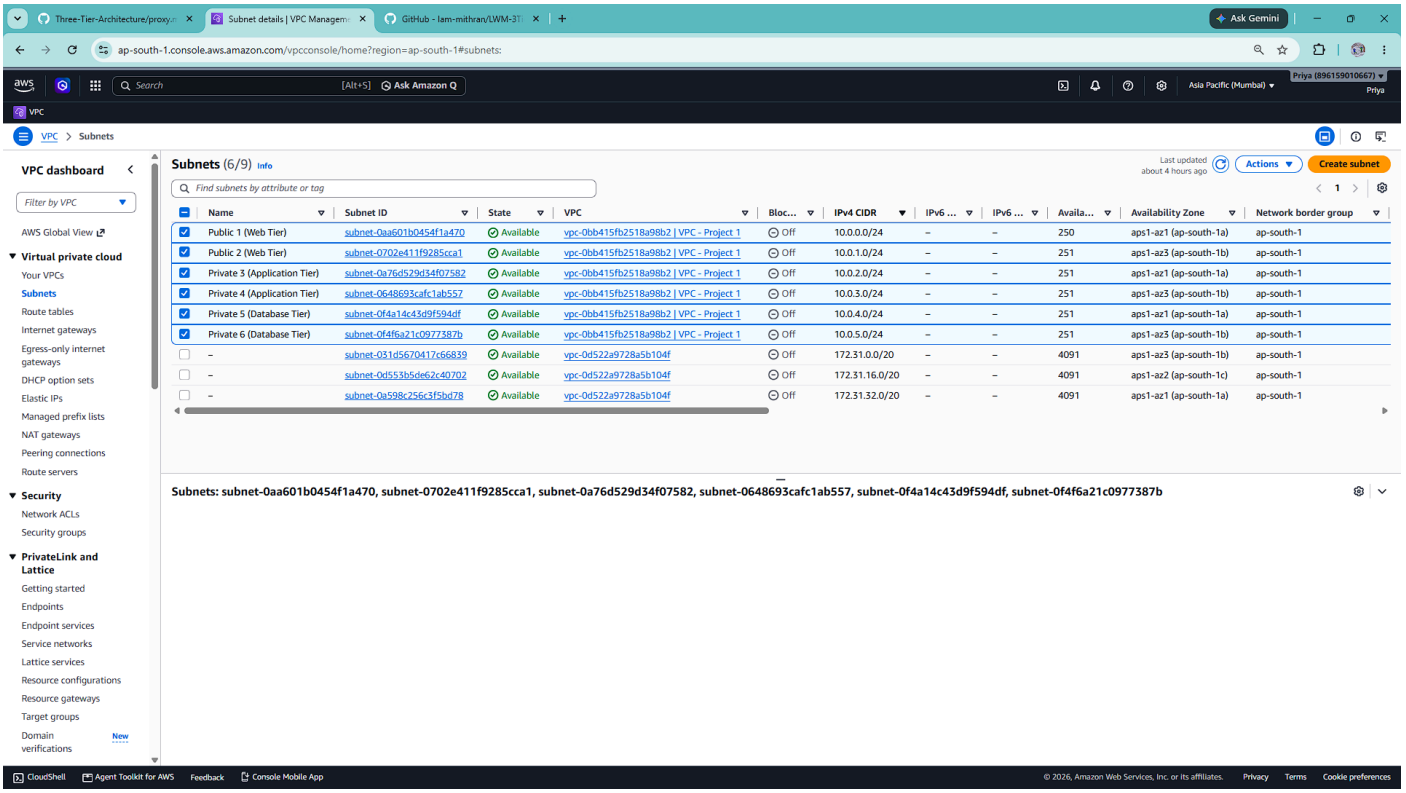


Task 1 - Design and Configure the AWS Networking Infrastructure

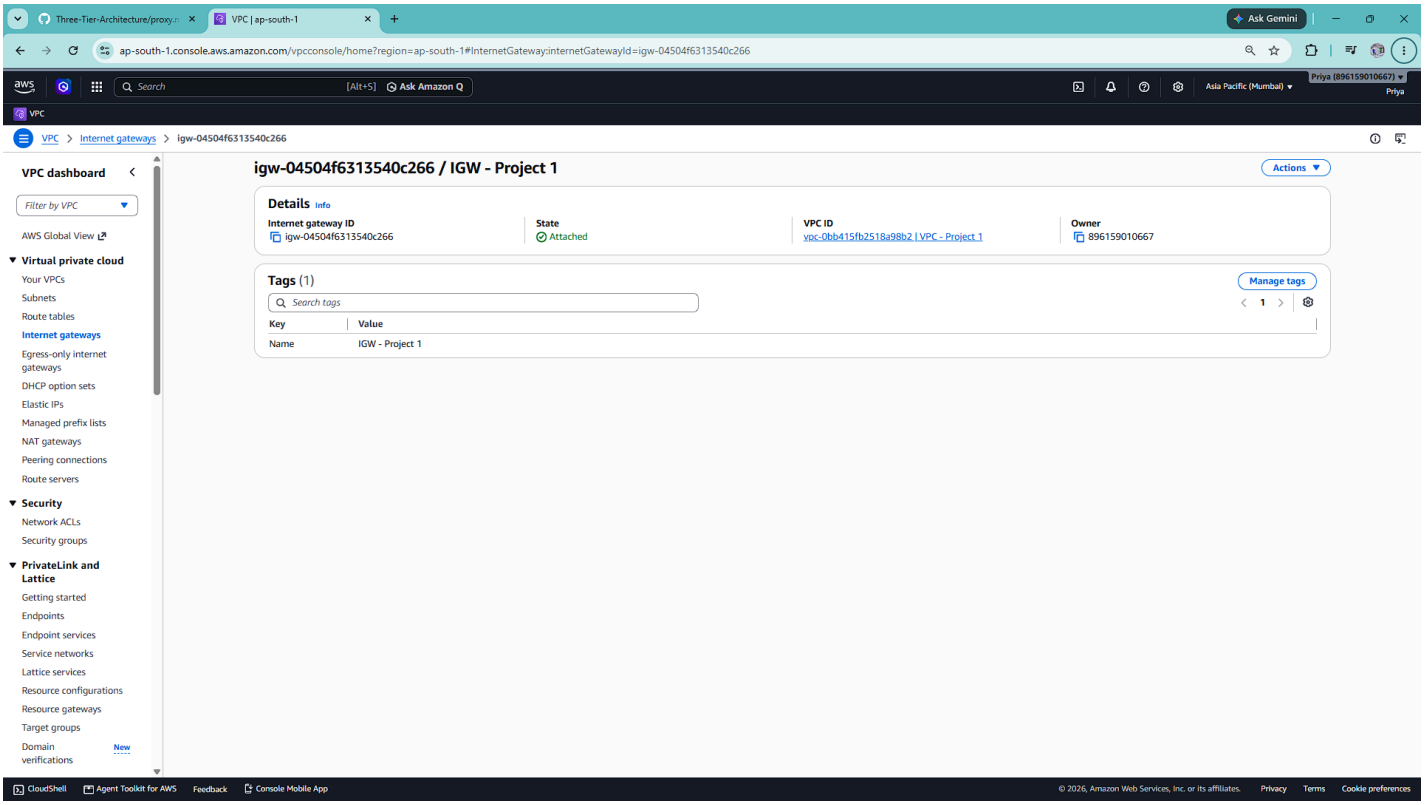
VPC : Created a custom VPC named "VPC - Project 1" with the CIDR block 10.0.0.0/16. It serves as the private network for all project resources.



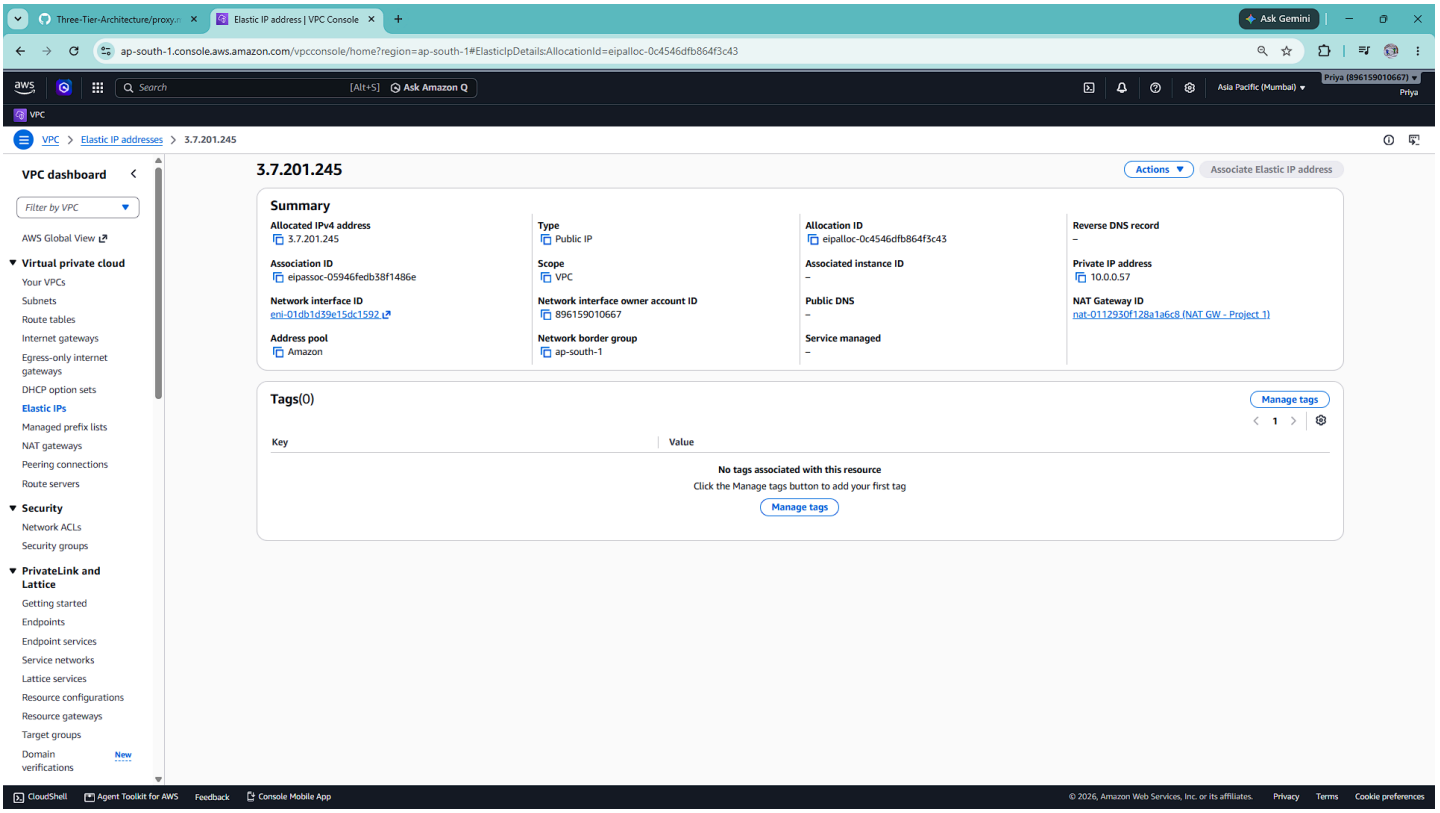
Subnets : Created six subnets across two Availability Zones (ap-south-1a and ap-south-1b) for high availability, with each subnet using a /24 CIDR block. The subnets are divided into three tiers - 2 Public subnets for the Web tier, 2 Private subnets for the Application tier, and 2 Private subnets for the Database tier.



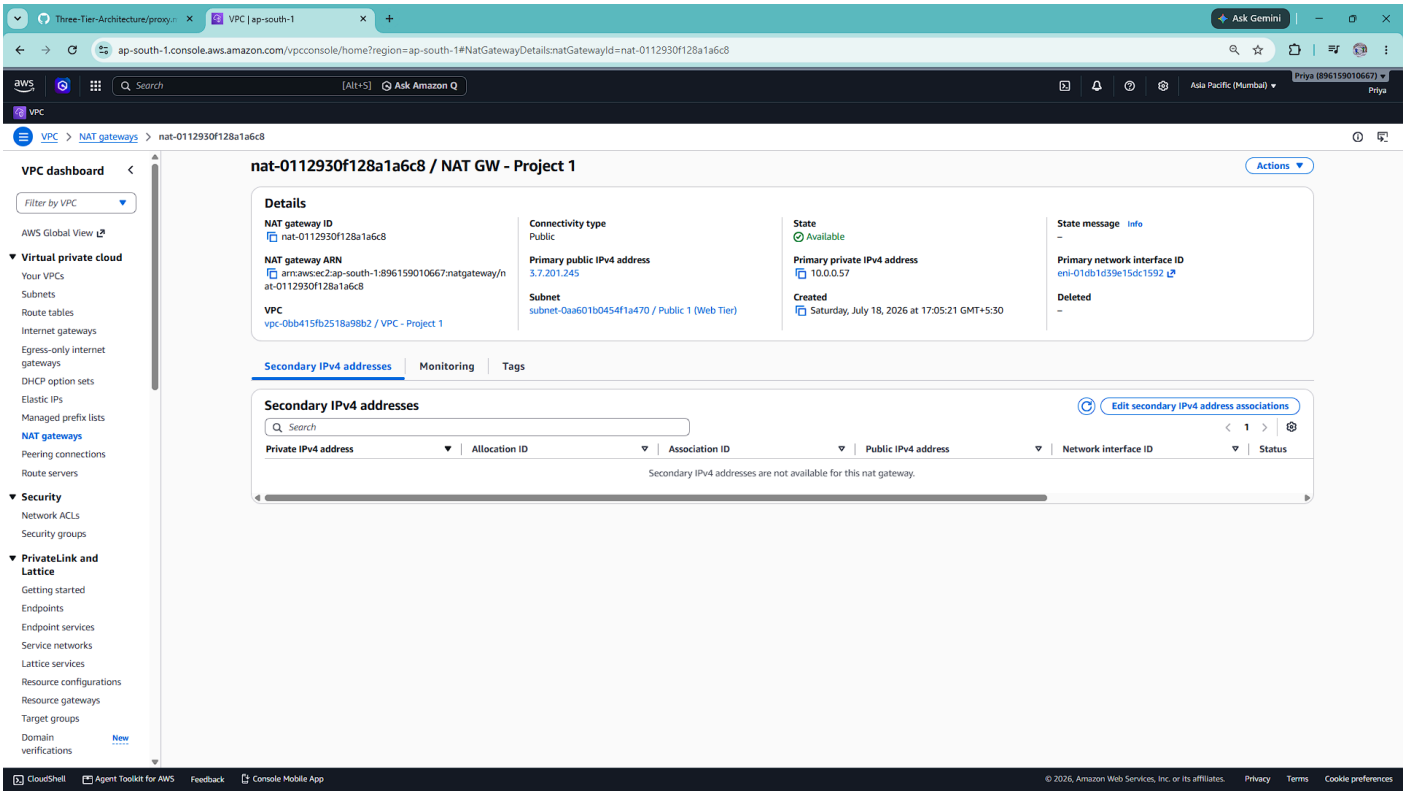
Internet Gateway : Created an Internet Gateway called "IGW - Project 1" and attached it to the VPC. It enables the public subnets to communicate with the internet.



Elastic IP : Allocated an Elastic IP and assigned it to the NAT Gateway. This gives the NAT Gateway a fixed public IP address.

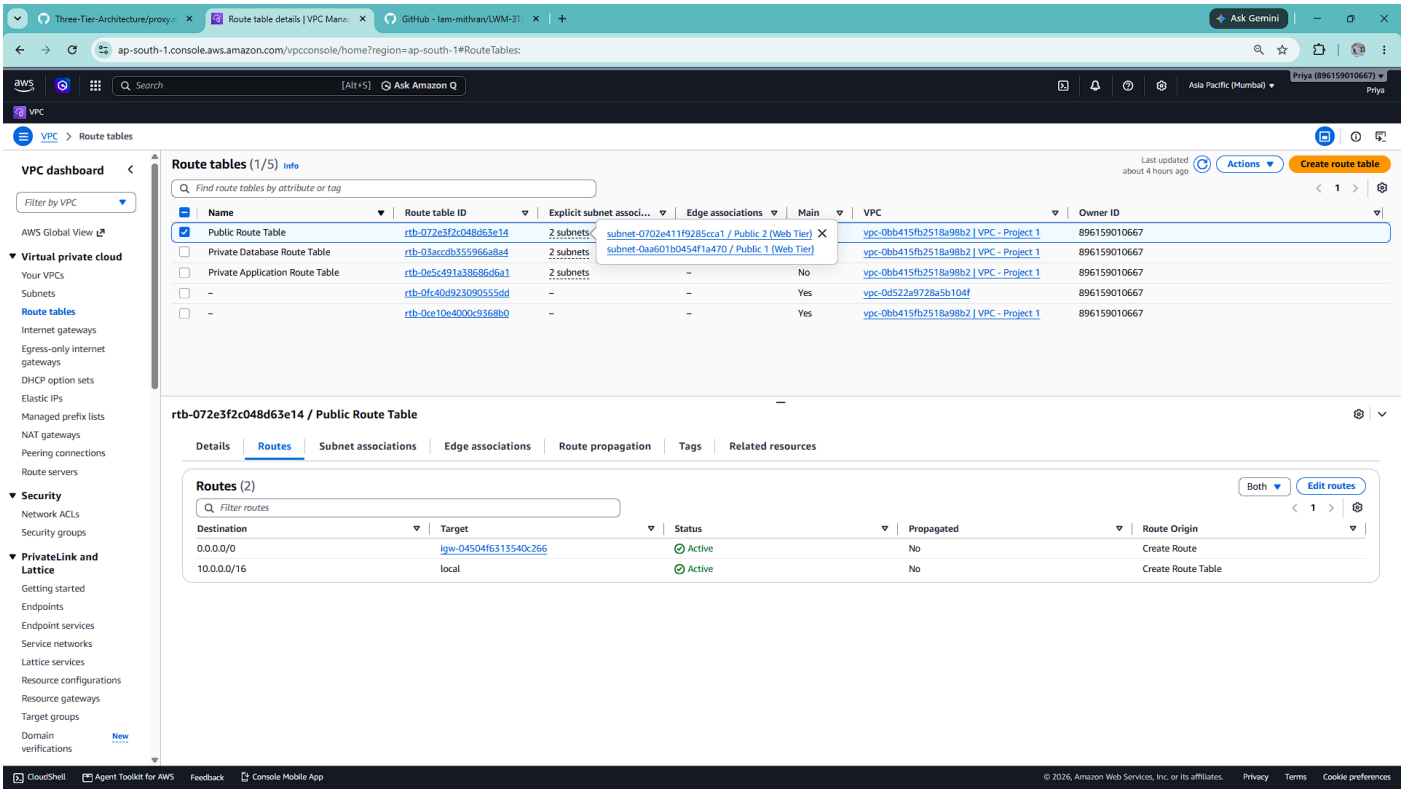


NAT Gateway : Created a NAT Gateway named "NAT GW - Project 1" in the “Public 1 (Web Tier)” subnet. It allows the EC2 instances in the private Application and Database subnets to access the internet for downloading updates and installing packages, while keeping the instances private and inaccessible from the internet.

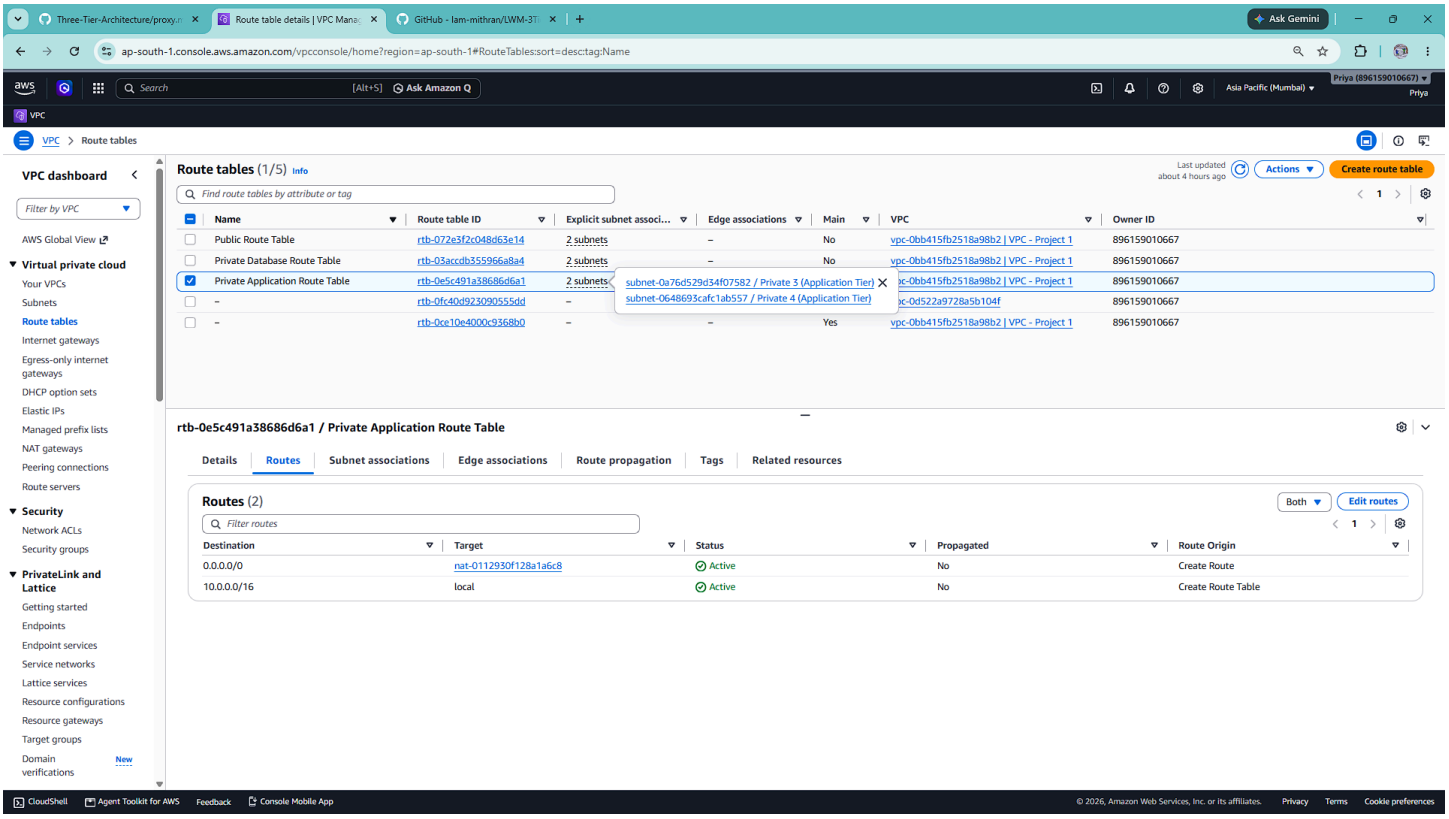


Route Tables : Created three route tables and associated them with the appropriate subnets.

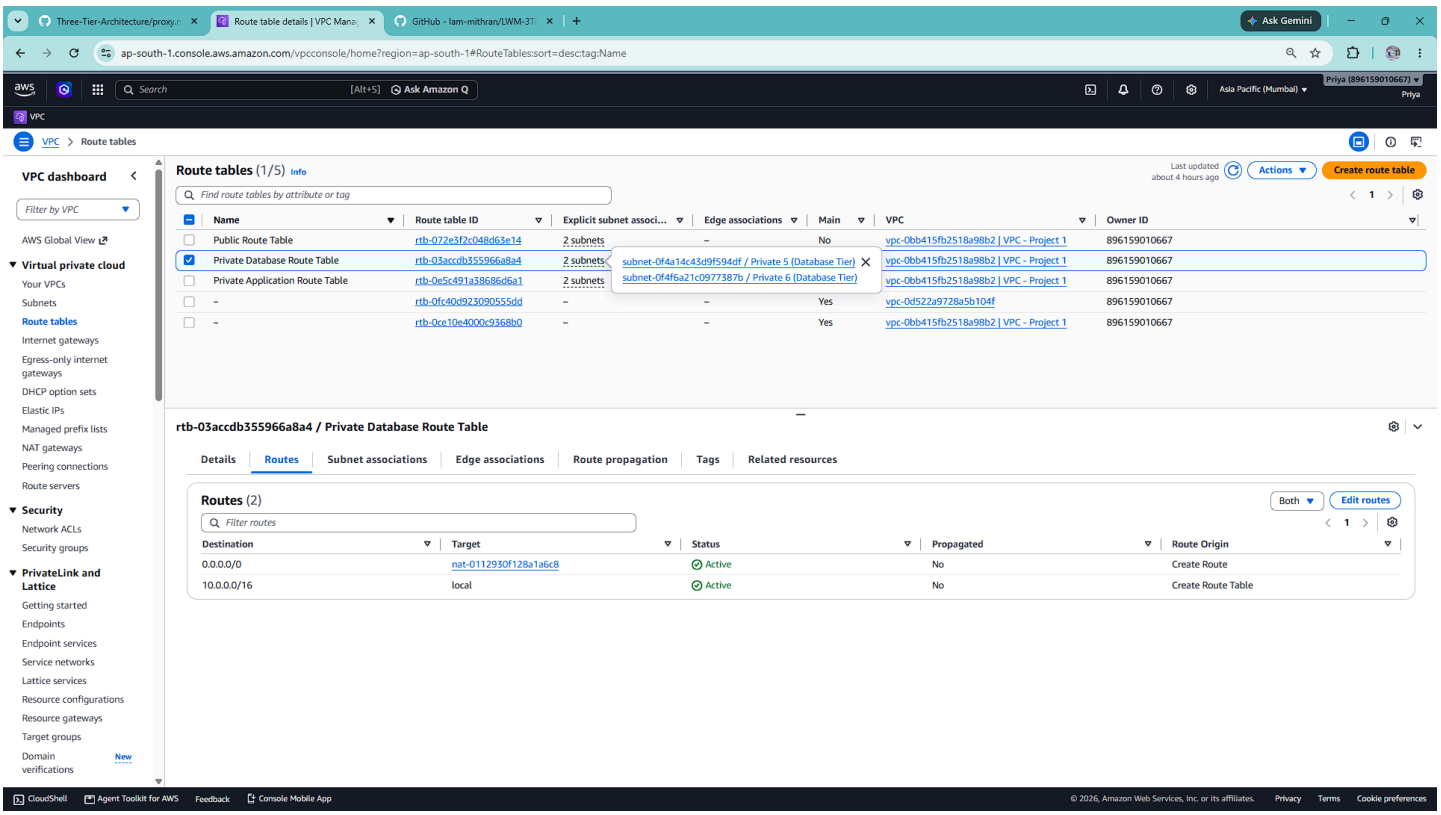
Public Route Table - Sends internet traffic to the IGW. Linked to Public 1 (Web Tier) and Public 2 (Web Tier).



Private Application Route Table - Sends internet traffic through the NAT Gateway so App servers can install packages. Linked to Private 3 (Application Tier) and Private 4 (Application Tier).

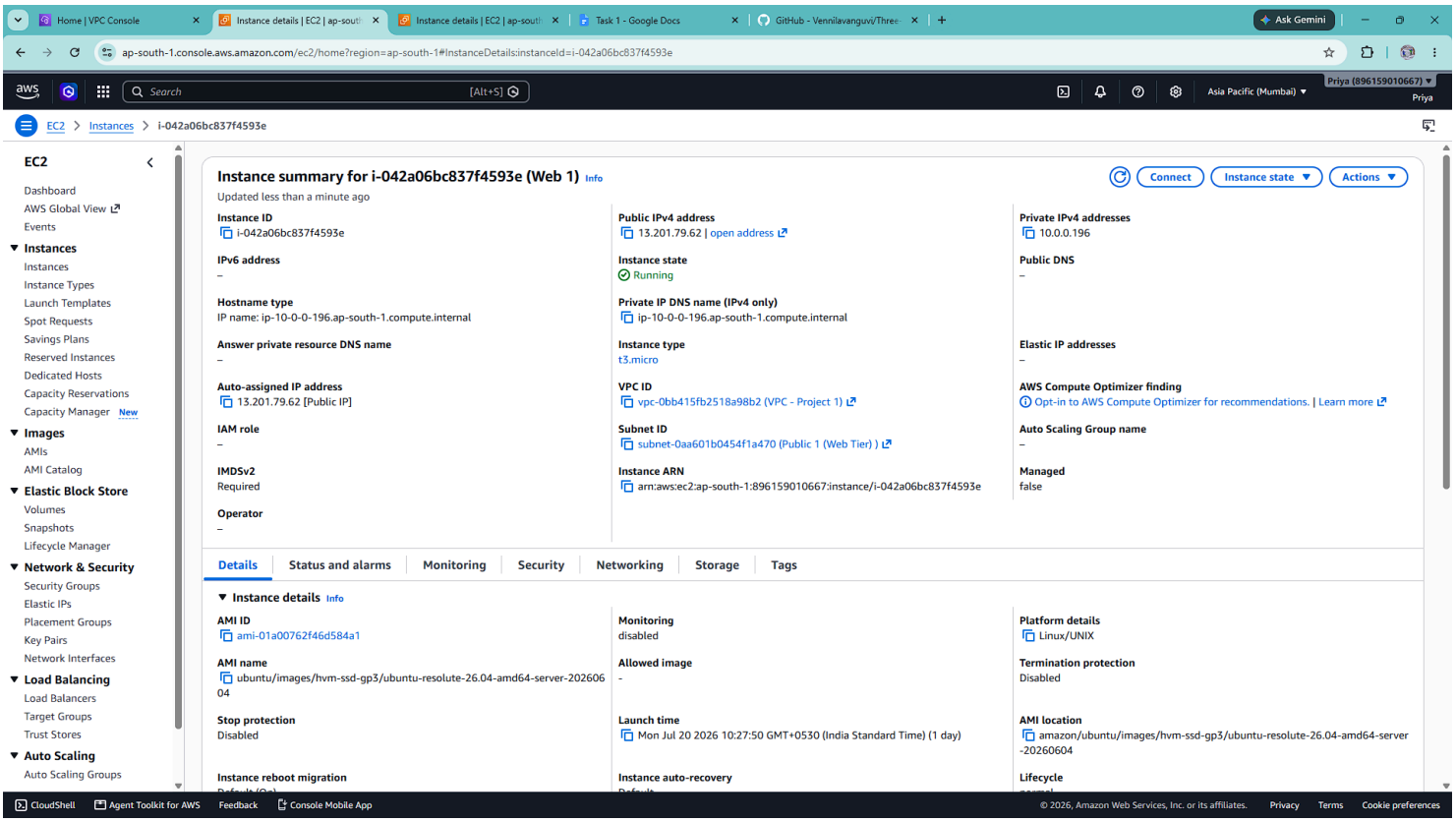


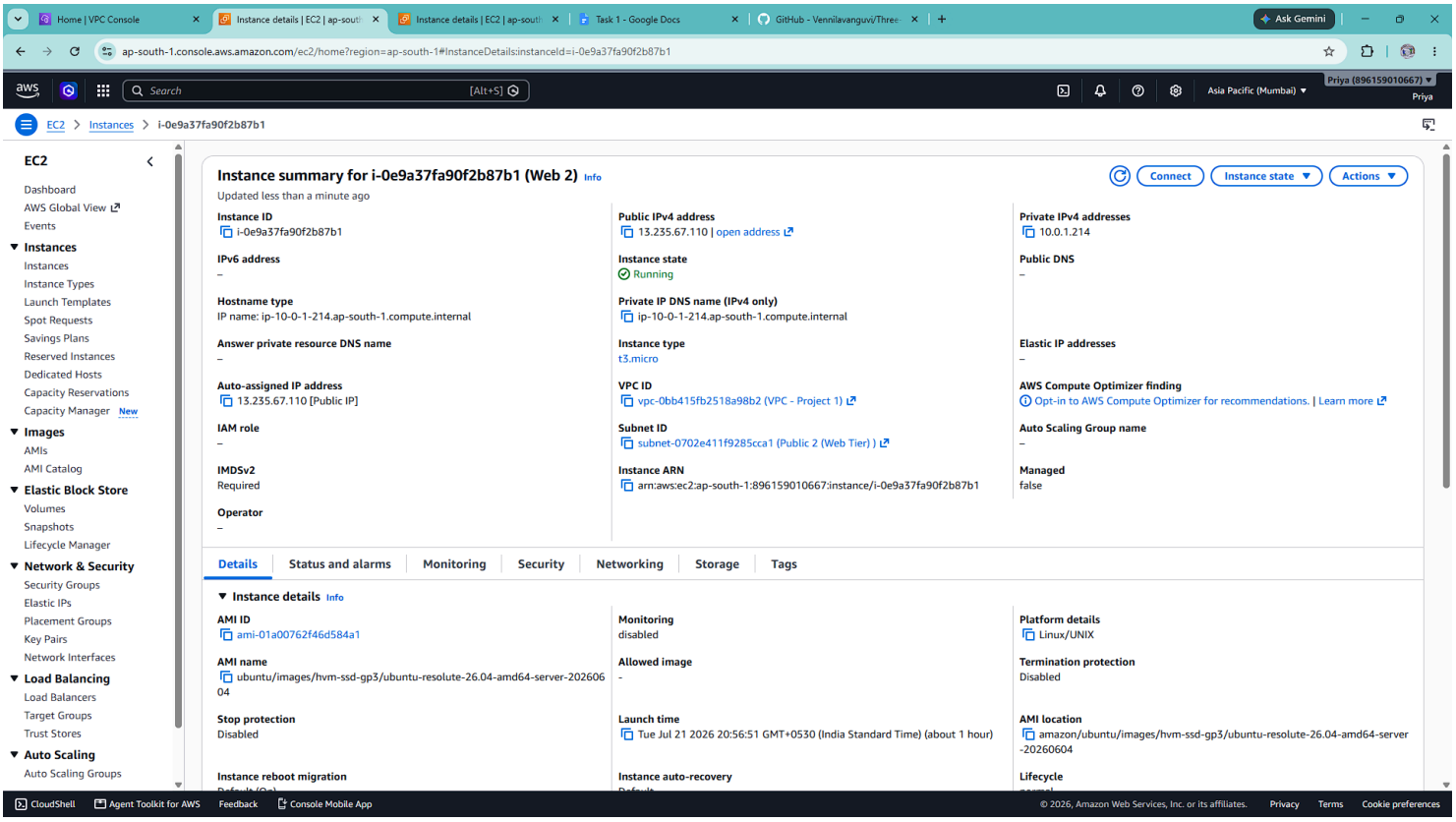
Private Database Route Table - Linked to Private 5 (Database Tier) and Private 6 (Database Tier) with a route to the NAT Gateway .



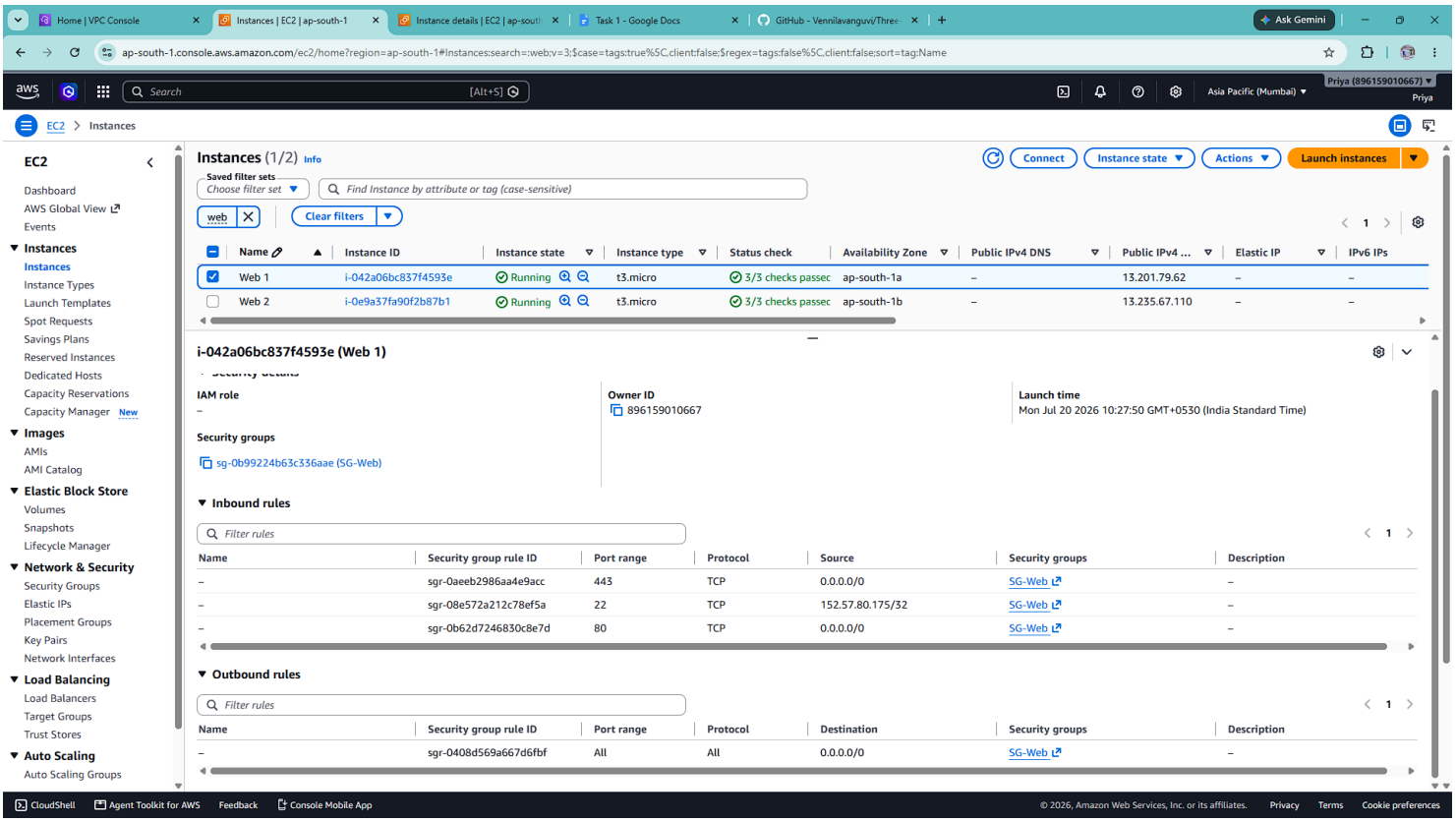
Task 2 - Deploy the Web Tier (Presentation Layer)

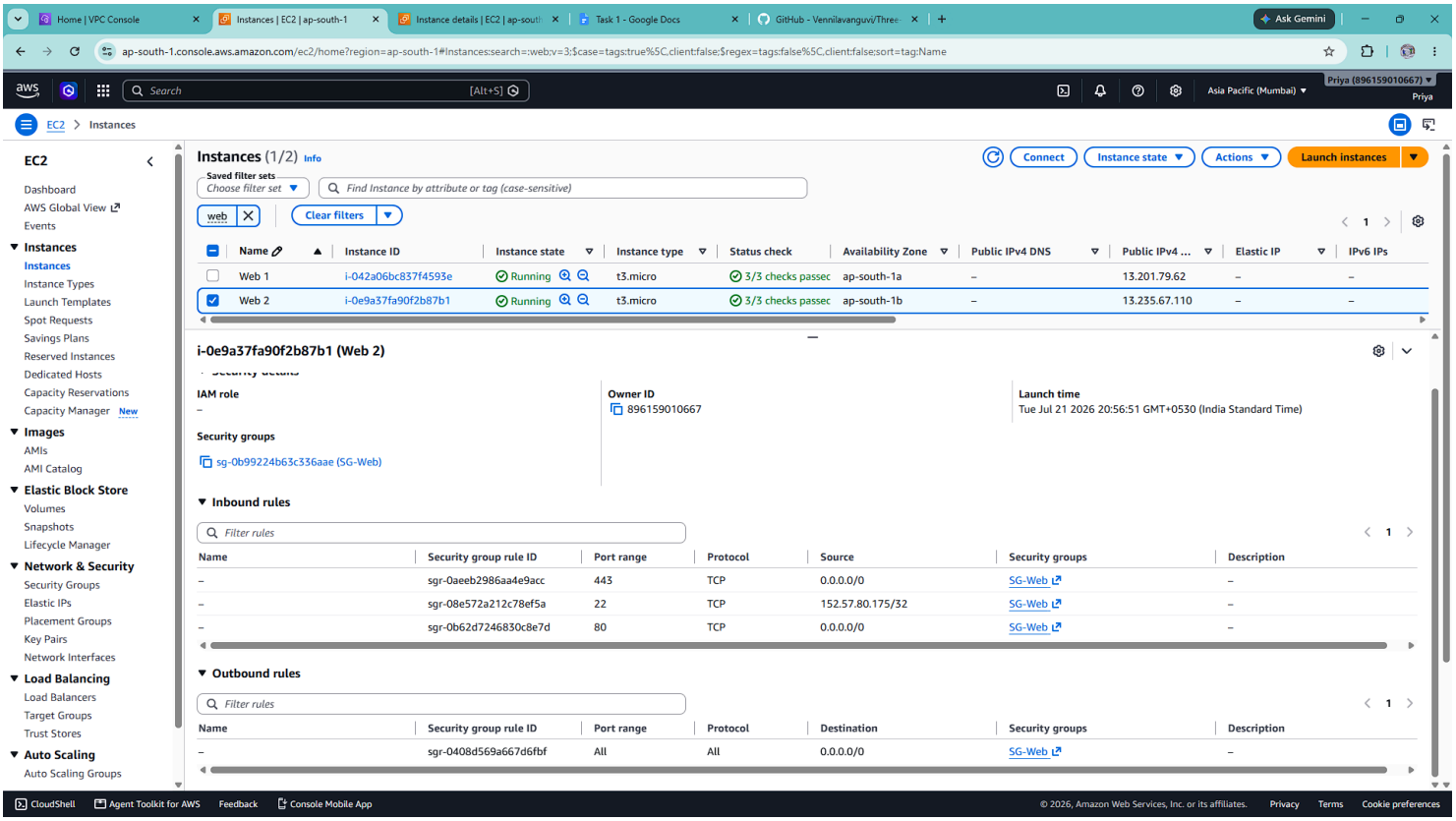
EC2 Instances : Launched two Ubuntu EC2 instances “Web 1” and “Web 2” in the Public Subnets across two Availability Zones (ap-south-1a and ap-south-1b) for high availability.



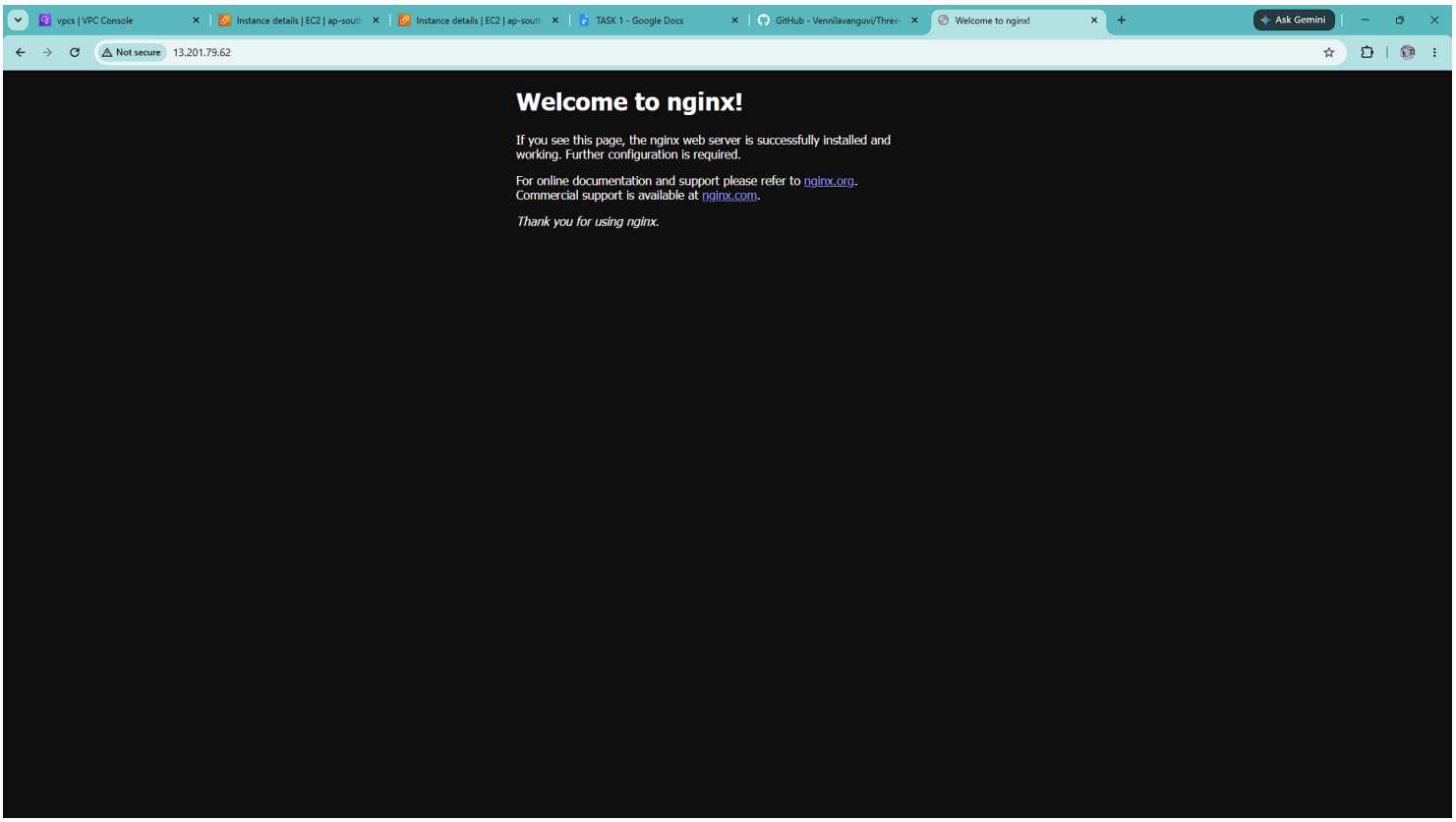


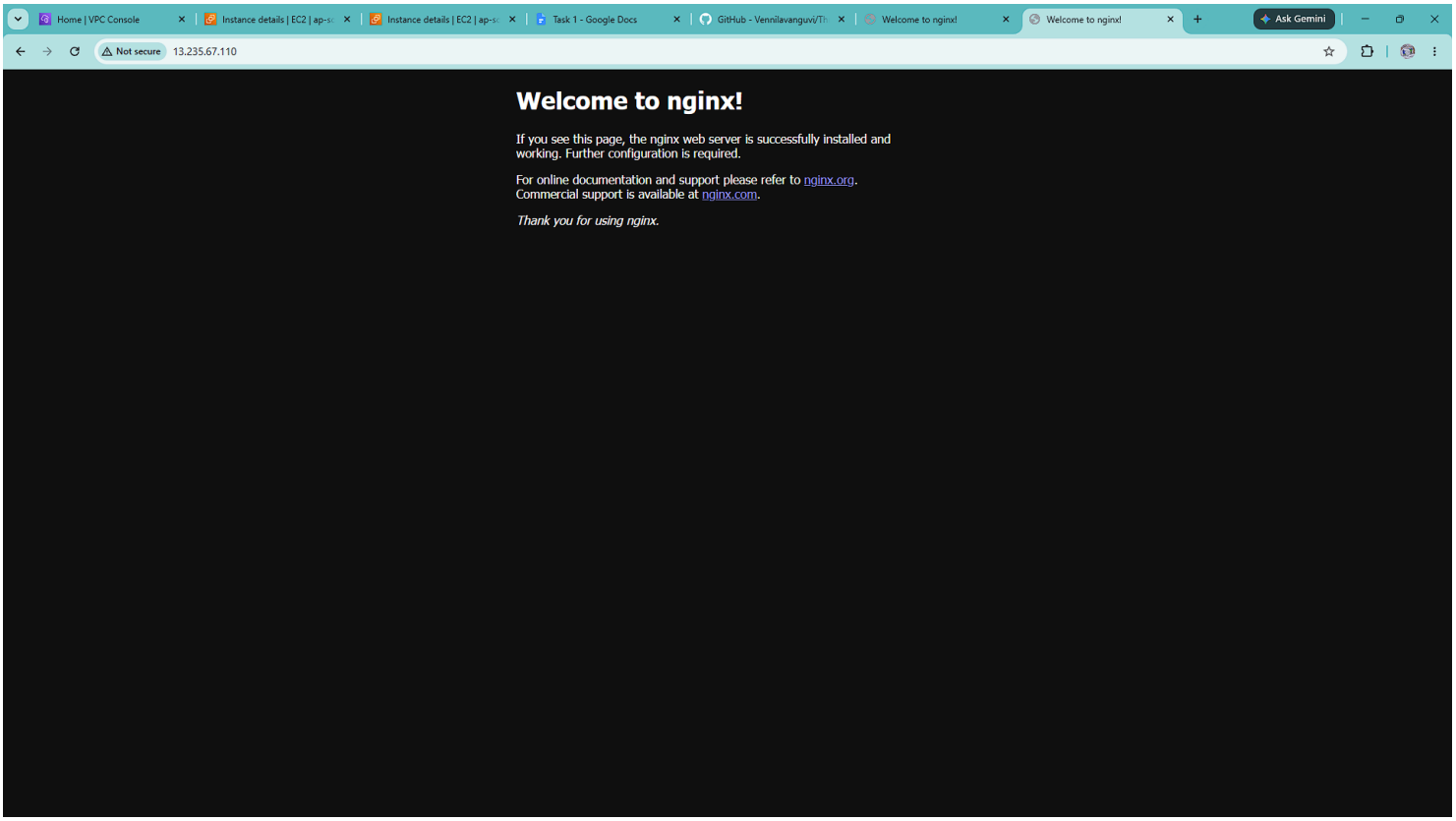
Security Group : Configured a Security Group named "SG-Web" with the following inbound rules - Port 80 (HTTP) open to all, Port 443 (HTTPS) open to all, and Port 22 (SSH) restricted to a specific IP (MyIP)



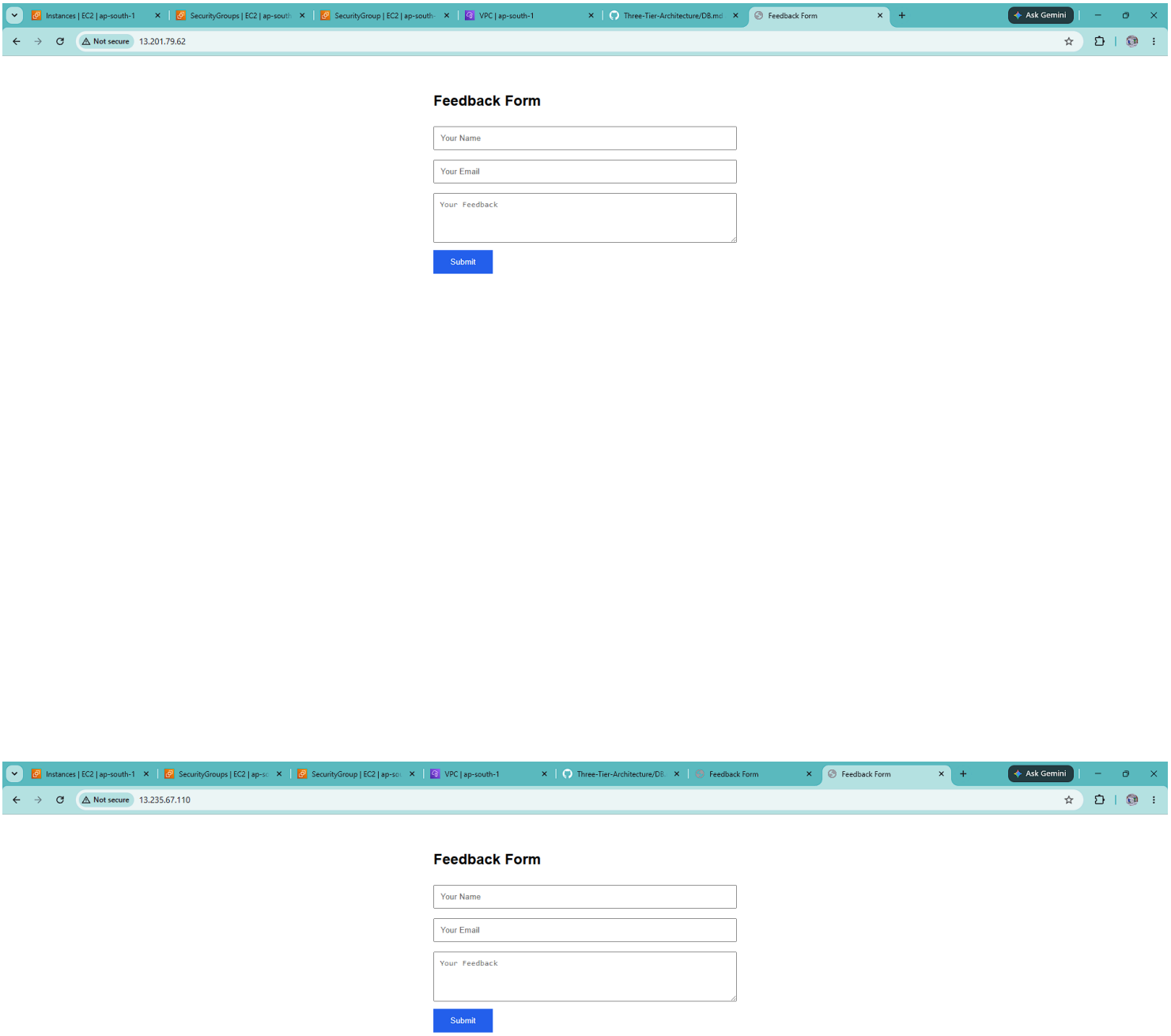


Nginx Installation : Connected to both instances and installed Nginx. Verified that Nginx is running successfully by accessing the public IP of both instances in the browser. Web 1 - 13.201.79.62 and Web 2 - 13.235.67.110



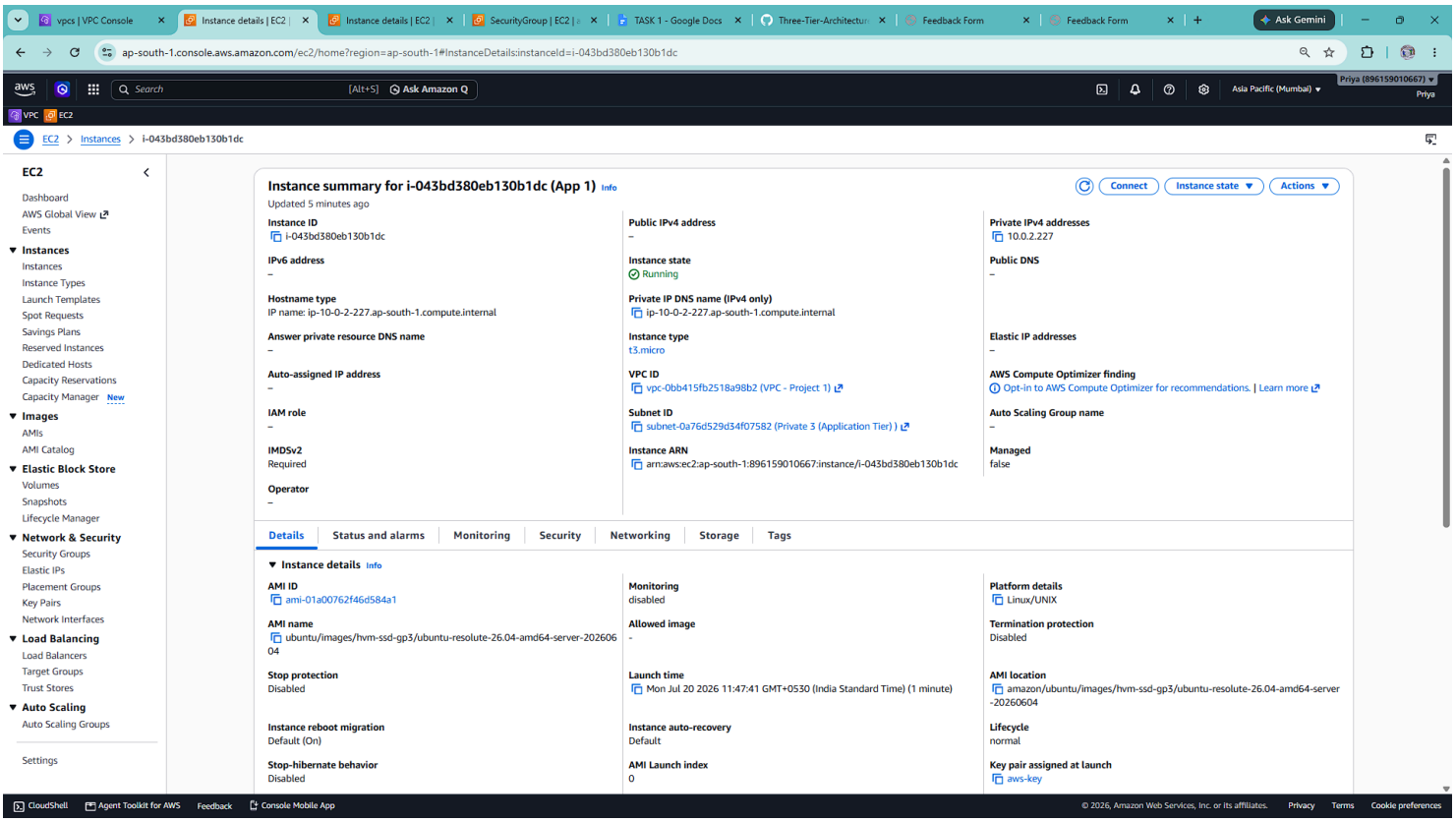


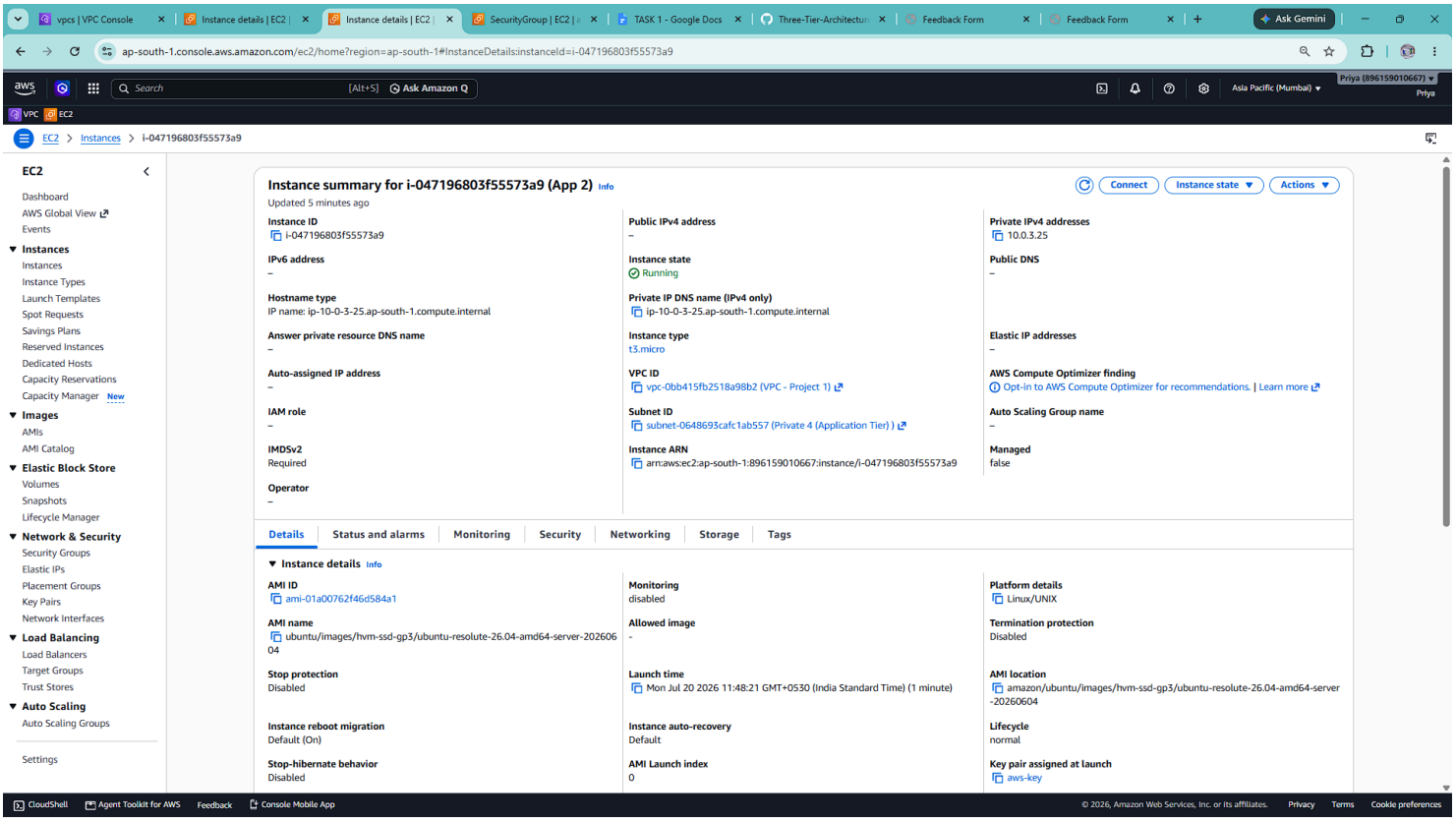
HTML Files : Copied the provided HTML files from the GitHub repository to both instances. The Feedback Form page is now accessible via the public IP of both Web instances, confirming the web tier is fully deployed and working.



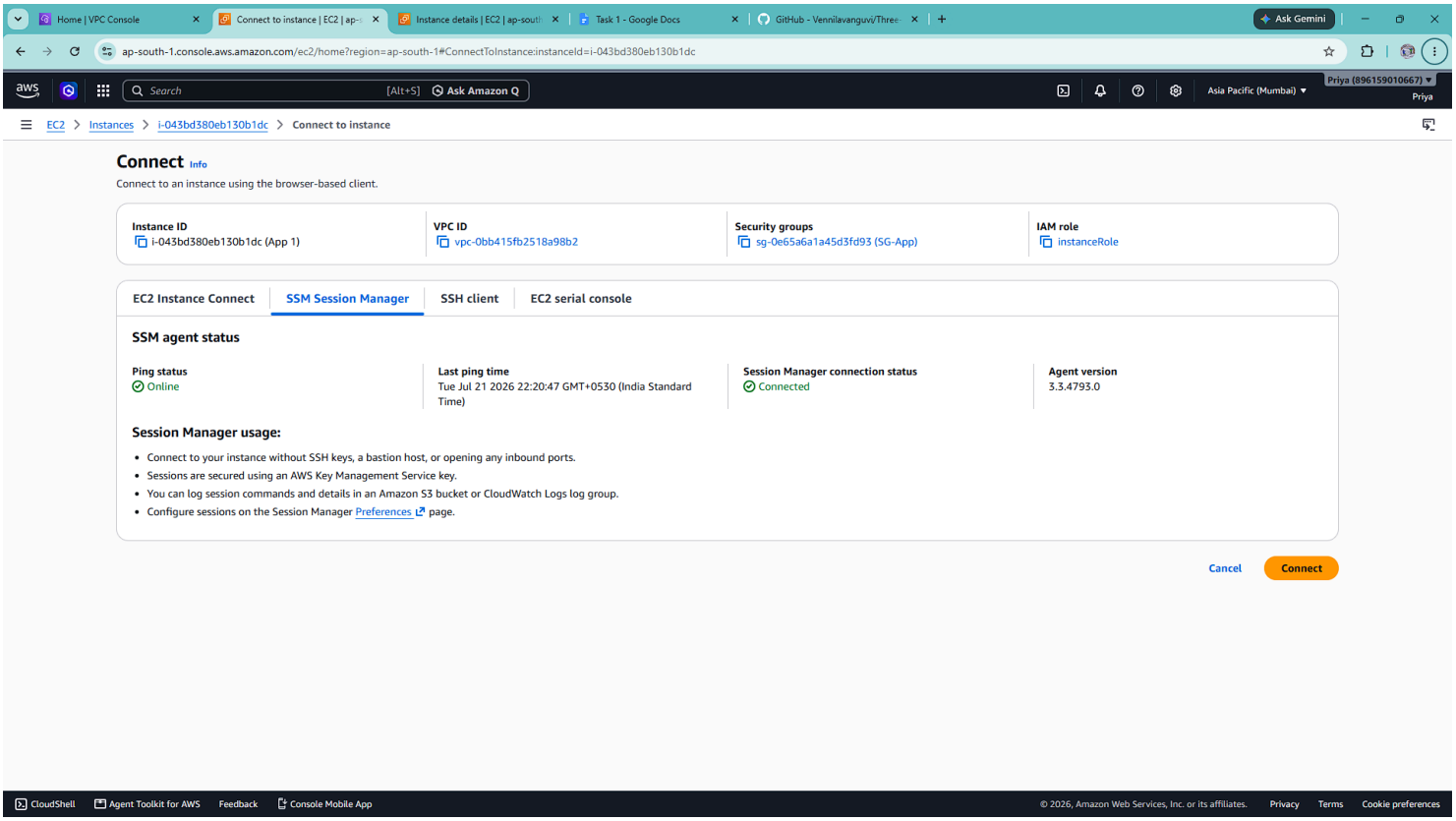
Task 3 - Deploy the Application Tier (Backend API)

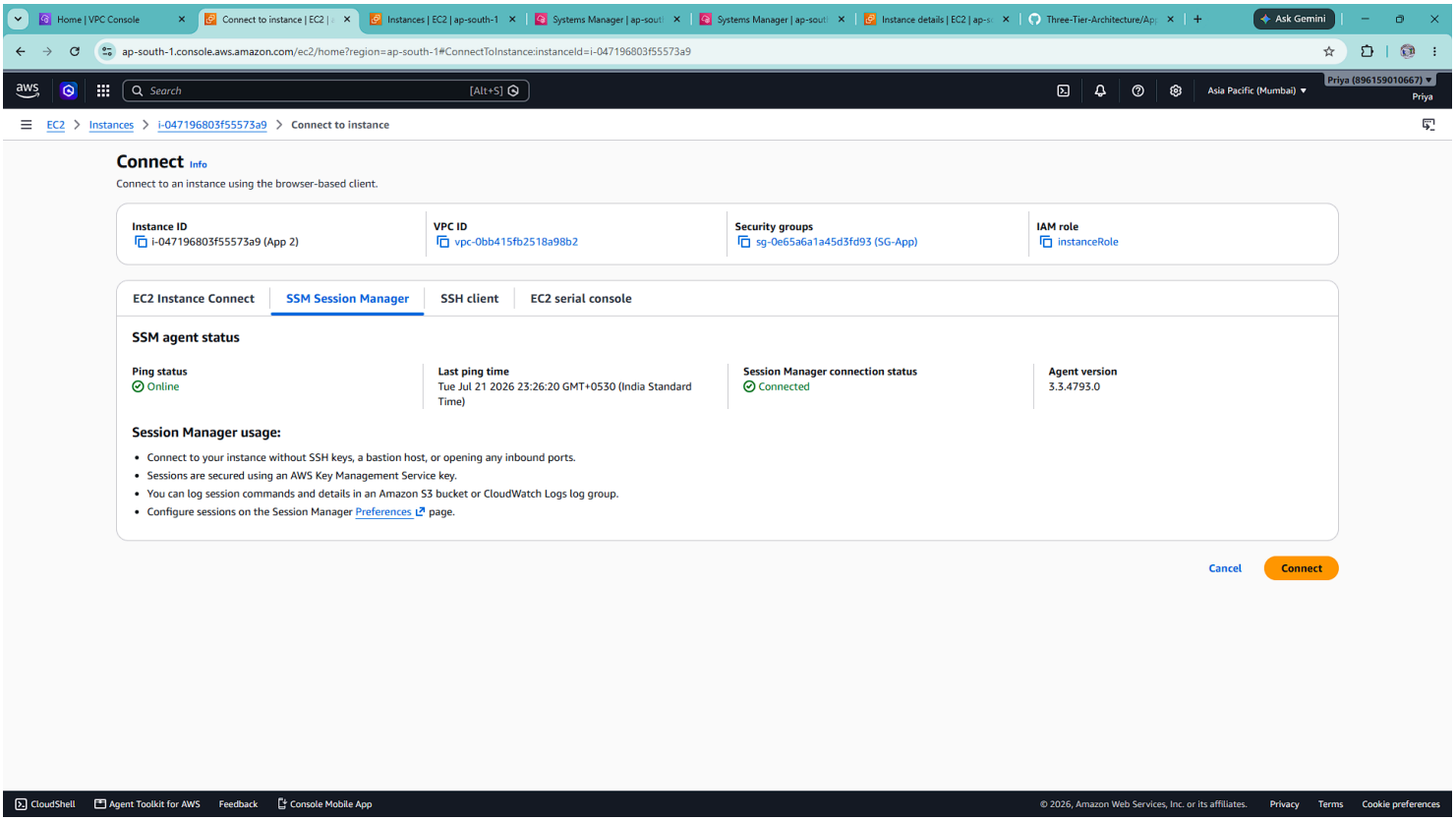
EC2 Instances : Launched two Ubuntu EC2 instances “App 1” and “App 2” in the Private Application Subnets across two Availability Zones (ap-south-1a and ap-south-1b). Two instances with no public IP, keeping them fully private and secure.



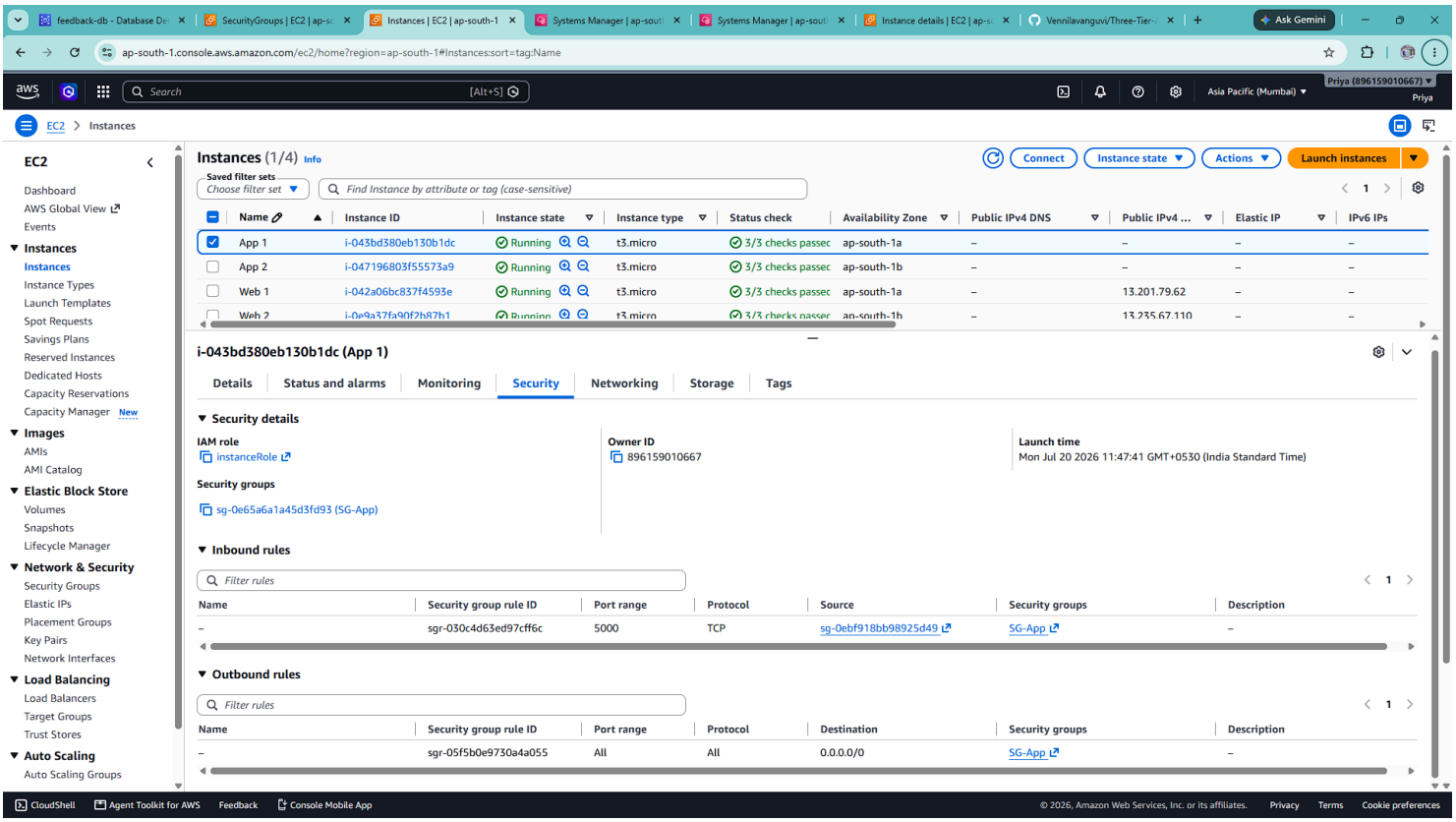


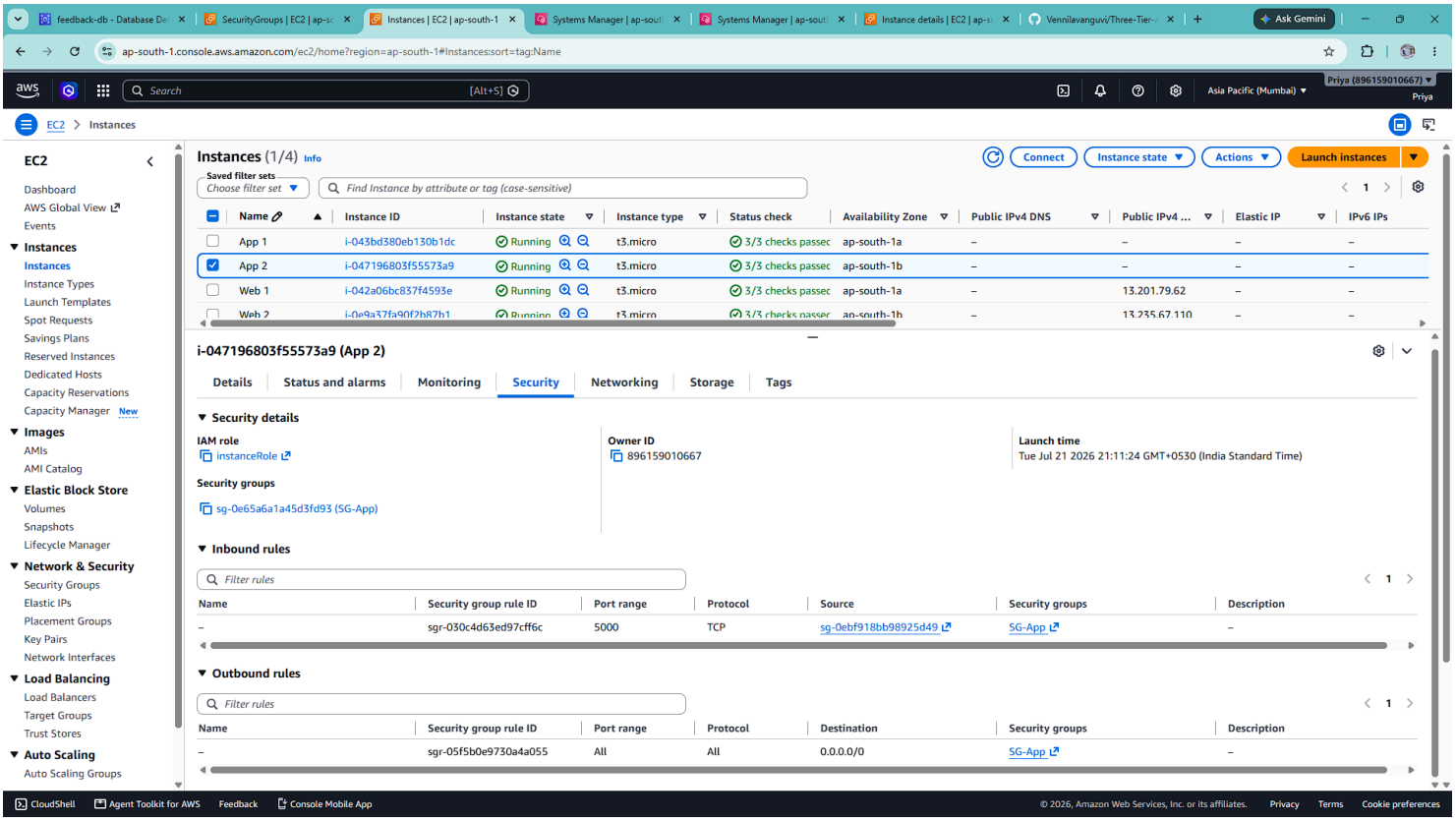
Session Manager Access : Since the Application EC2 instances are deployed in private subnets with no public IP addresses, SSM Session Manager was used to securely connect to the instances.



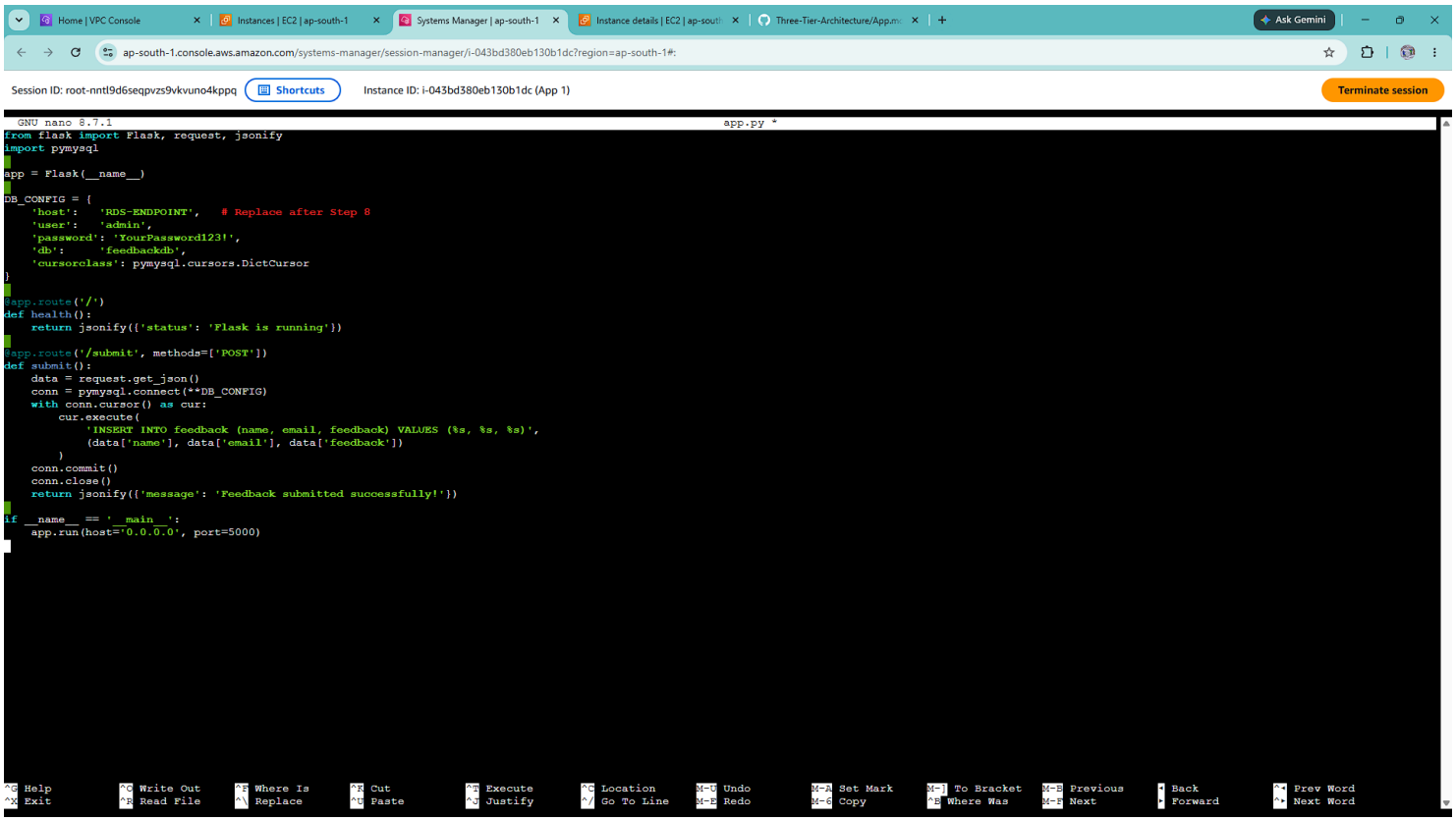


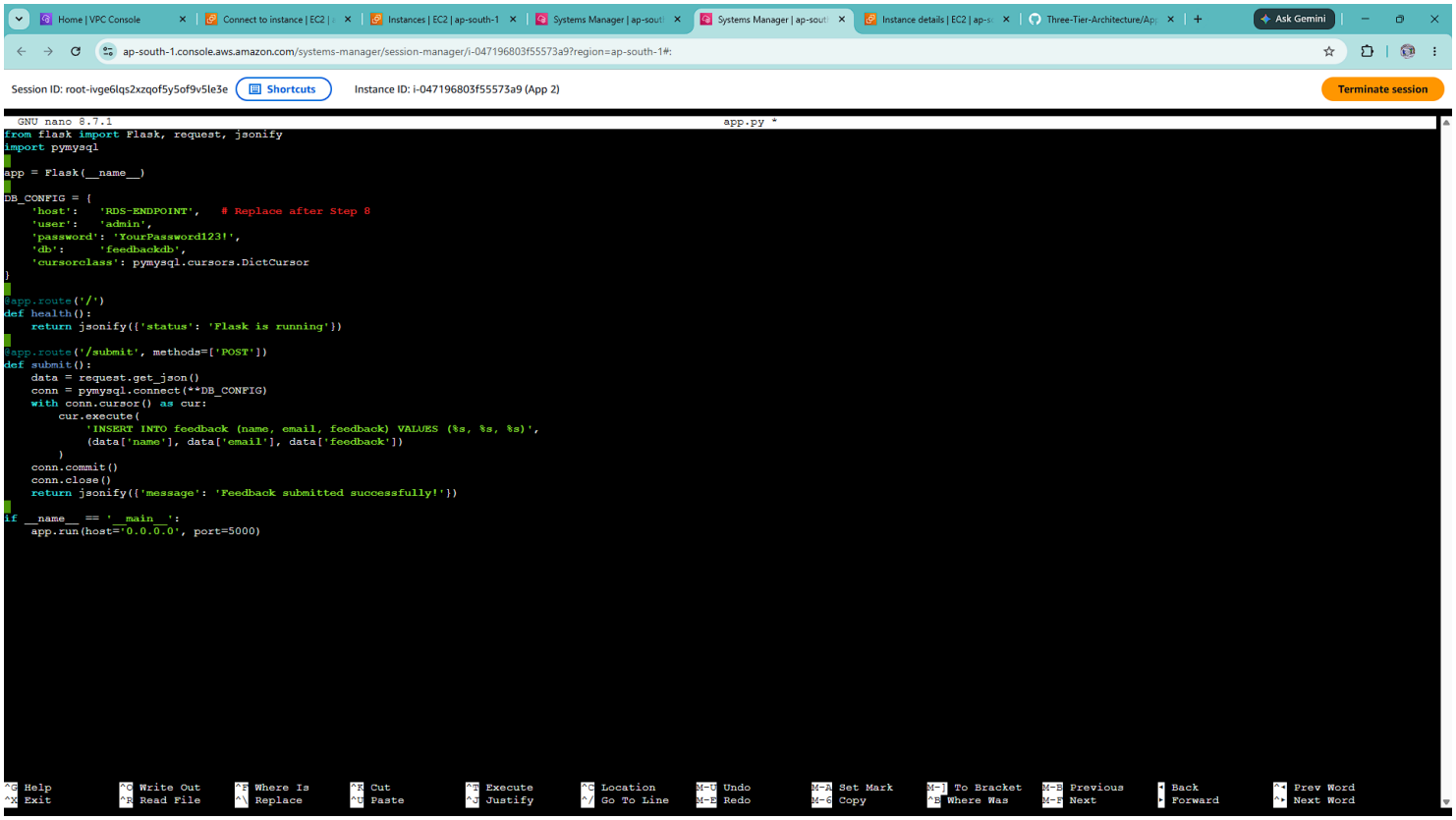
Security Group : Configured Security Group "SG-App" with inbound rule - Port 5000 allowed only from the Web tier Security Group (SG-Web). This ensures only the Web tier can reach the Application tier.





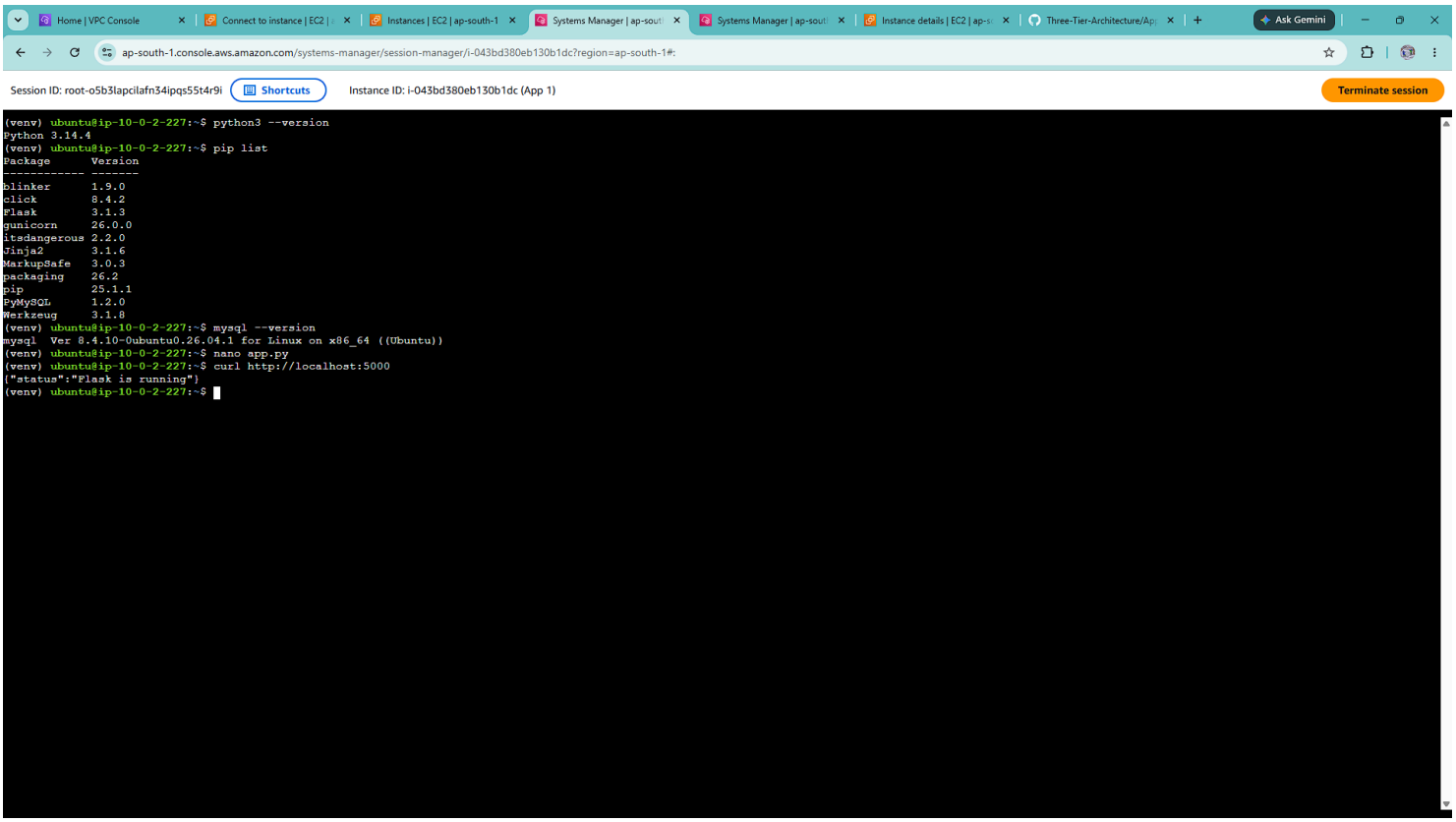
app.py File : Connected to both Application EC2 instances using SSM Session Manager. The app.py file was copied from the GitHub repository. And pasted it into both instances (App 1 - 10.0.2.227 and App 2 - 10.0.3.25)

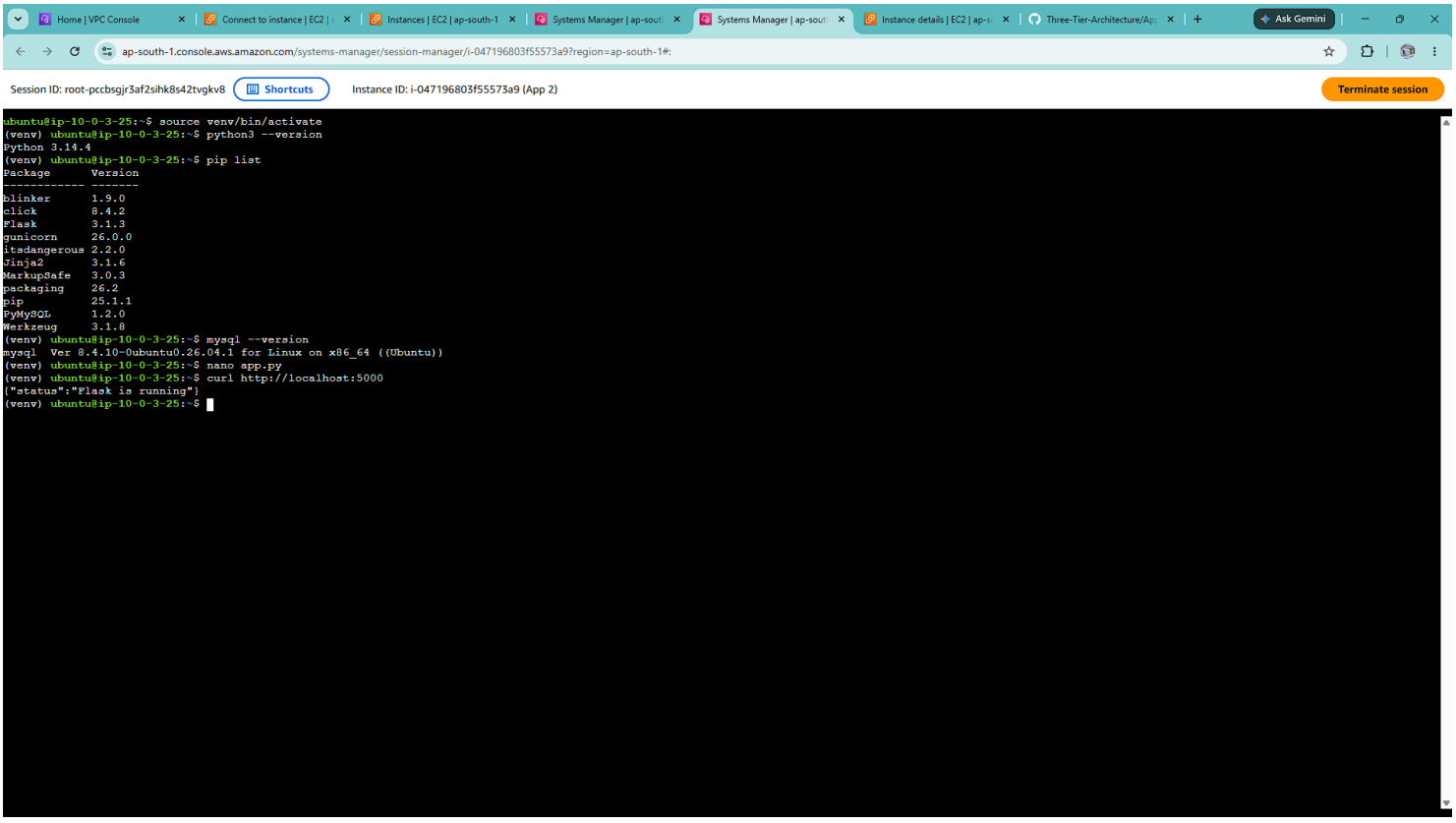




Python 3, Flask and Dependencies : Installed Python, activated a virtual environment, and installed all required packages including Flask, PyMySQL and Gunicorn, using the commands provided in the GitHub repository.

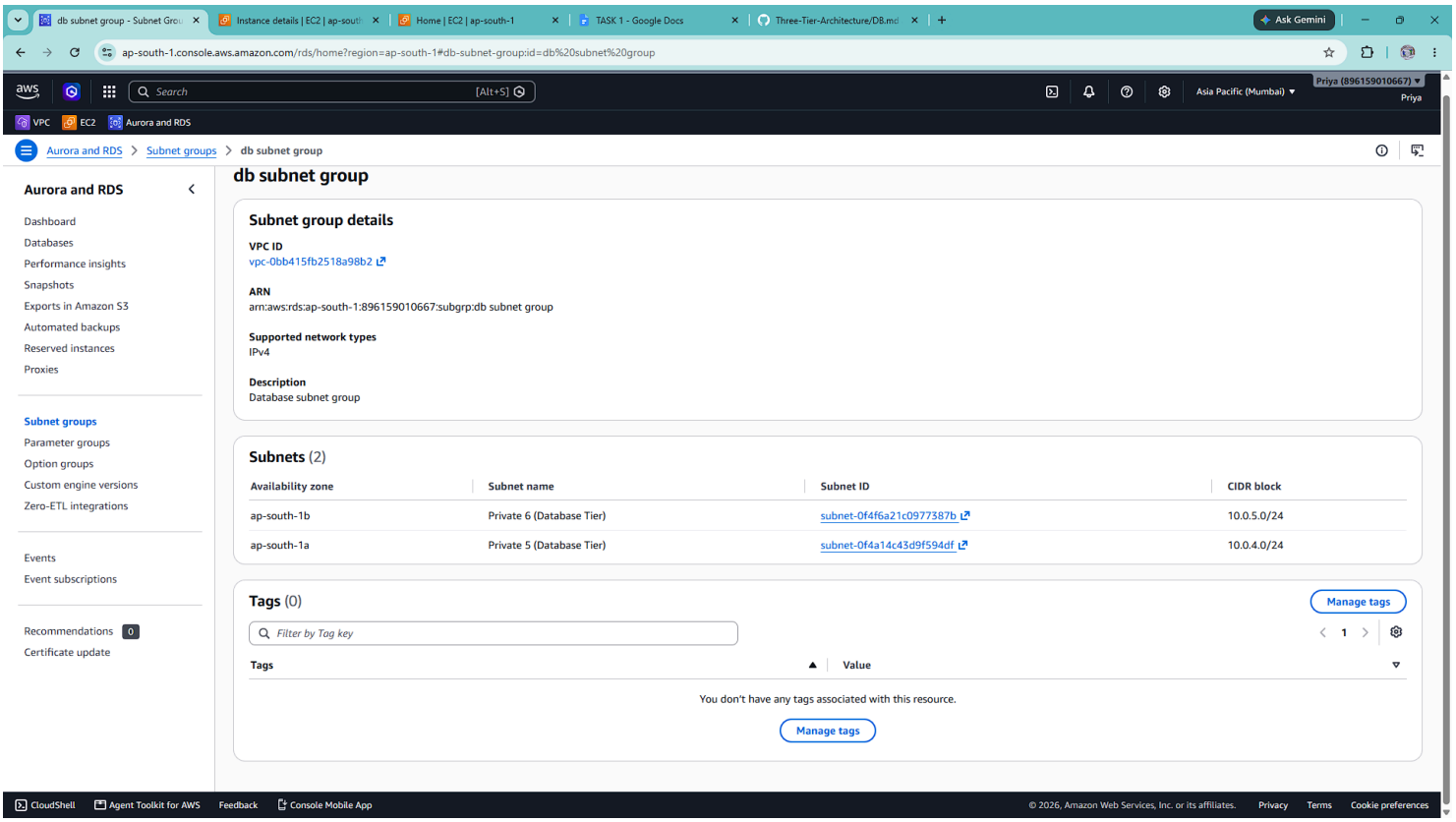
Flask Application Verified : Ran curl localhost:5000 on both instances. The response {"status":"Flask is running"} confirms the Flask application is running successfully on Port 5000 on both the instances.



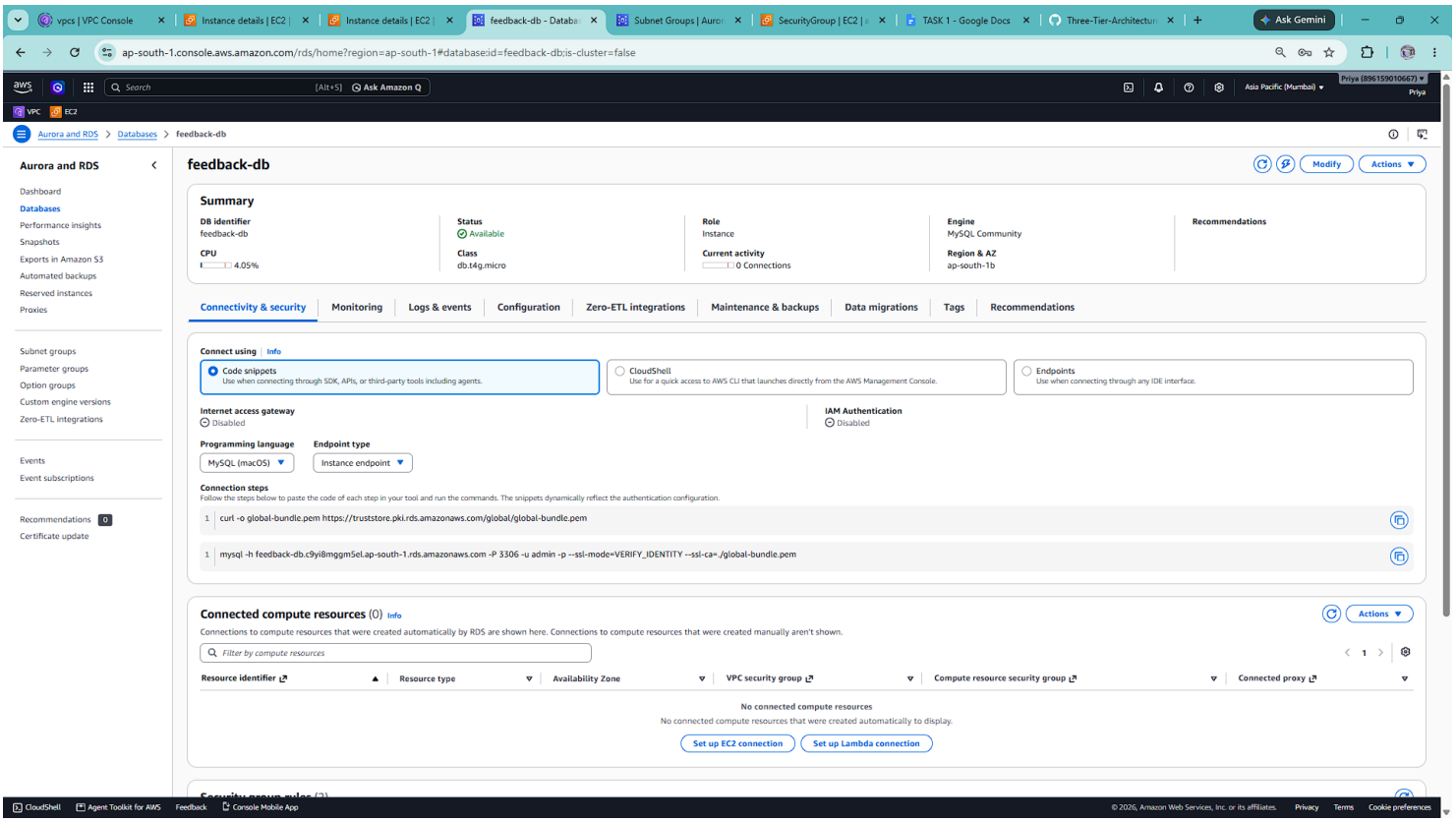


Task 4 - Deploy and Configure the Database Tier

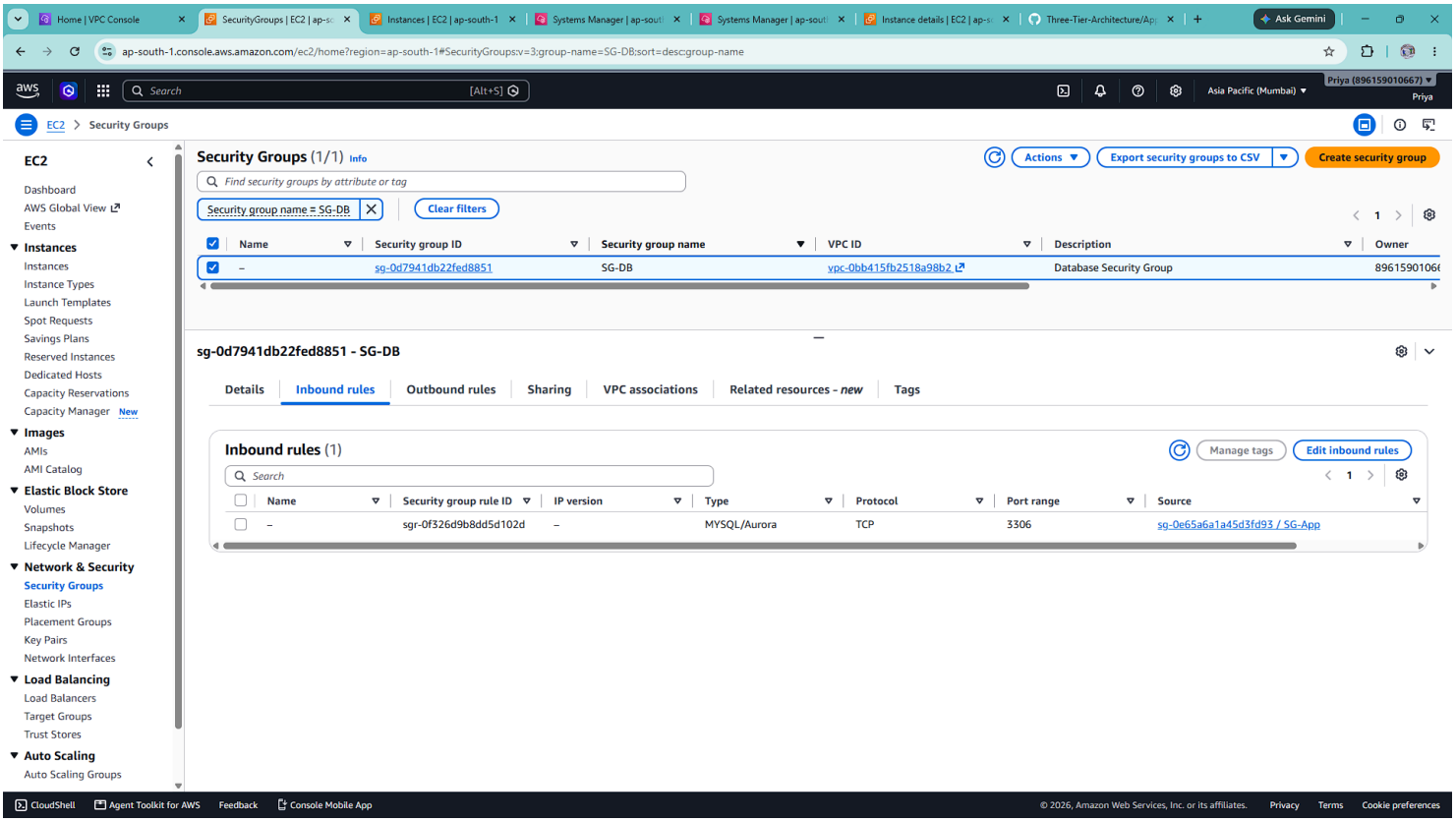
DB Subnet Group : Created a DB Subnet Group for the two private Database subnets across both Availability Zones - Private 5 (Database Tier) in ap-south-1a and Private 6 (Database Tier) in ap-south-1b. This ensures the RDS instance is fully isolated in the private database tier.

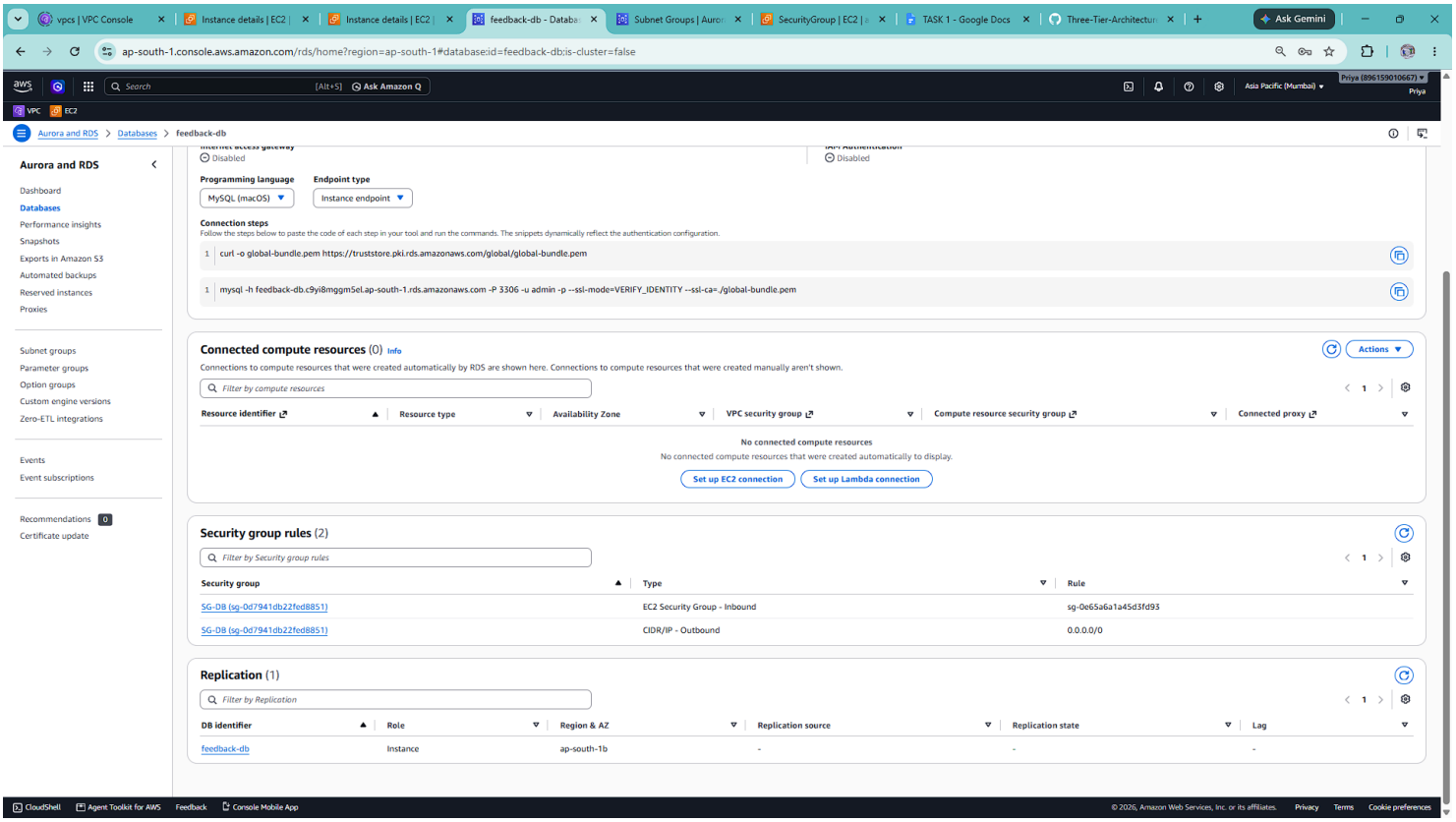


RDS MySQL : Created an Amazon RDS MySQL instance named "feedback-db", deployed in ap-south-1b. The instance is not publicly accessible, ensuring the database is only reachable from within the VPC.

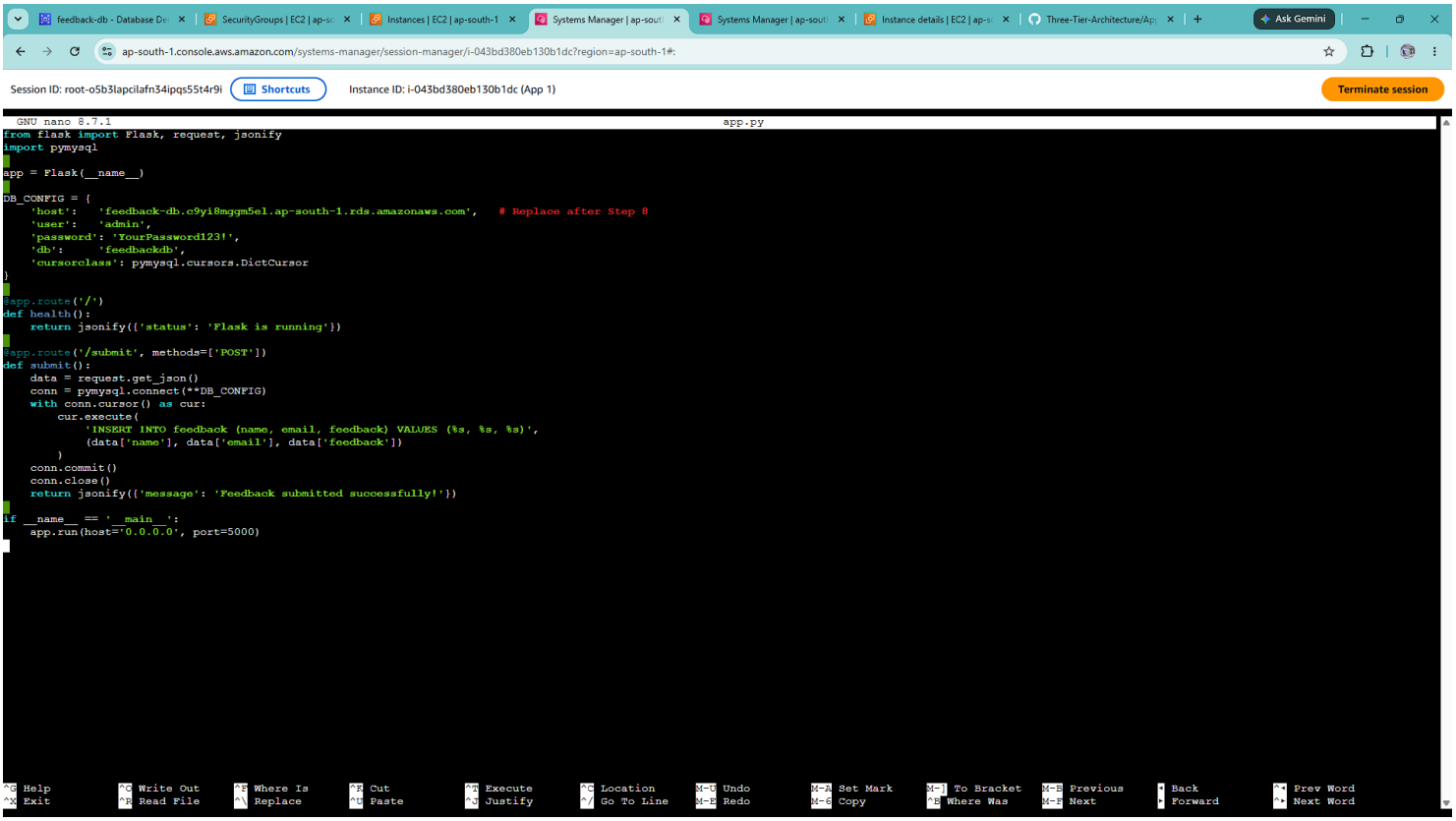


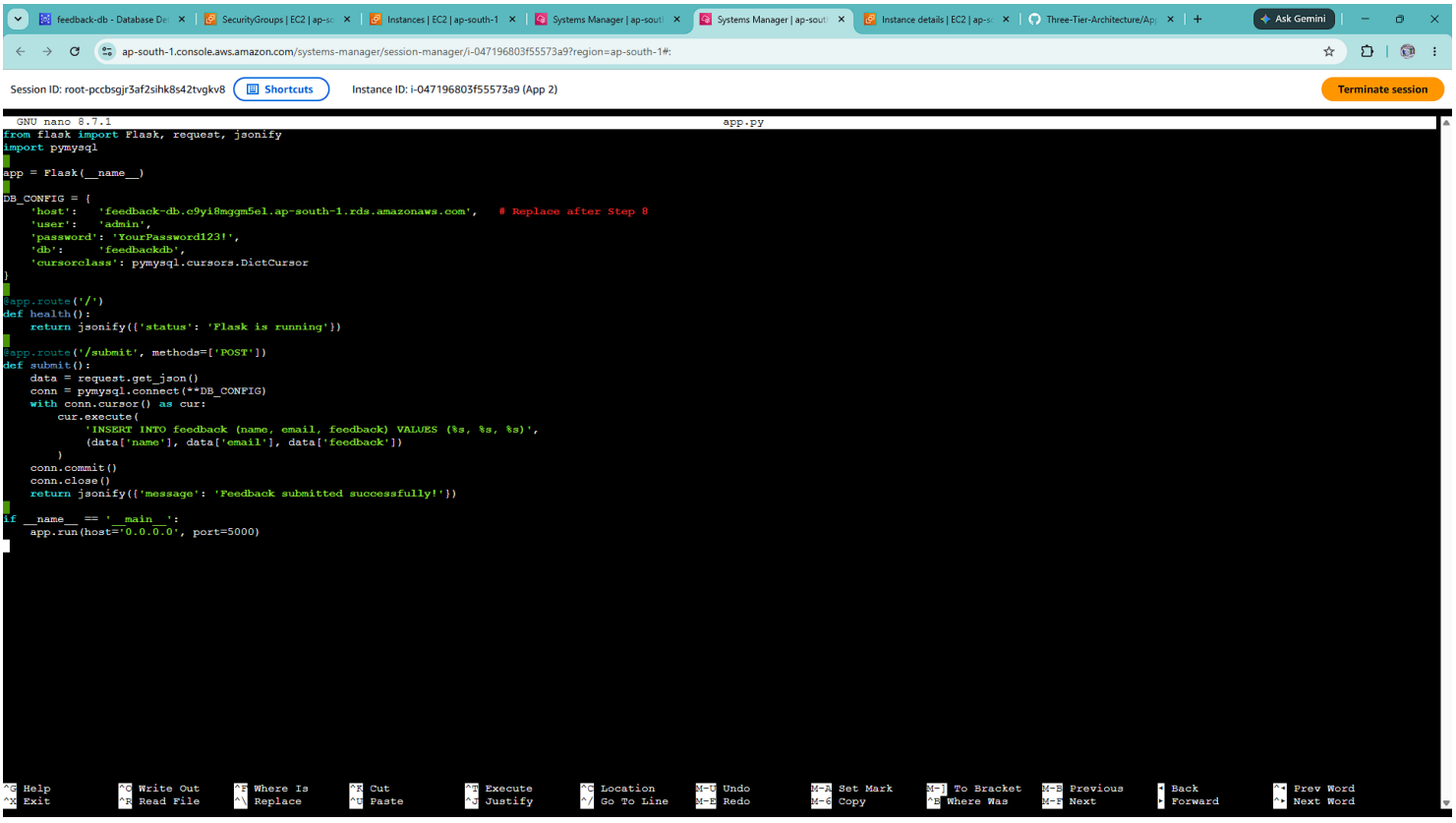
Security Group & Connectivity : Configured Security Group "SG-DB" with inbound access allowed only from the Application tier Security Group (SG-App). This means only the App EC2 instances can connect to the database on Port 3306.



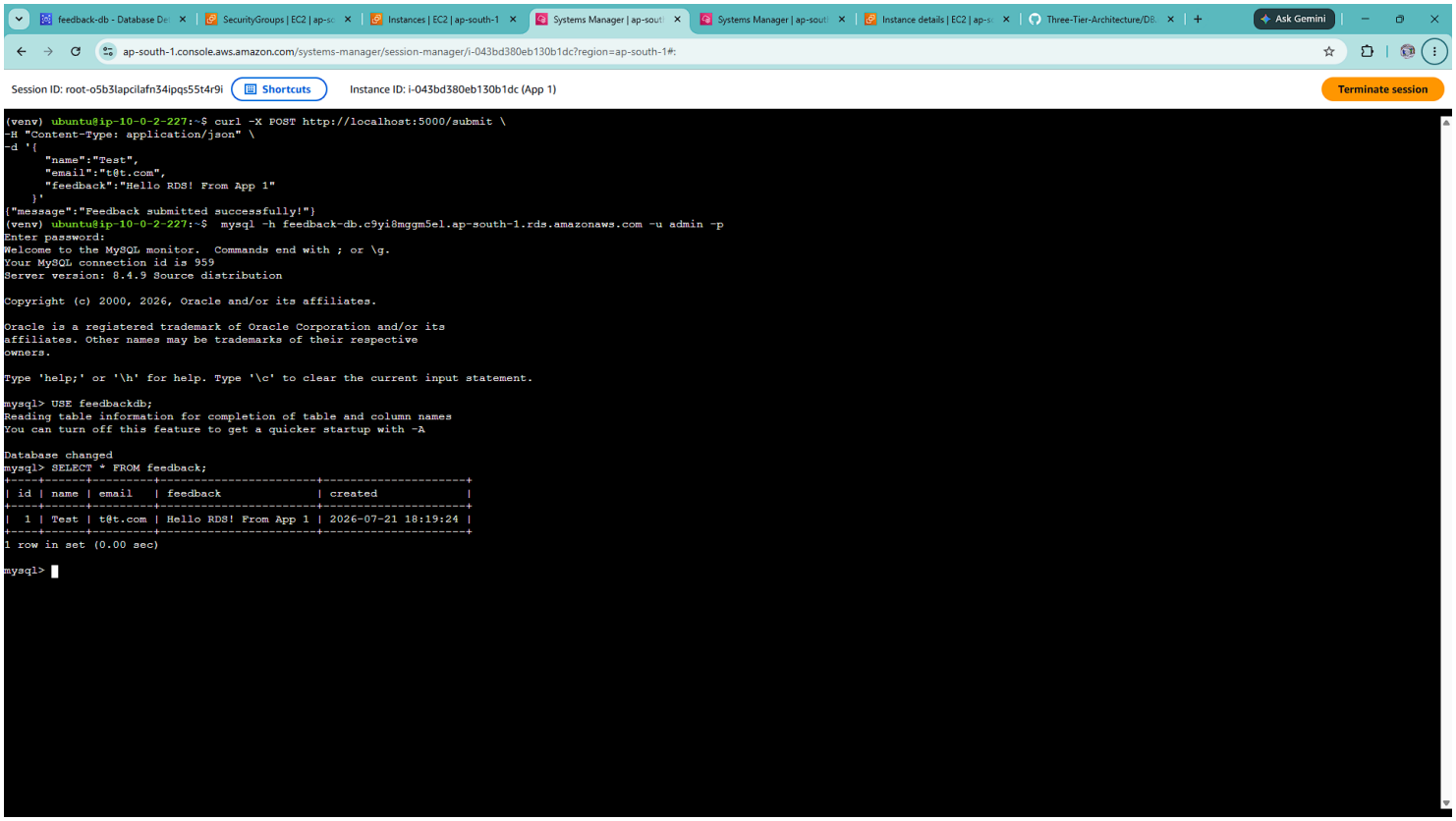


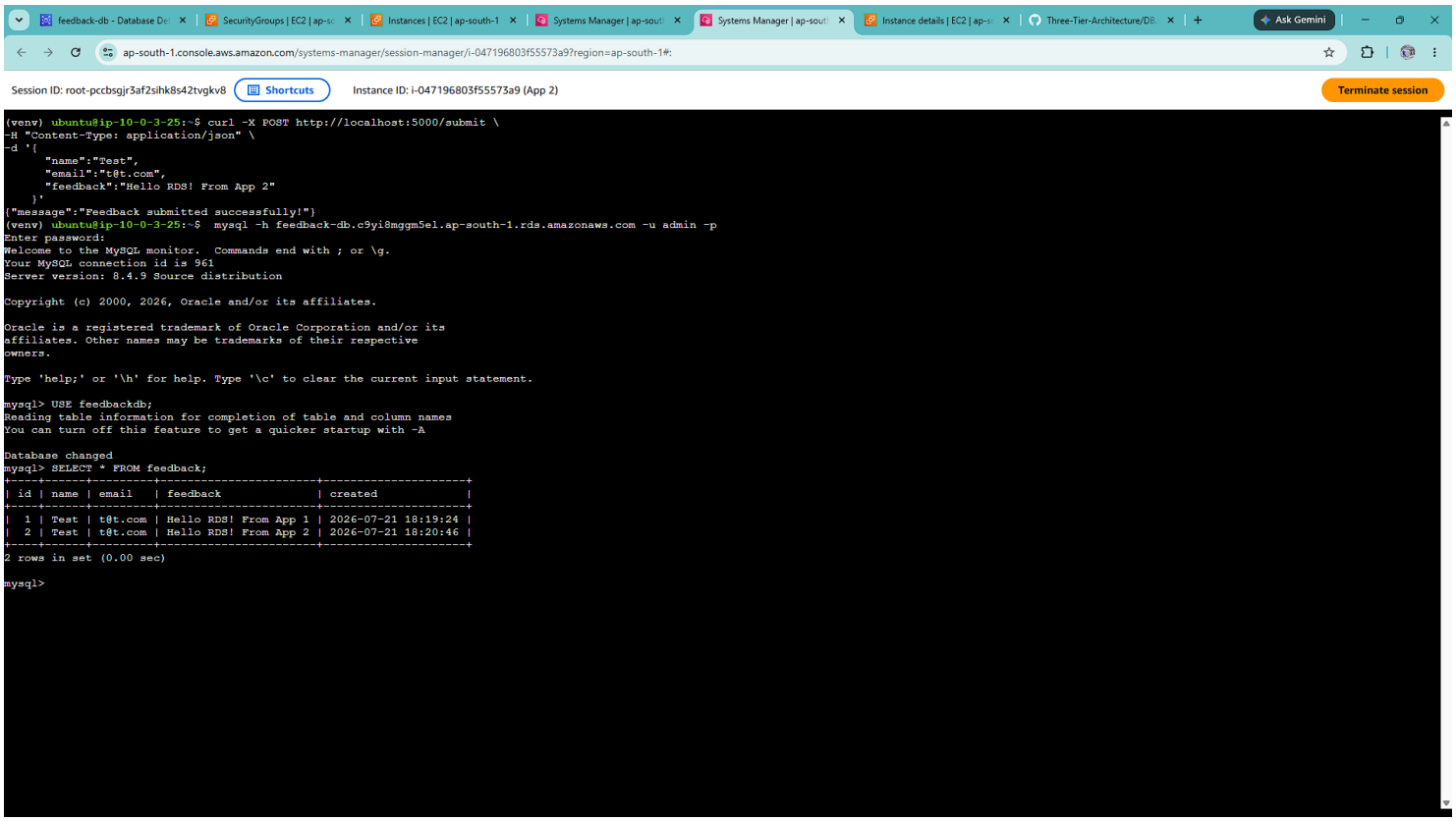
Database Connection from App EC2 : Connected to the RDS instance from the App EC2 terminal using the RDS endpoint. Updated the app.py file on both application instances with the actual RDS endpoint so Flask can connect to the database.





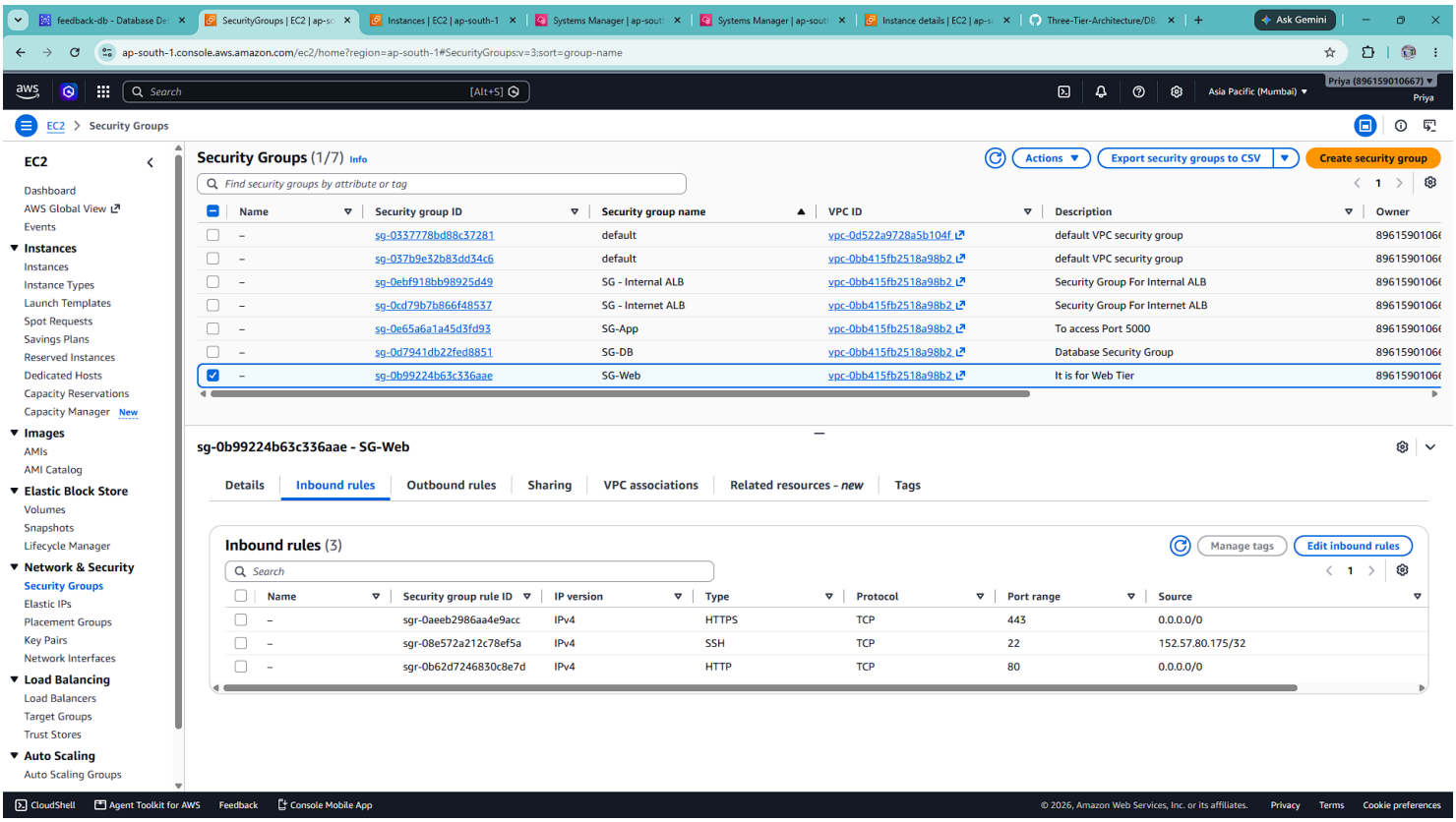
SQL Commands & Data Verification : Executed SQL commands to create the feedbackdb database and feedback table. Submitted test feedback using App 1 ("Hello RDS! From App 1") and also from App 2 ("Hello RDS! From App 2") using curl POST requests. Ran SELECT * FROM feedback to confirm both records are stored successfully in RDS.

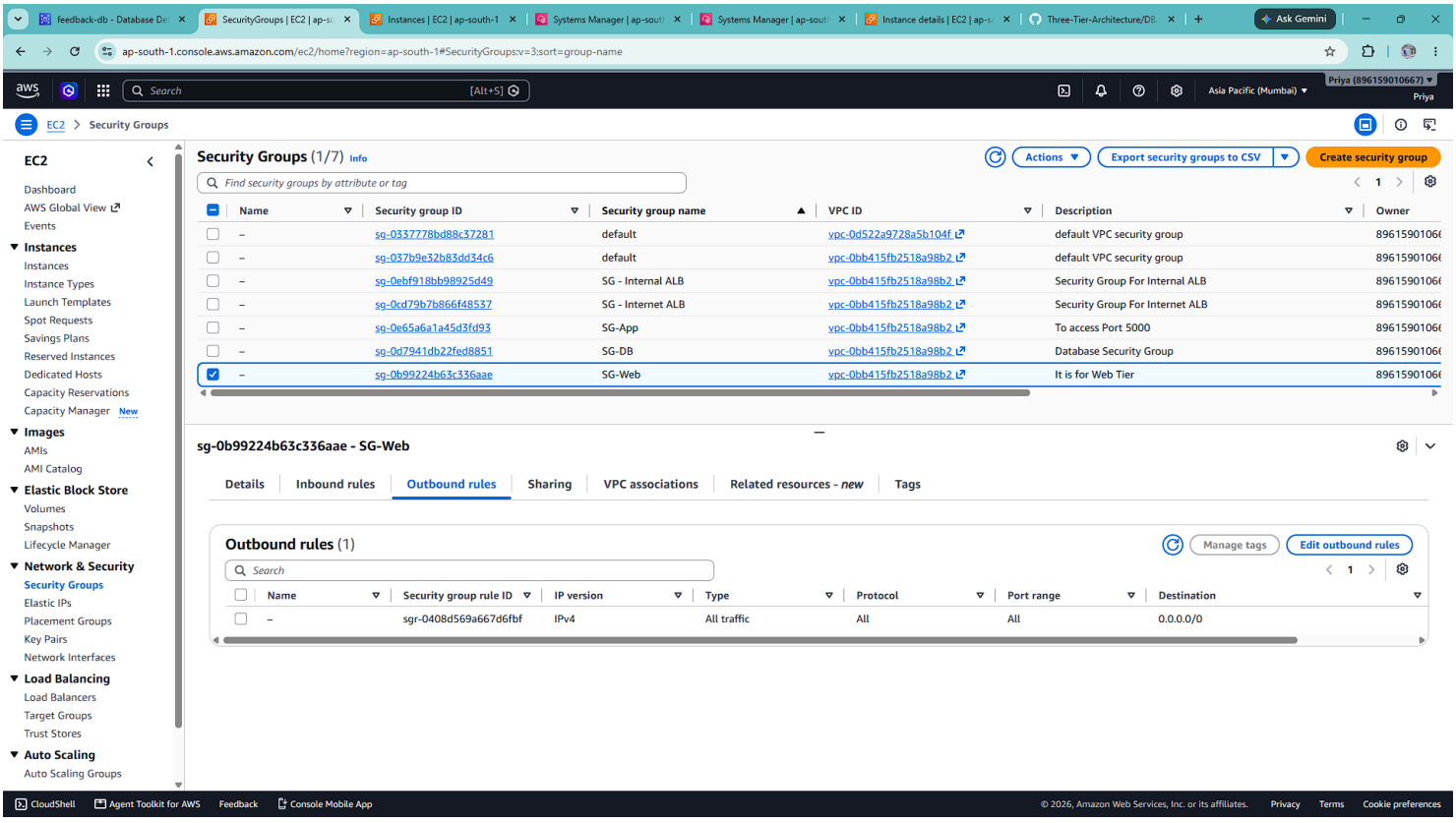




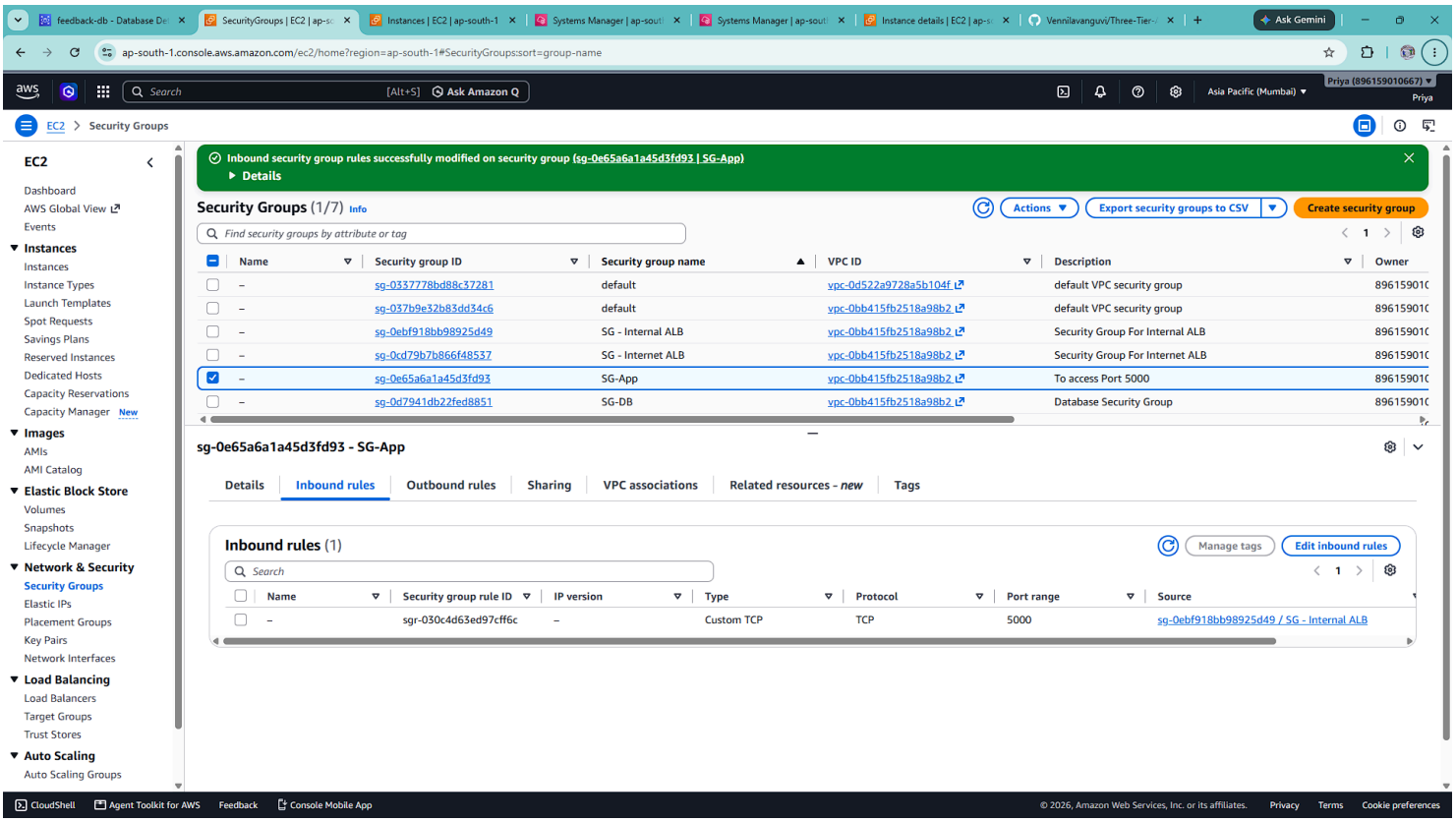
Task 5 - Implement Network and Instance Security

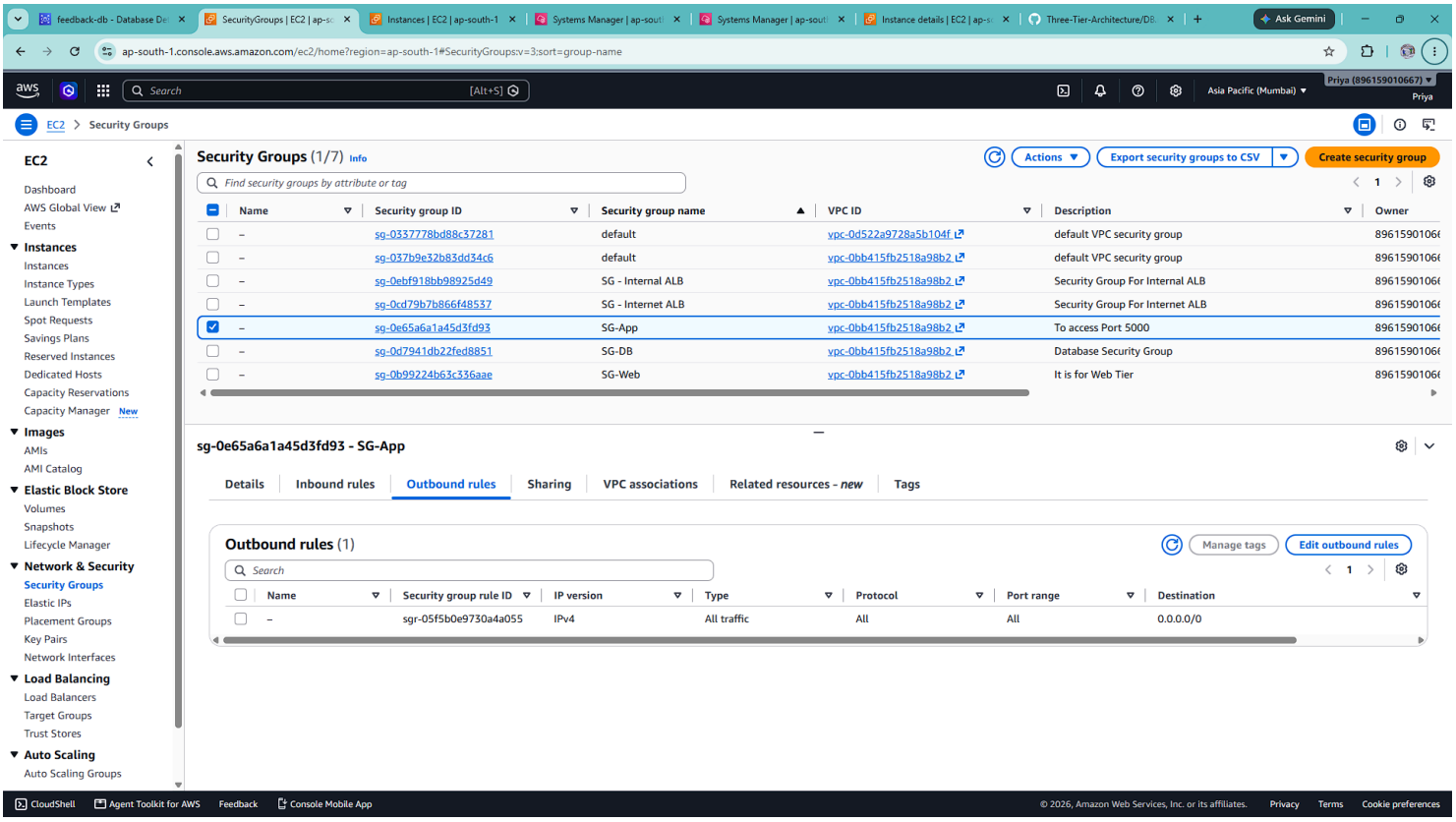
Web Tier : Configured a Security Group named "SG-Web" for the Web tier EC2 instances. Inbound rules allow Port 80 (HTTP) and Port 443 (HTTPS) open to all (0.0.0.0/0) for public access, and Port 22 (SSH) restricted to a specific IP (MyIP) for secure admin access.



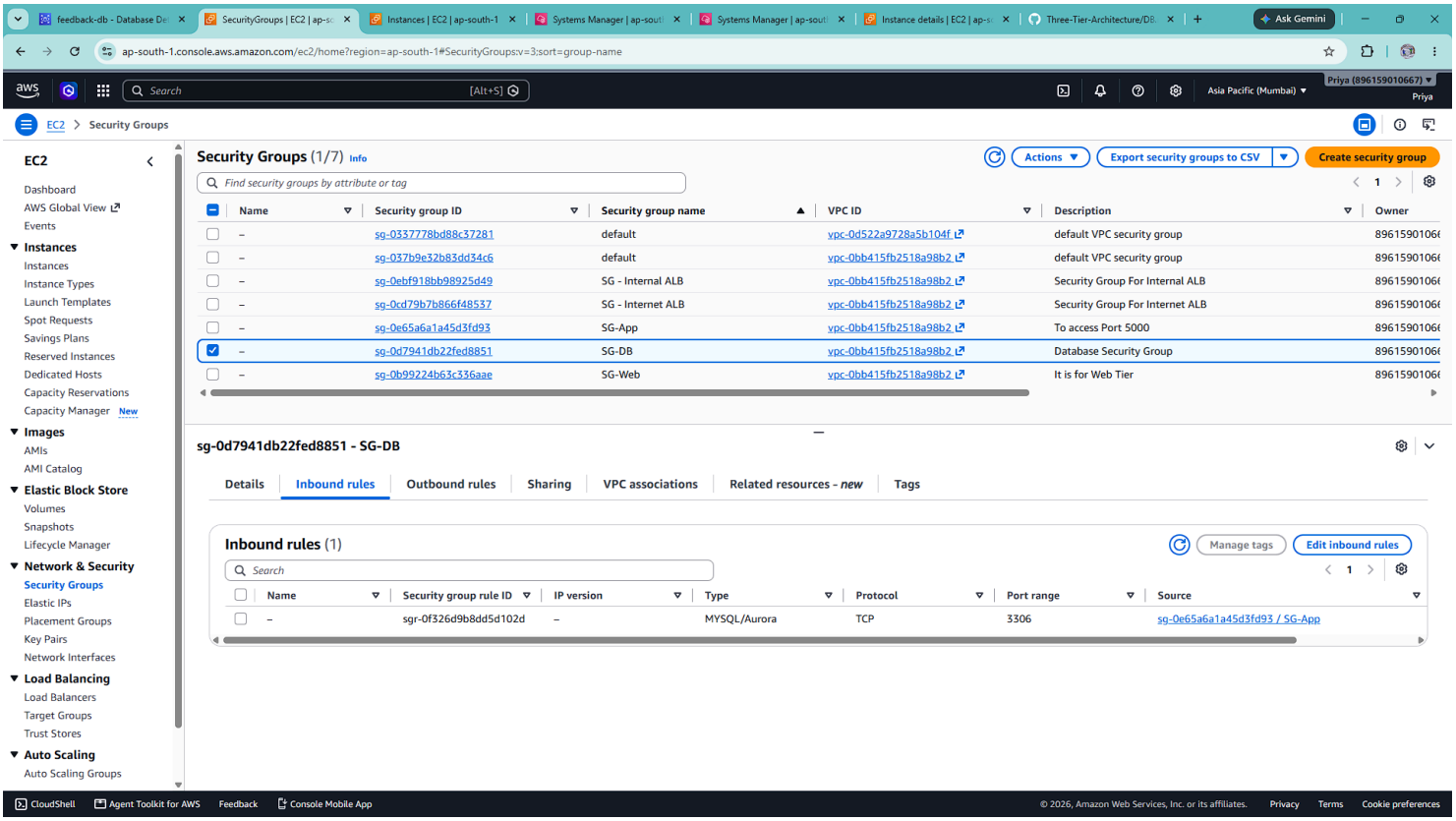


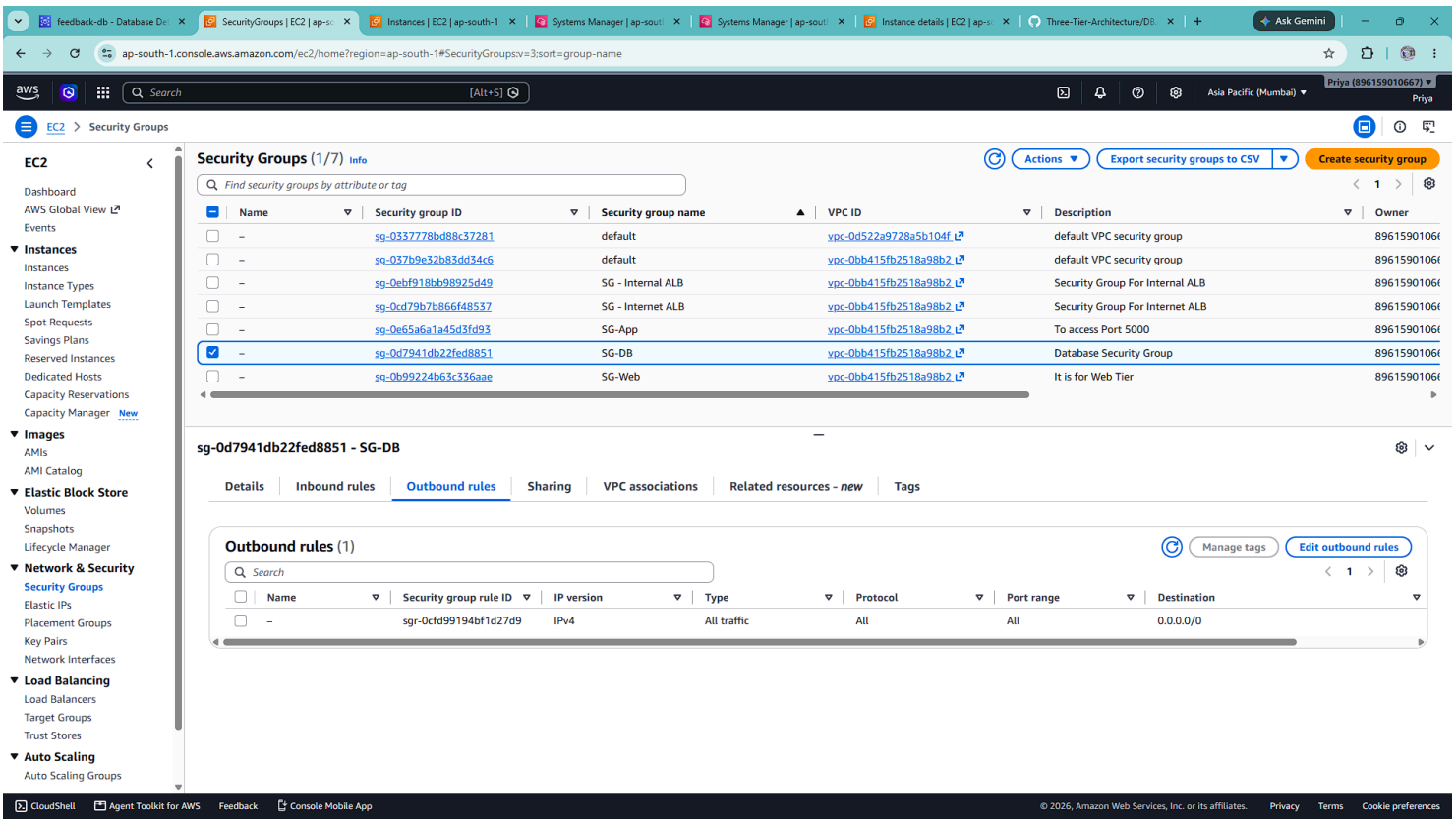
Application Tier : Configured a Security Group named "SG-App" for the Application tier EC2 instances. Inbound rule allows Port 5000 (Flask) only from the Web tier Security Group (SG-Web), ensuring only the Web tier can communicate with the Application tier. The Application EC2 instances are accessed securely using SSM Session Manager.



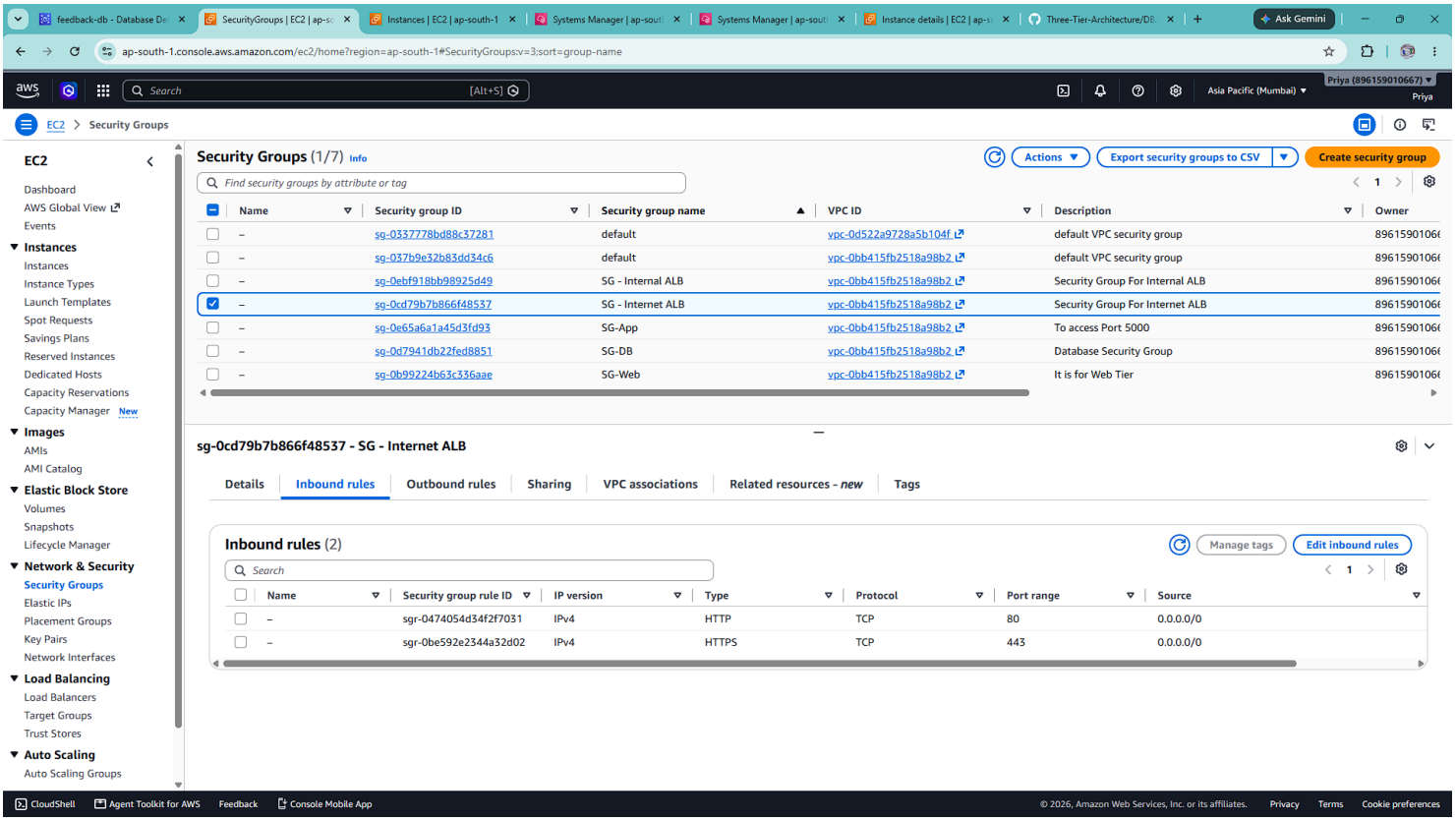


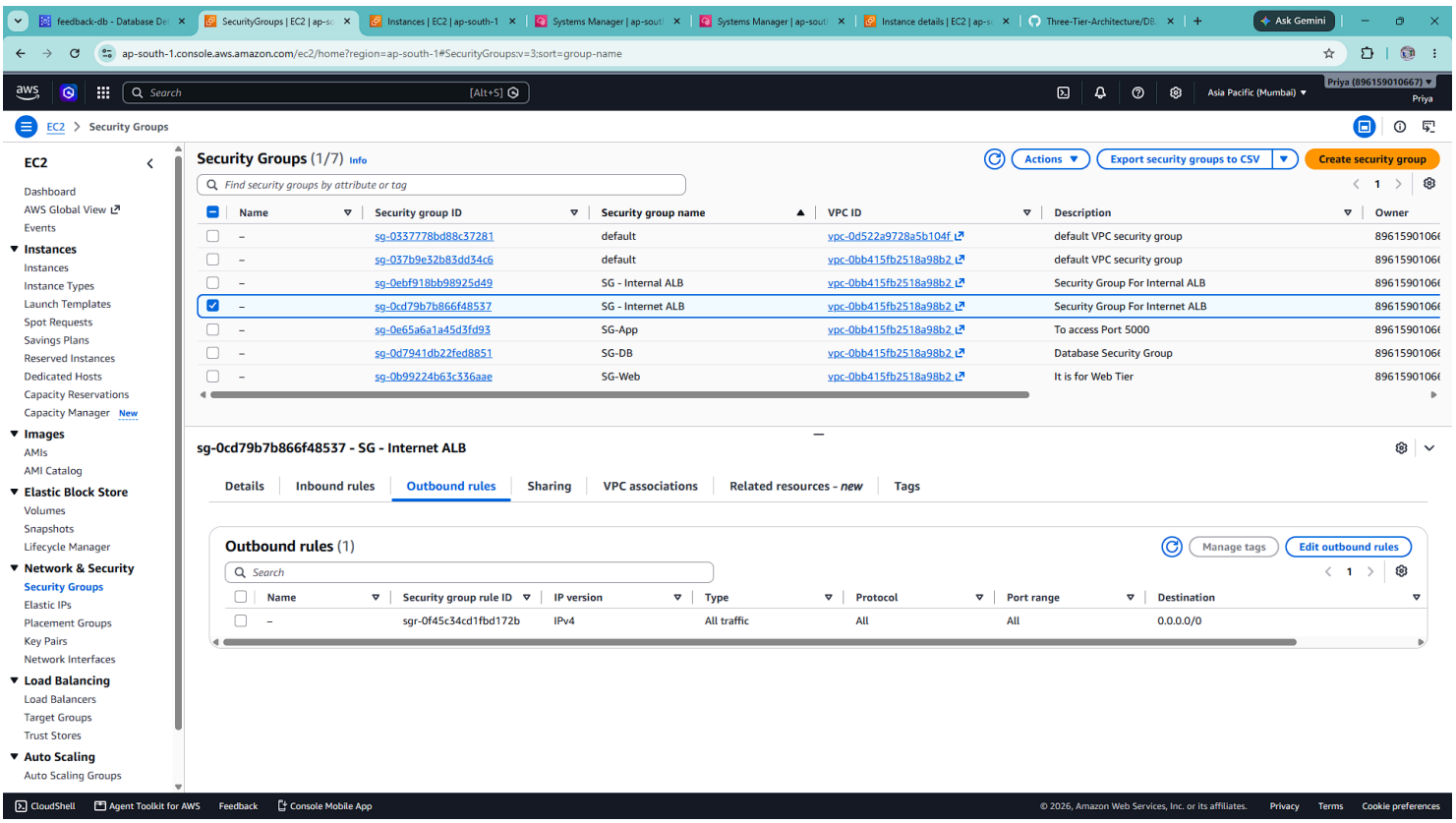
Database Tier : Configured a Security Group named "SG-DB" for the RDS MySQL instance. Inbound rule allows Port 3306 (MySQL/Aurora) only from the Application tier Security Group (SG-App), ensuring only the App EC2 instances can reach the database.



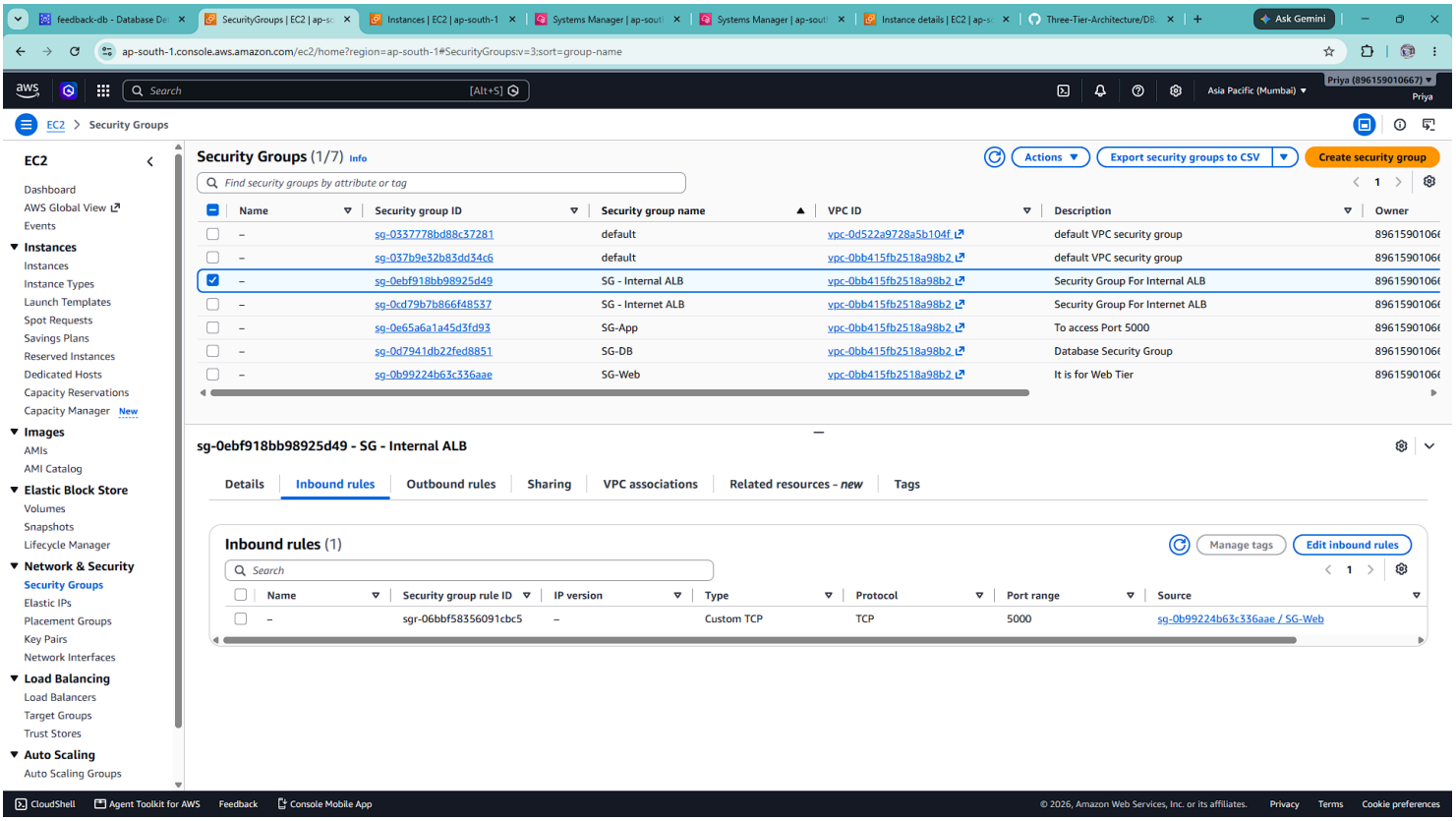


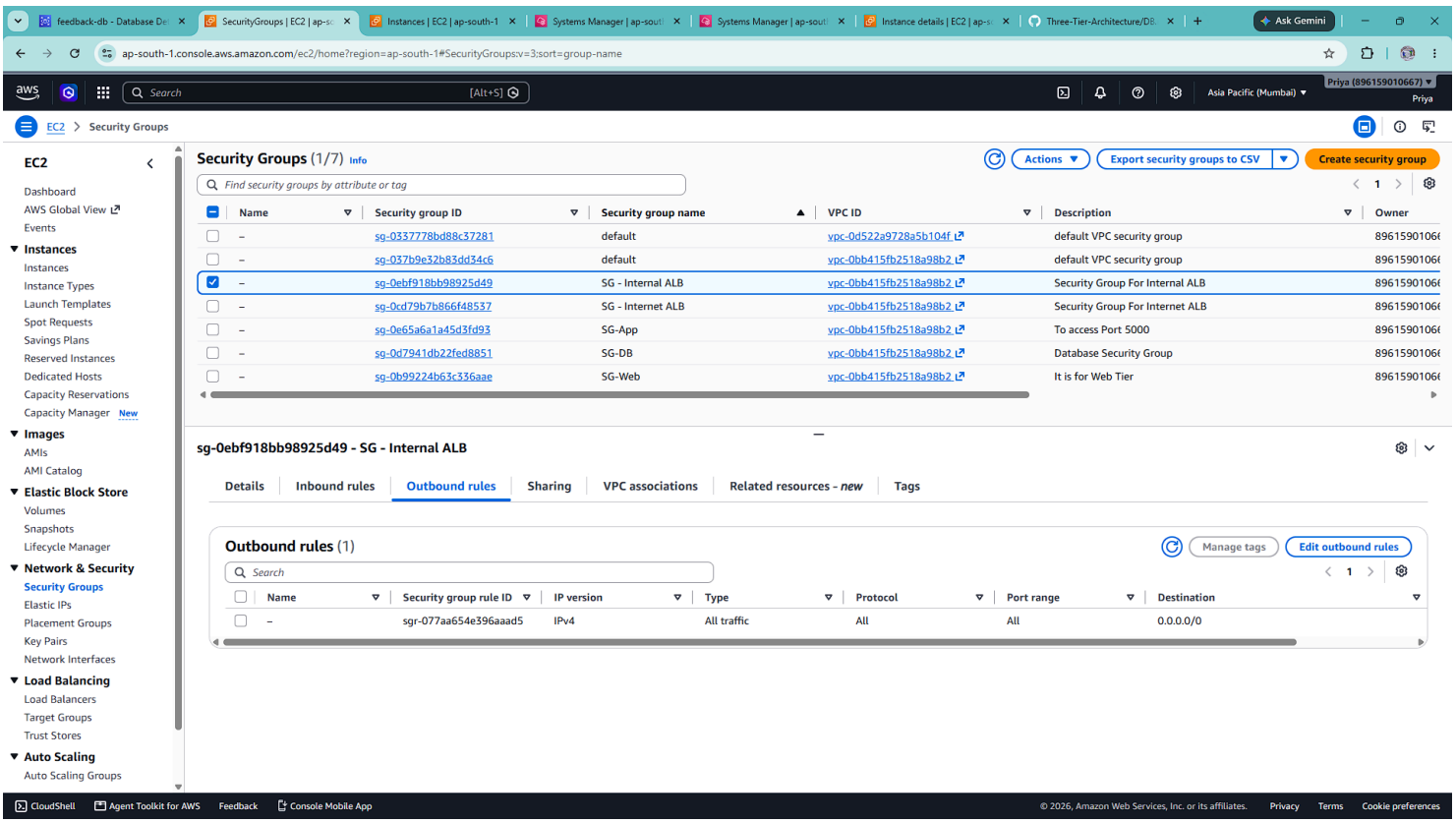
Internet Facing Load Balancer : Configured a Security Group named "SG - Internet ALB" for the Internet-facing Application Load Balancer. Inbound rules allow Port 80 (HTTP) and Port 443 (HTTPS) open to all (0.0.0.0/0) to accept incoming public internet traffic.





Internal Load Balancer : Configured a Security Group named "SG - Internal ALB" for the Internal Application Load Balancer. Inbound rule allows Port 5000 (Custom TCP) only from the Web tier Security Group (SG-Web), ensuring only the Web tier can forward requests to the Internal Load Balancer.



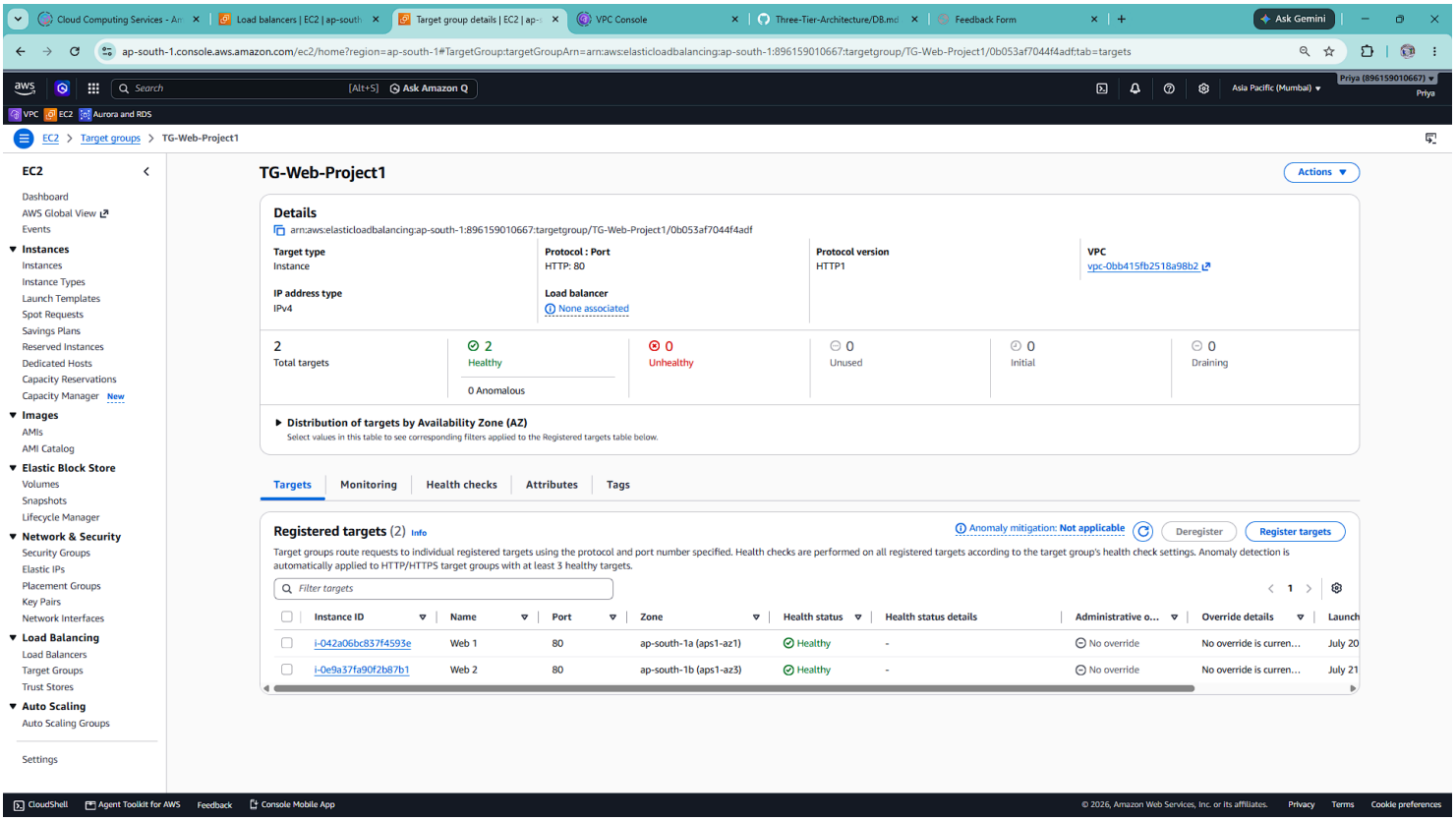


Task 6 - Configure the Internet-Facing Application Load Balancer

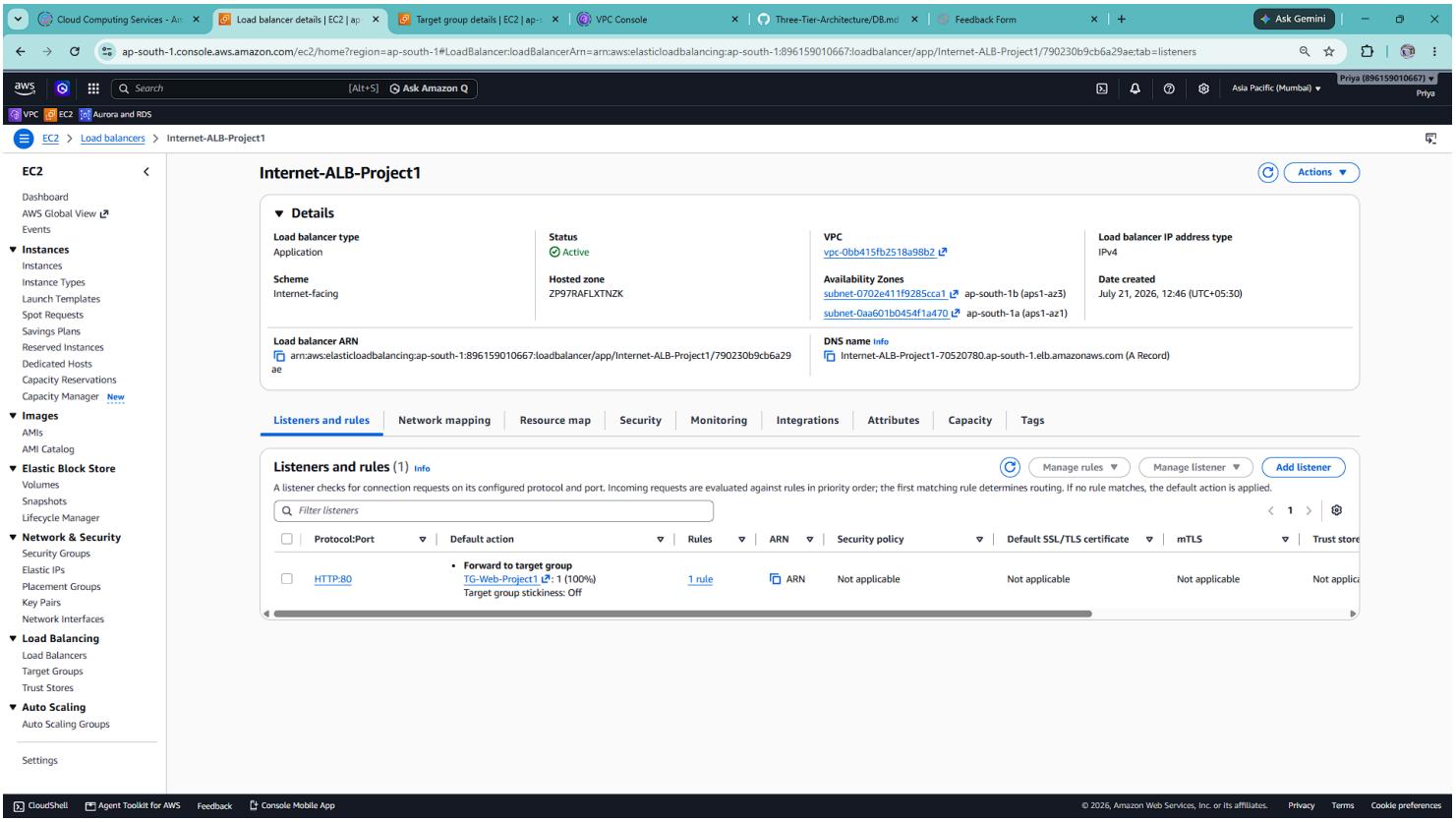
Target Group : Created a Target Group named "TG-Web-Project1"

Register Web EC2 Instances : Registered both Web EC2 instances (Web 1 in ap-south-1a and Web 2 in ap-south-1b) to the Target Group TG-Web-Project-1 on Port 80. Total 2 targets are registered across both Availability Zones.

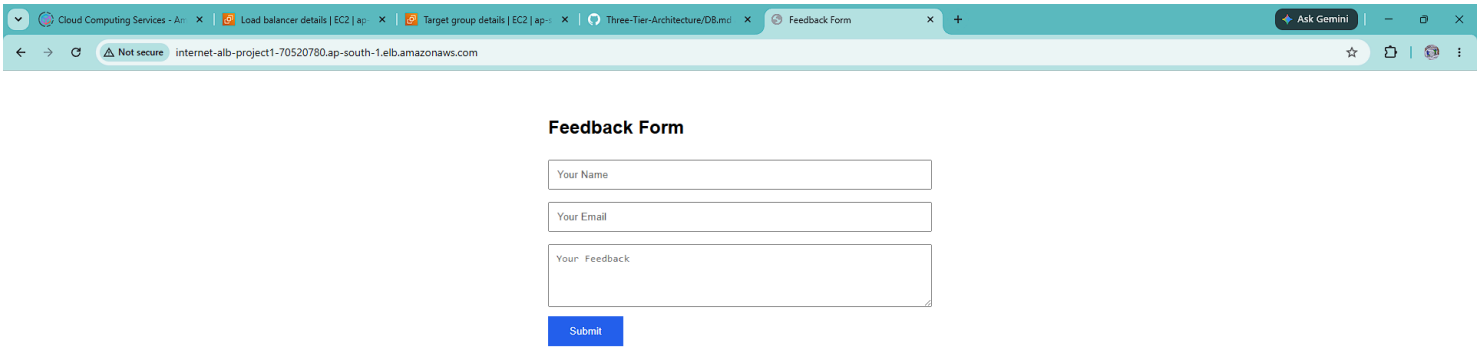
Health Checks : Configured Health Checks for the Target Group. Both registered targets (Web 1 and Web 2) show Health Status as Healthy with 0 Unhealthy targets, confirming the instances are running and responding correctly.



Load Balancer : Created an internet-facing Application Load Balancer named "Internet-ALB-Project-1". The Load Balancer is attached to the Public Subnets across both Availability Zones (ap-south-1a and ap-south-1b). The listener is configured on HTTP:80 and forwards 100% of traffic to TG-Web-Project1.

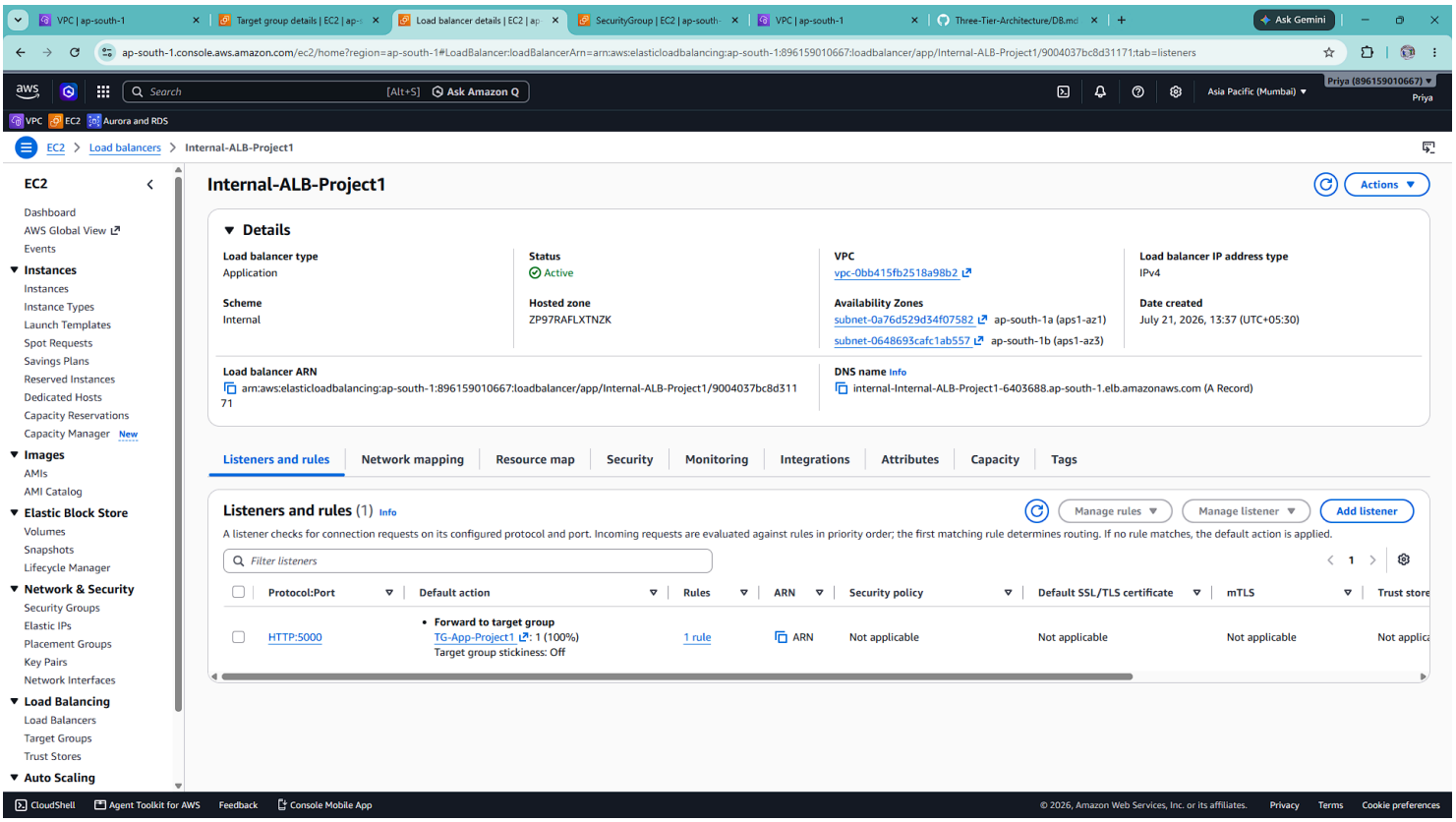


Verification : Accessed the application using the ALB DNS name : "Internet-ALB-Project1-70520780.ap-south-1.elb.amazonaws.com" in the browser. The Feedback Form page loaded successfully, confirming the Load Balancer is routing internet traffic correctly to the Web tier instances.



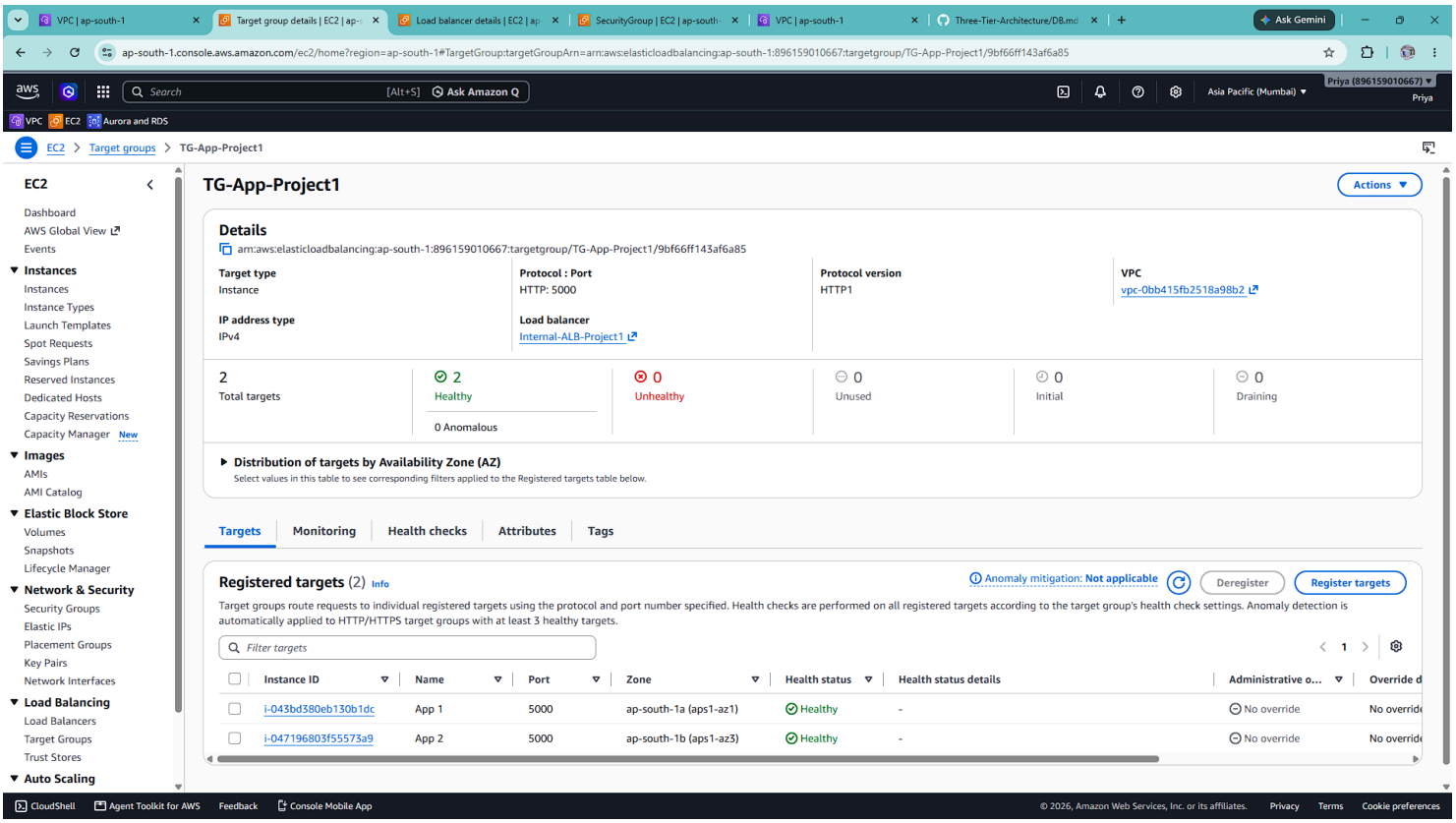
Task 7 - Configure Internal Application Load Balancer

Internal ALB : Created an Internal Application Load Balancer named "Internal-ALB-Project1". The Load Balancer is attached to the Private Application Subnets across both Availability Zones (ap-south-1a and ap-south-1b). The listener is configured on HTTP:5000 and forwards 100% of traffic to TG-App-Project1.

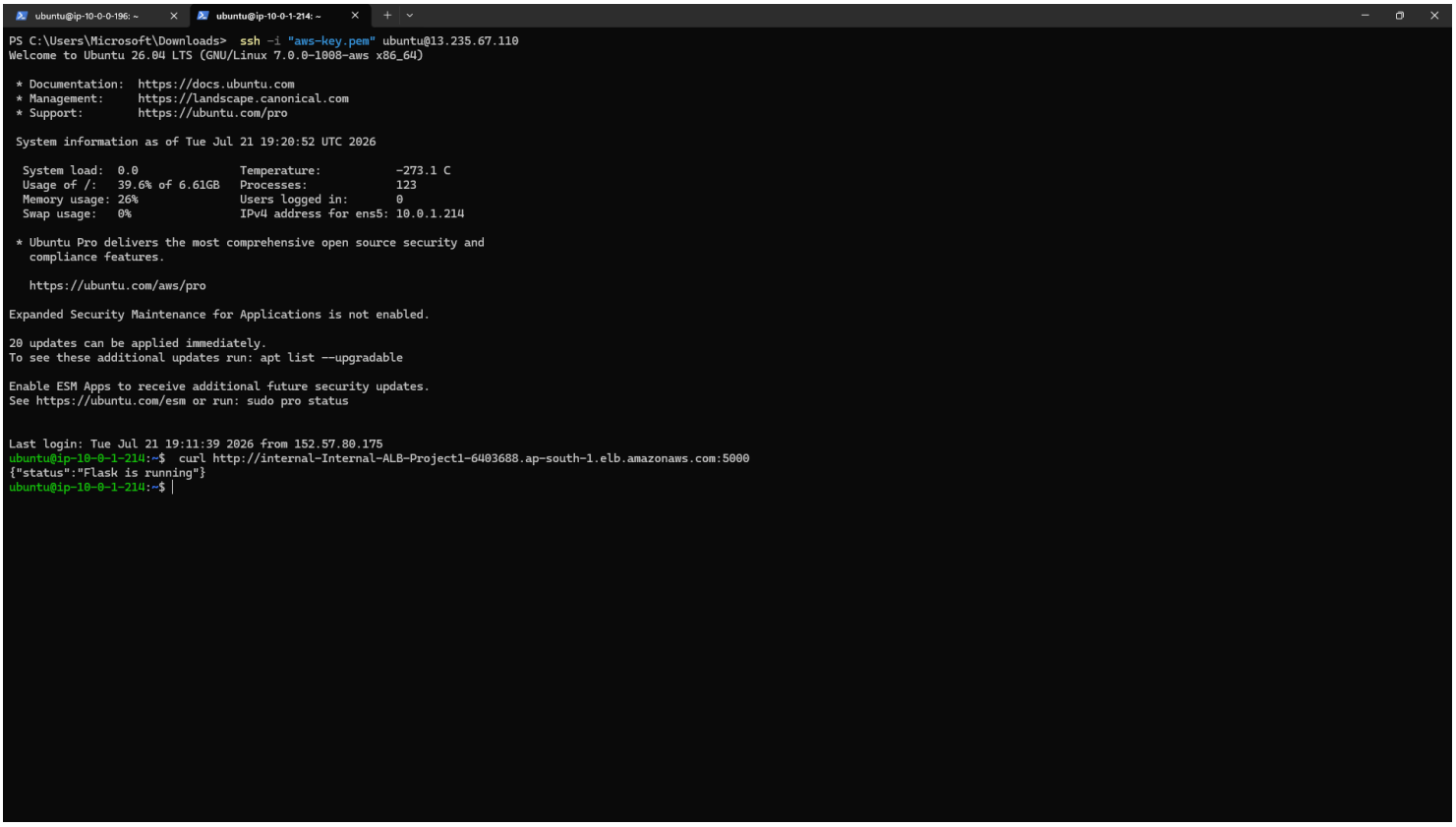


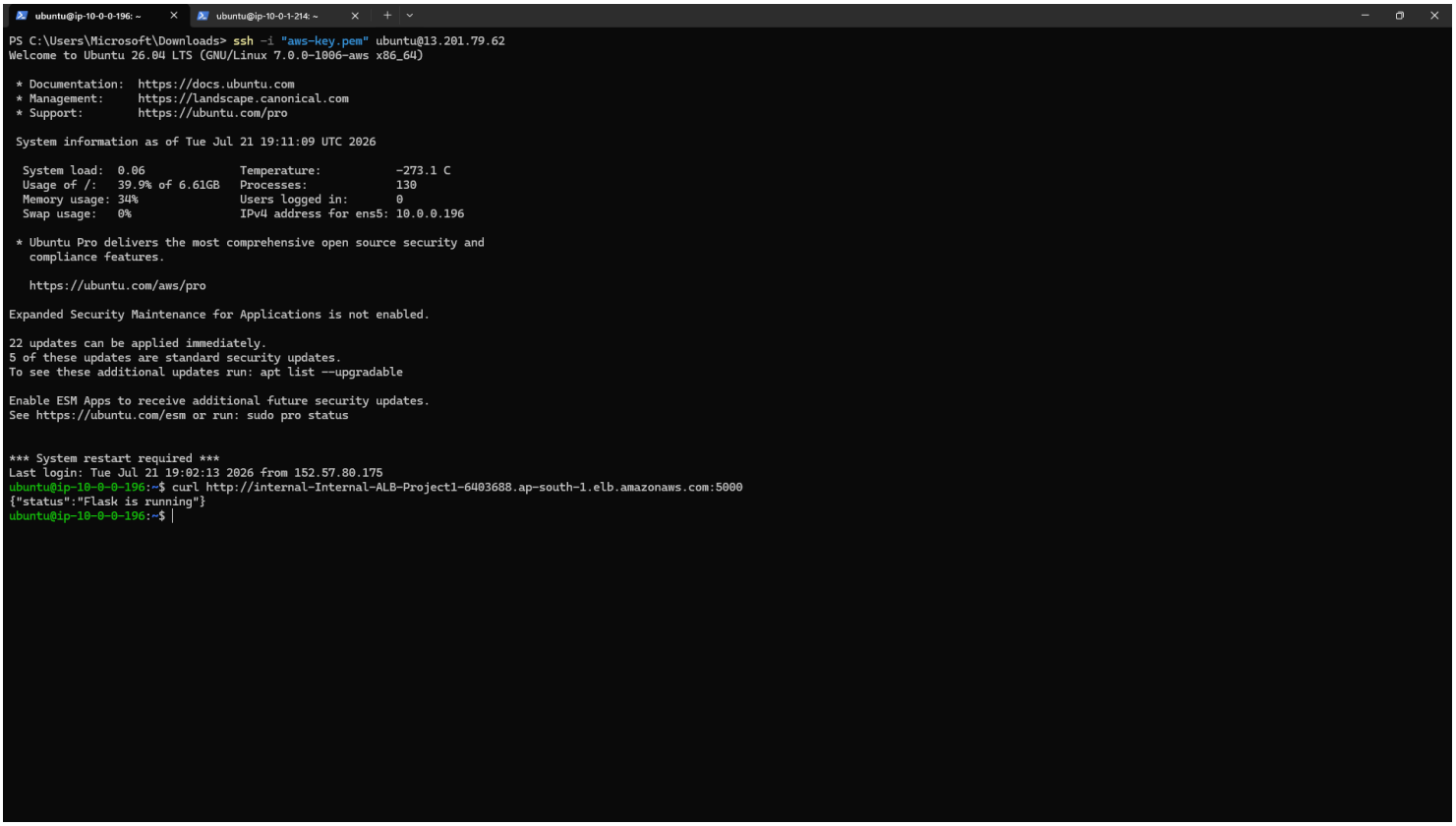
Target Group : Created a Target Group named "TG-App-Project1". Registered both App EC2 instances (App 1 in ap-south-1a and App 2 in ap-south-1b) on Port 5000.

Health Checks : Both registered App instances (App 1 and App 2) show Health Status as Healthy with 0 Unhealthy targets, confirming the Flask application is running and responding correctly on both instances.



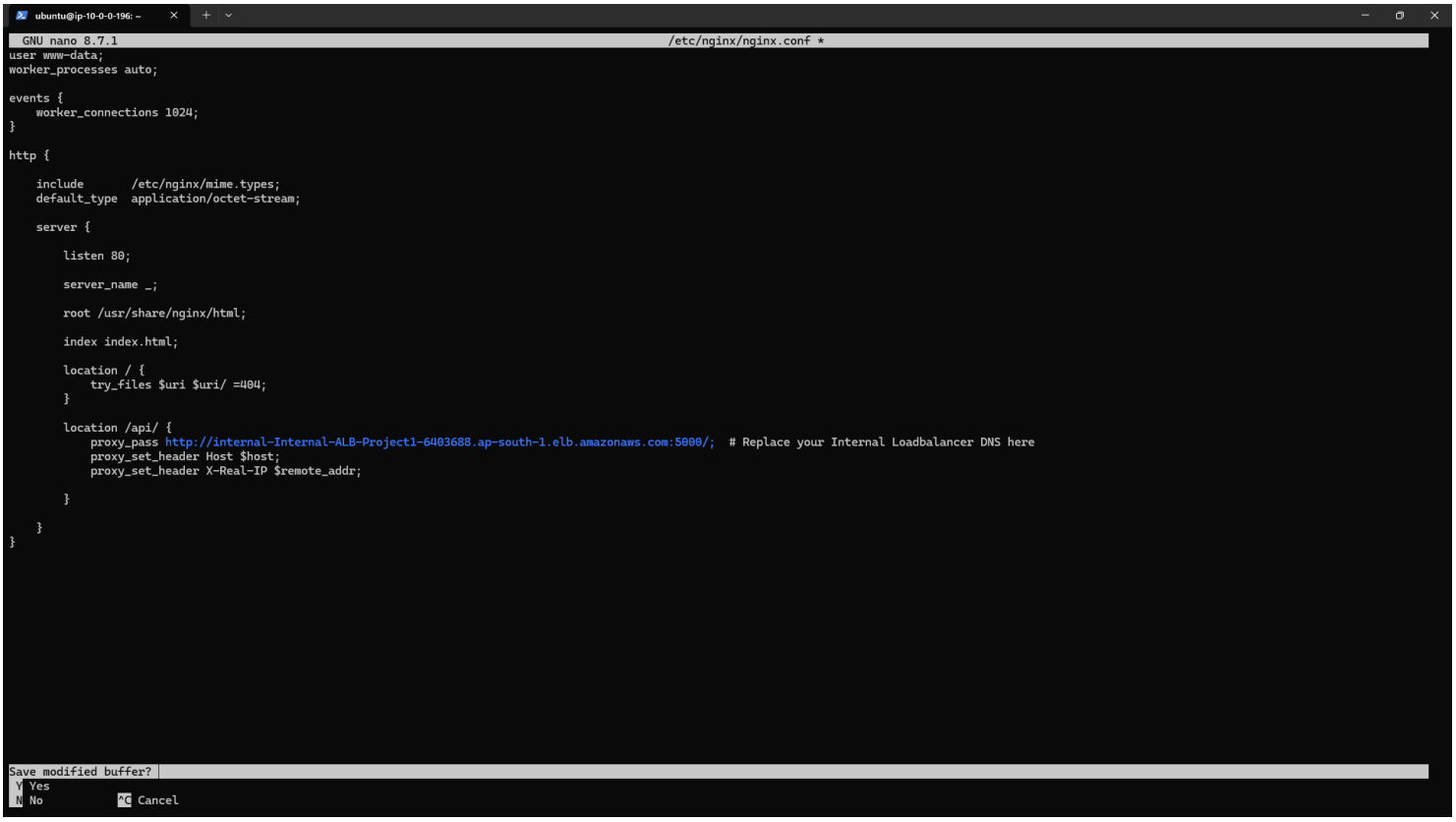
Verification : Connected to Web 1 (10.0.0.196) and Web 2 (10.0.1.214) via SSH and ran curl on the Internal ALB DNS name "internal-Internal-ALB-Project1-6403688.ap-south-1.elb.amazonaws.com:5000". Both instances returned {"status":"Flask is running"}, confirming successful communication between the Web Tier and Application Tier through the Internal Load Balancer.

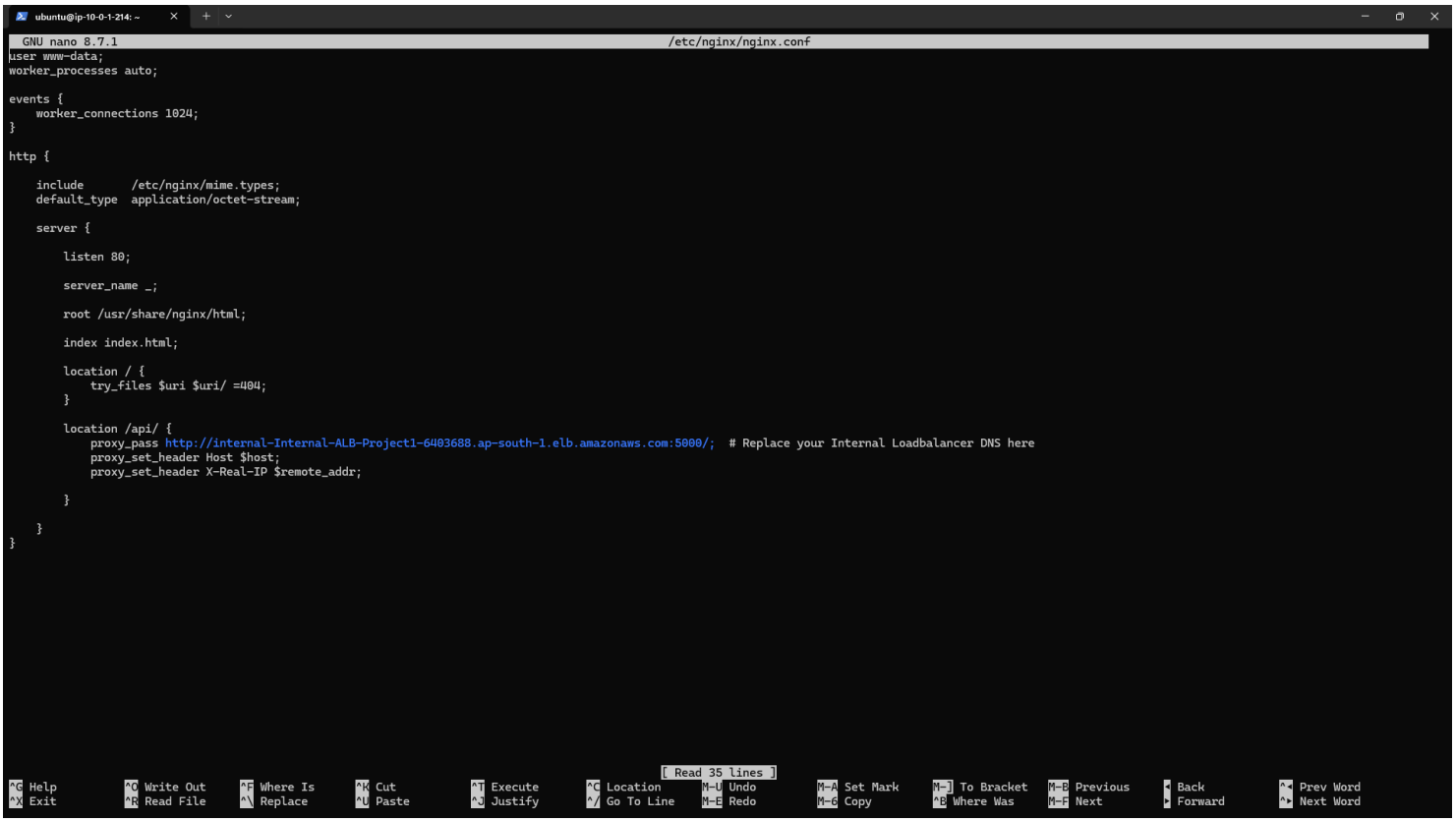




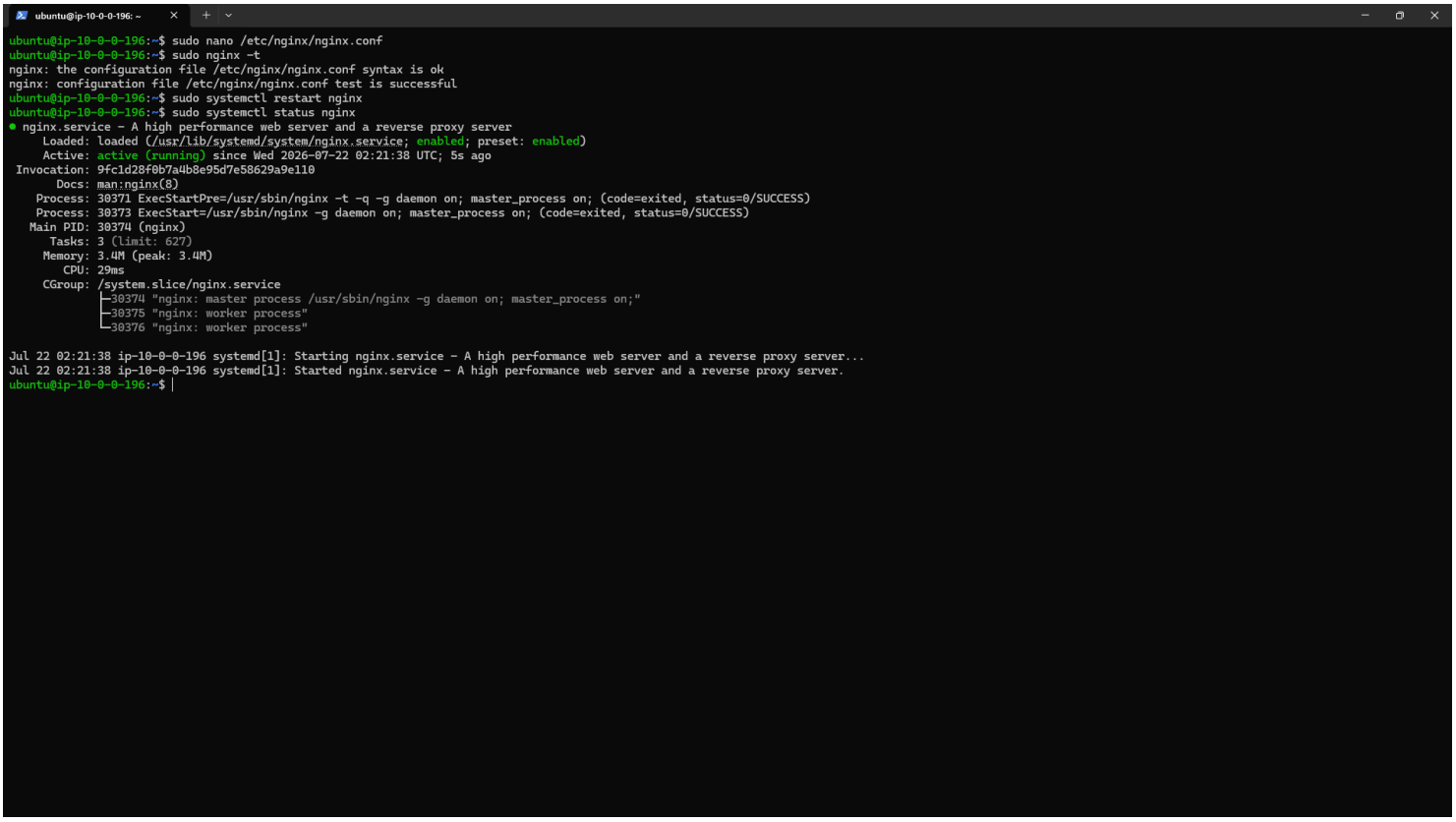
Task 8 - Configure Nginx Reverse Proxy for API Routing

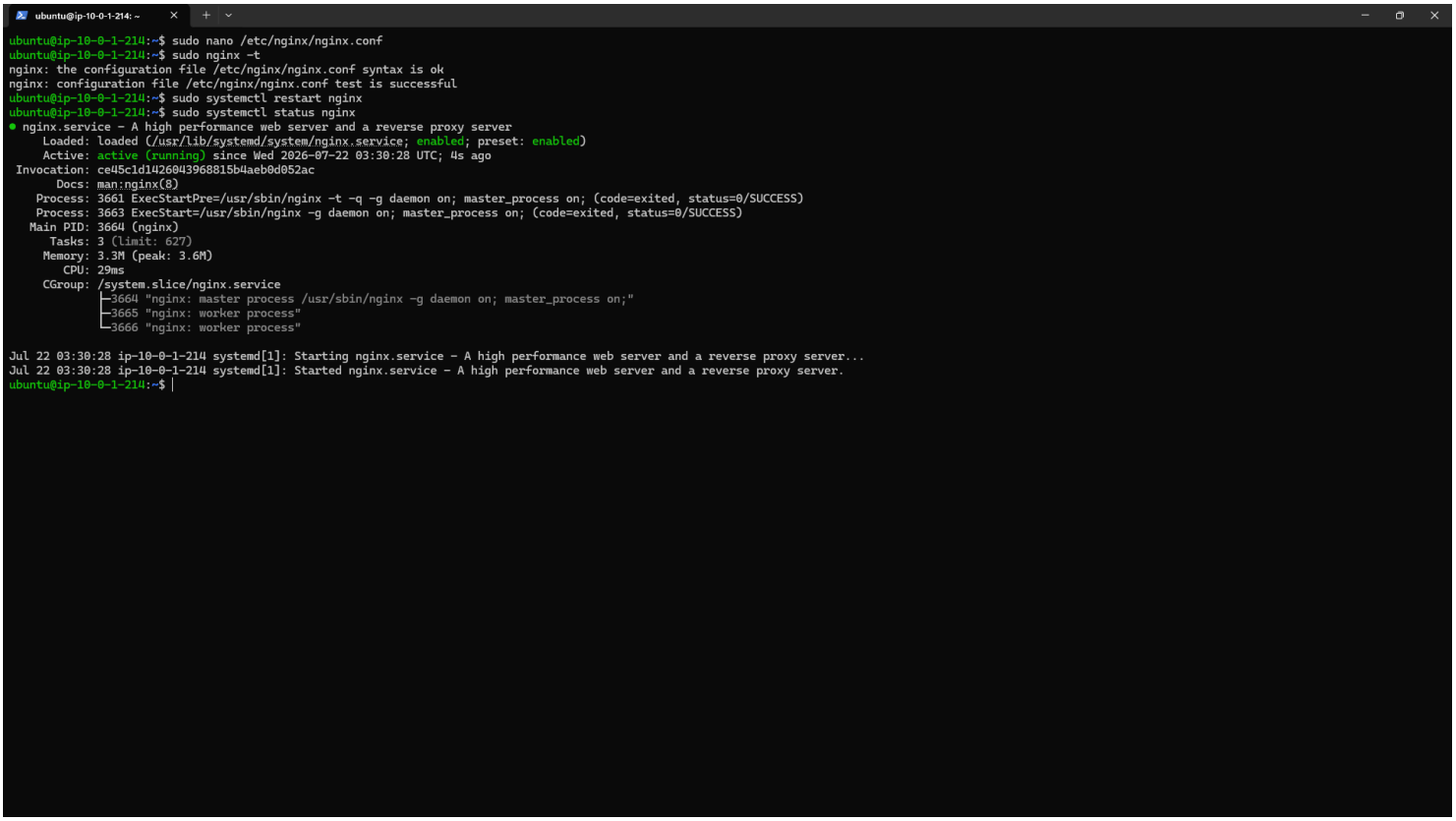
Nginx Configuration : Replaced the default Nginx config with the custom nginx.conf from GitHub on both Web 1 (10.0.0.196) and Web 2 (10.0.1.214). The "location /api/" block was configured to forward all API requests to the Internal ALB DNS on port 5000.



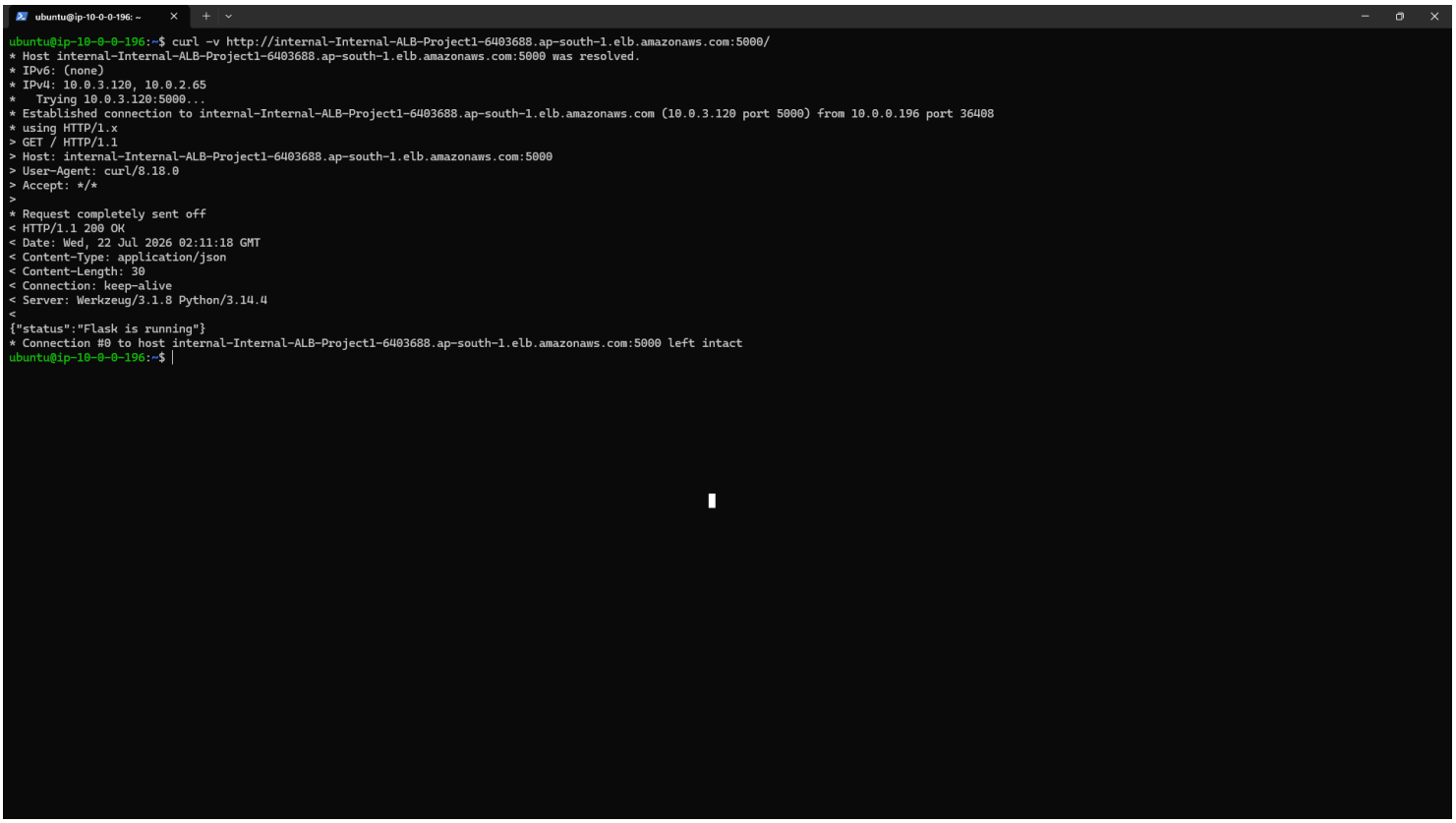


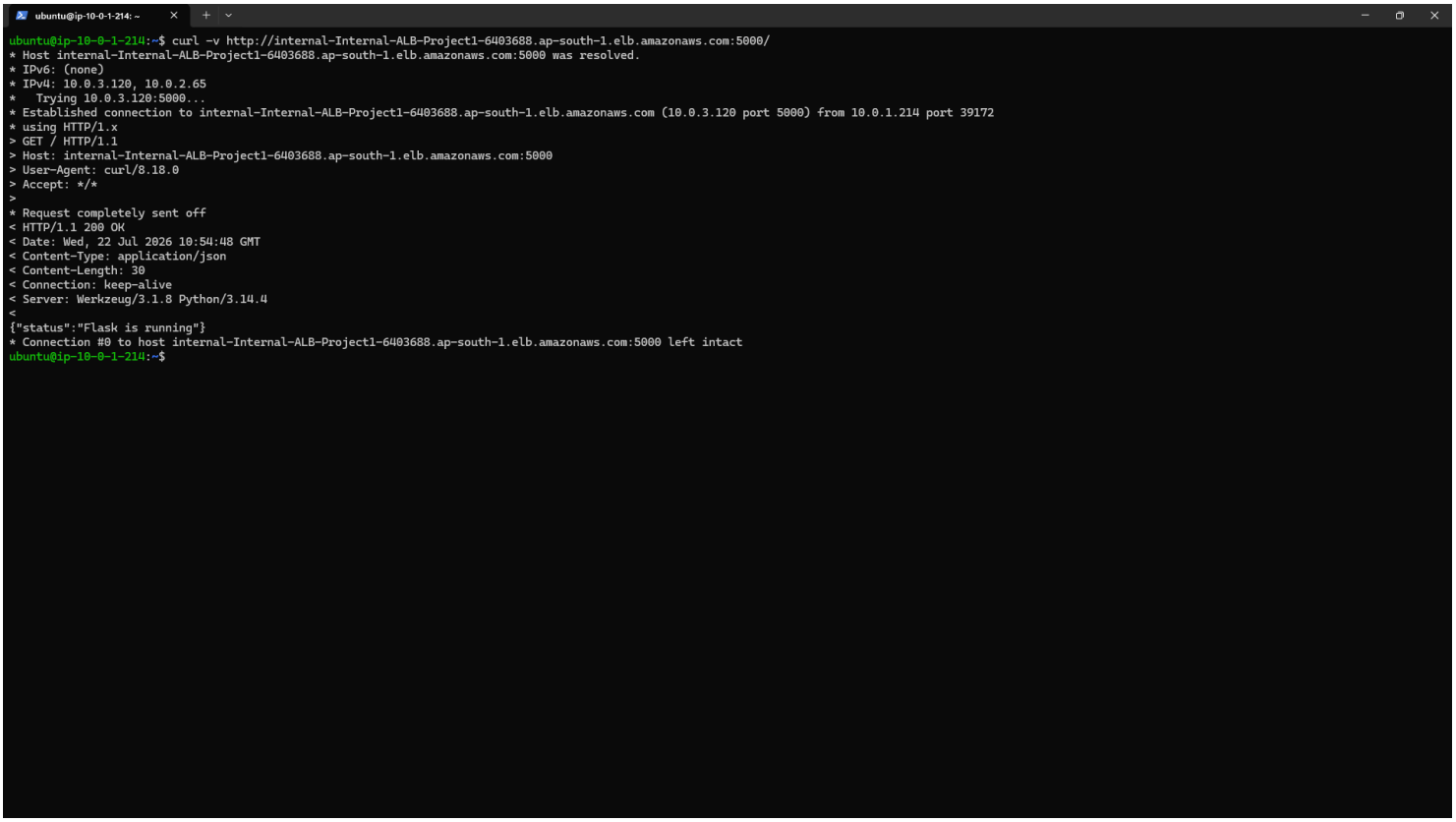
Syntax Test & Restart : The updated configuration was tested and confirmed valid on both instances, then Nginx was restarted. The service status showed active (running) on both Web 1 and Web 2, confirming the new config loaded successfully.



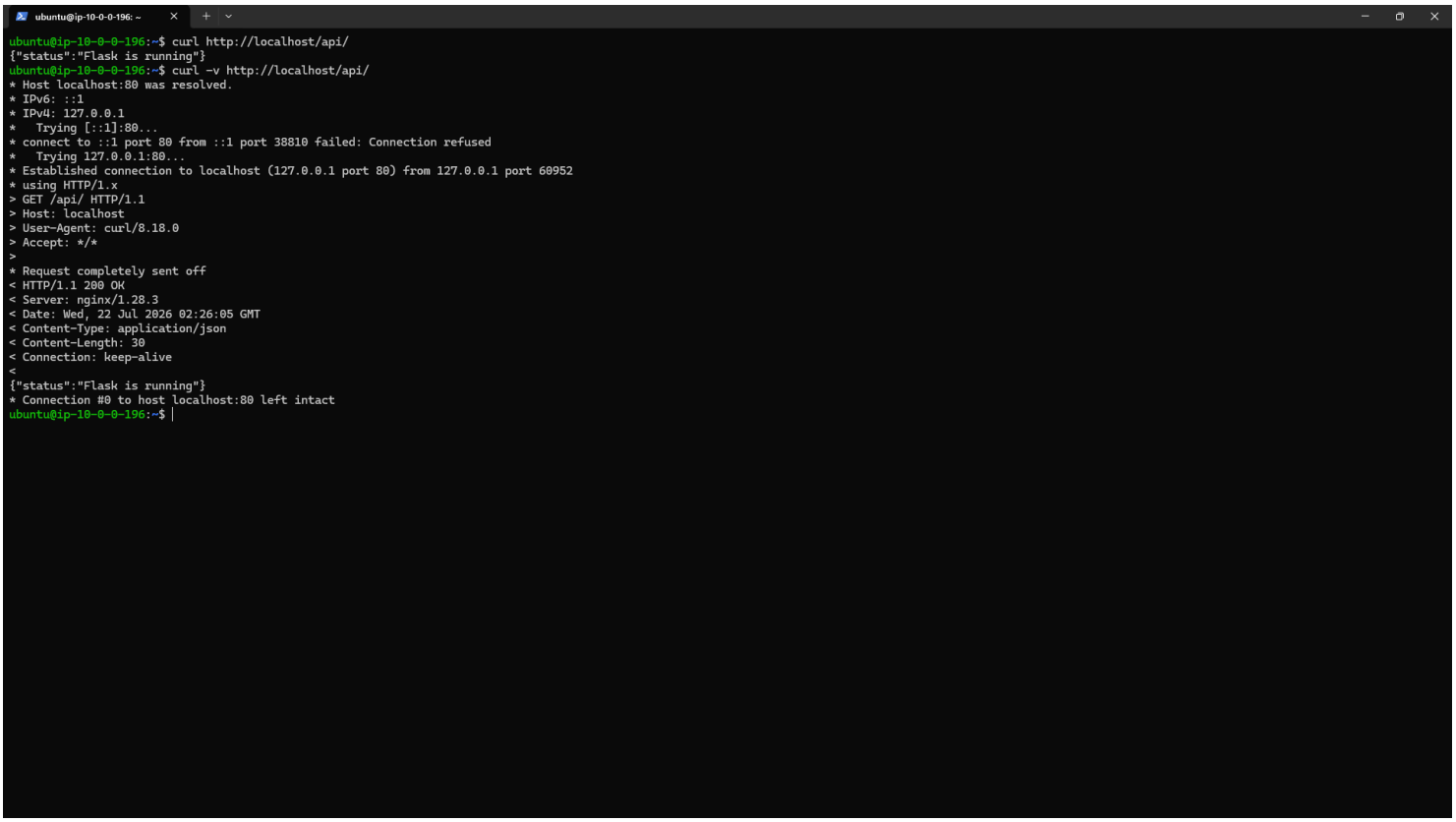


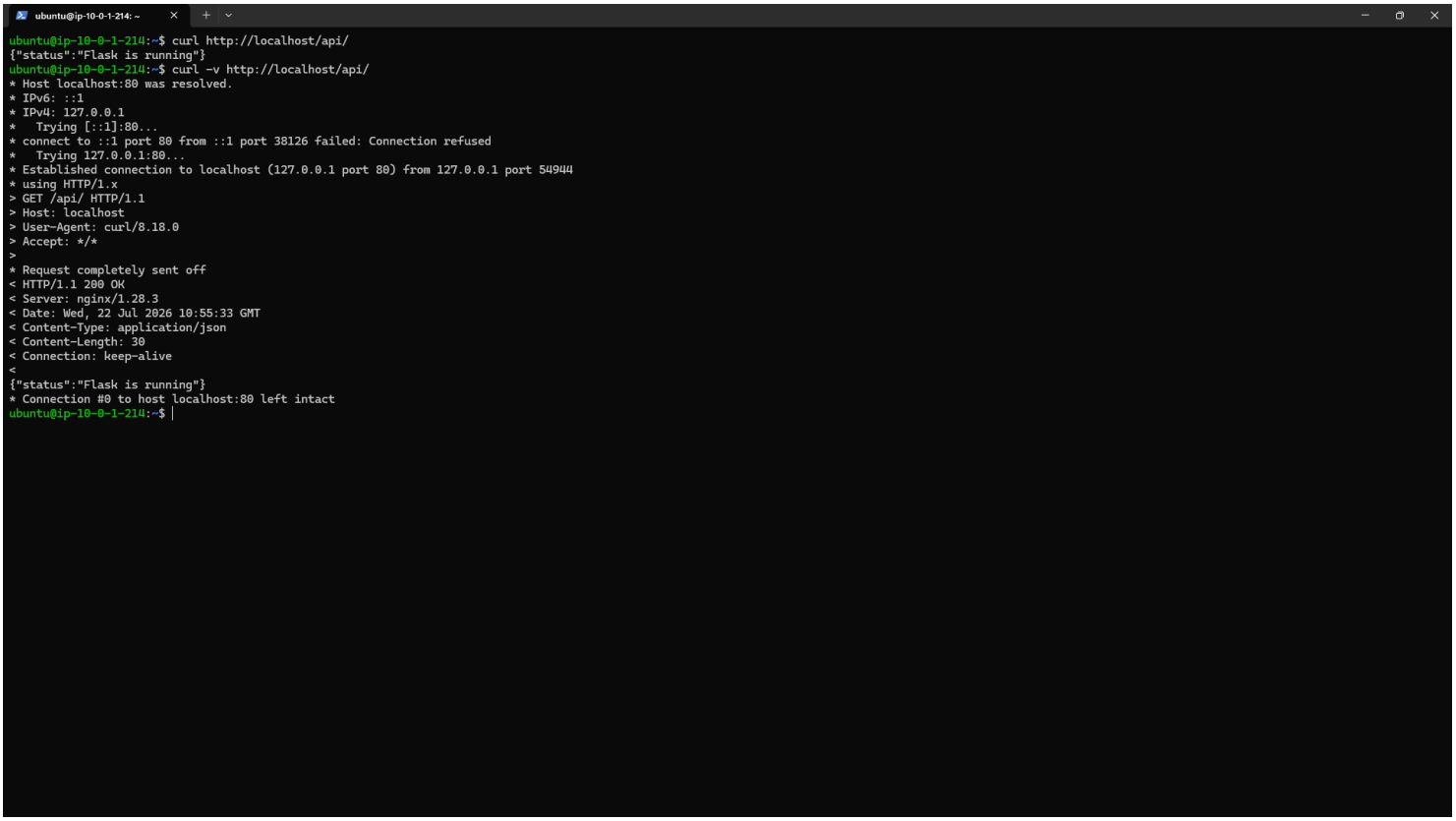
Internal ALB Verification : From both Web 1 and Web 2, a test request was sent to the Internal ALB DNS on port 5000. Both instances returned HTTP 200 OK with {"status":"Flask is running"}, confirming Web Tier to App Tier connectivity through the Internal Load Balancer.



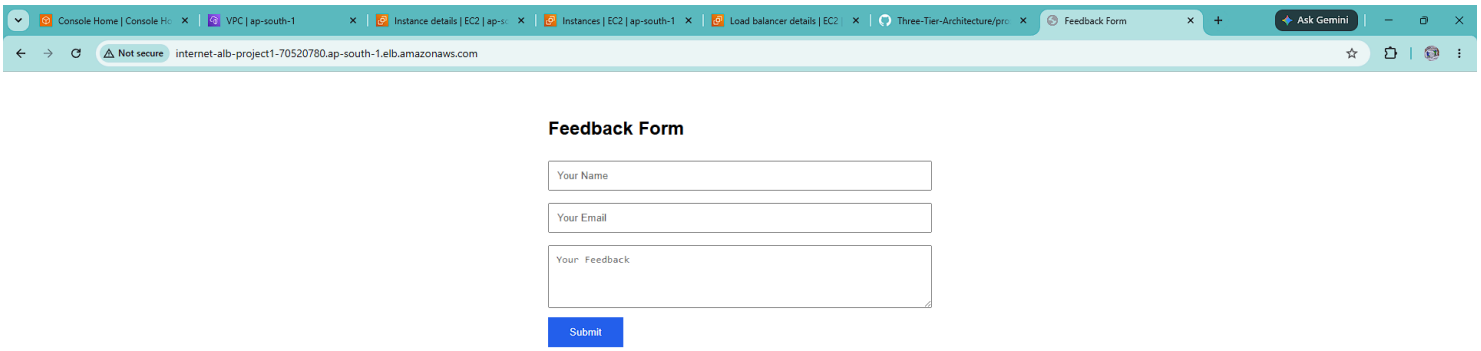


Reverse Proxy Verification : A request to localhost/api/ on both Web 1 and Web 2 returned HTTP 200 OK with {"status":"Flask is running"} via Nginx, confirming the proxy routing is working correctly on both instances.



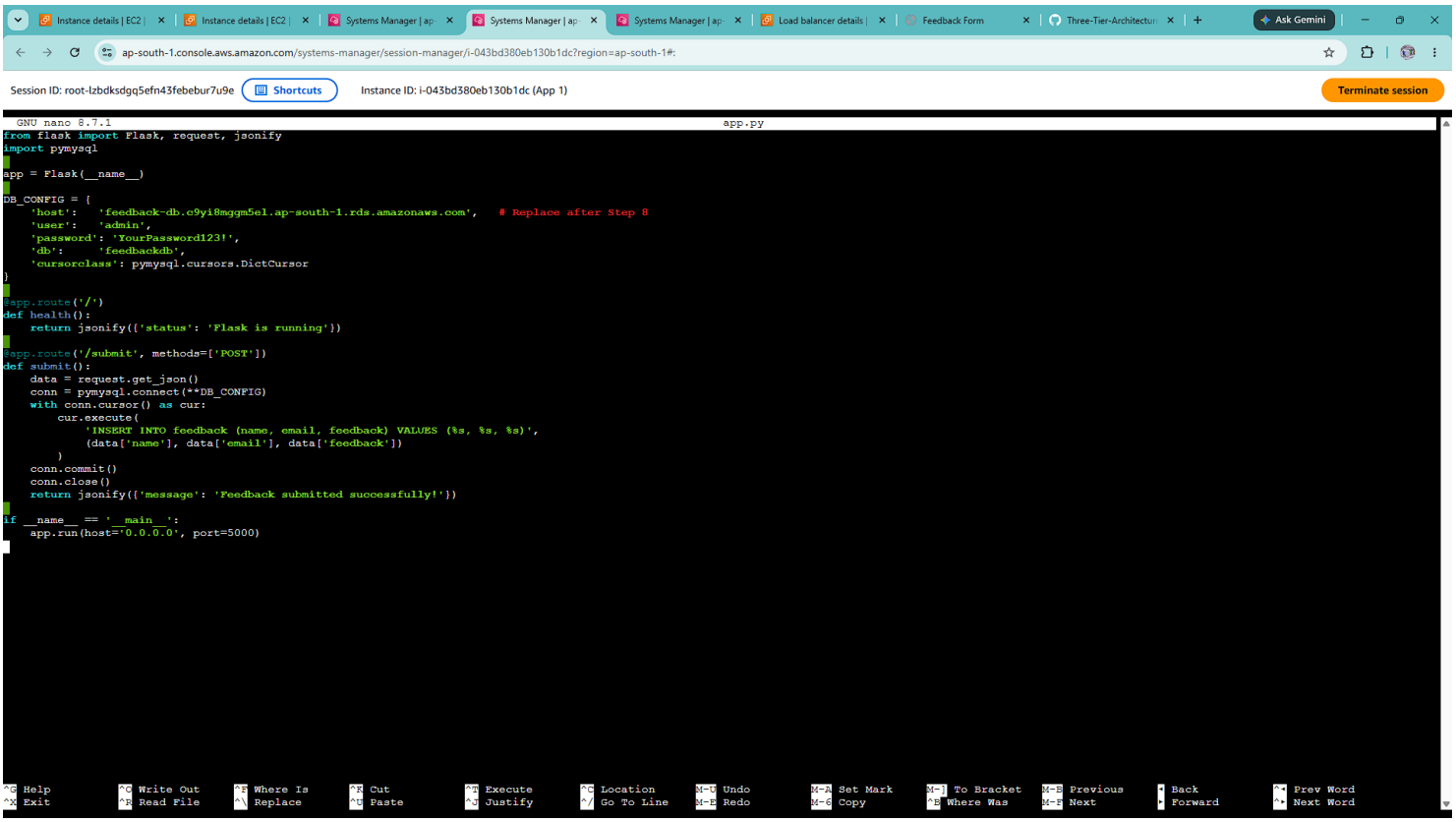


End-to-End Verification : The Internet ALB URL was accessed in the browser and the Feedback Form loaded successfully, confirming the full 3-tier architecture is functioning end to end.

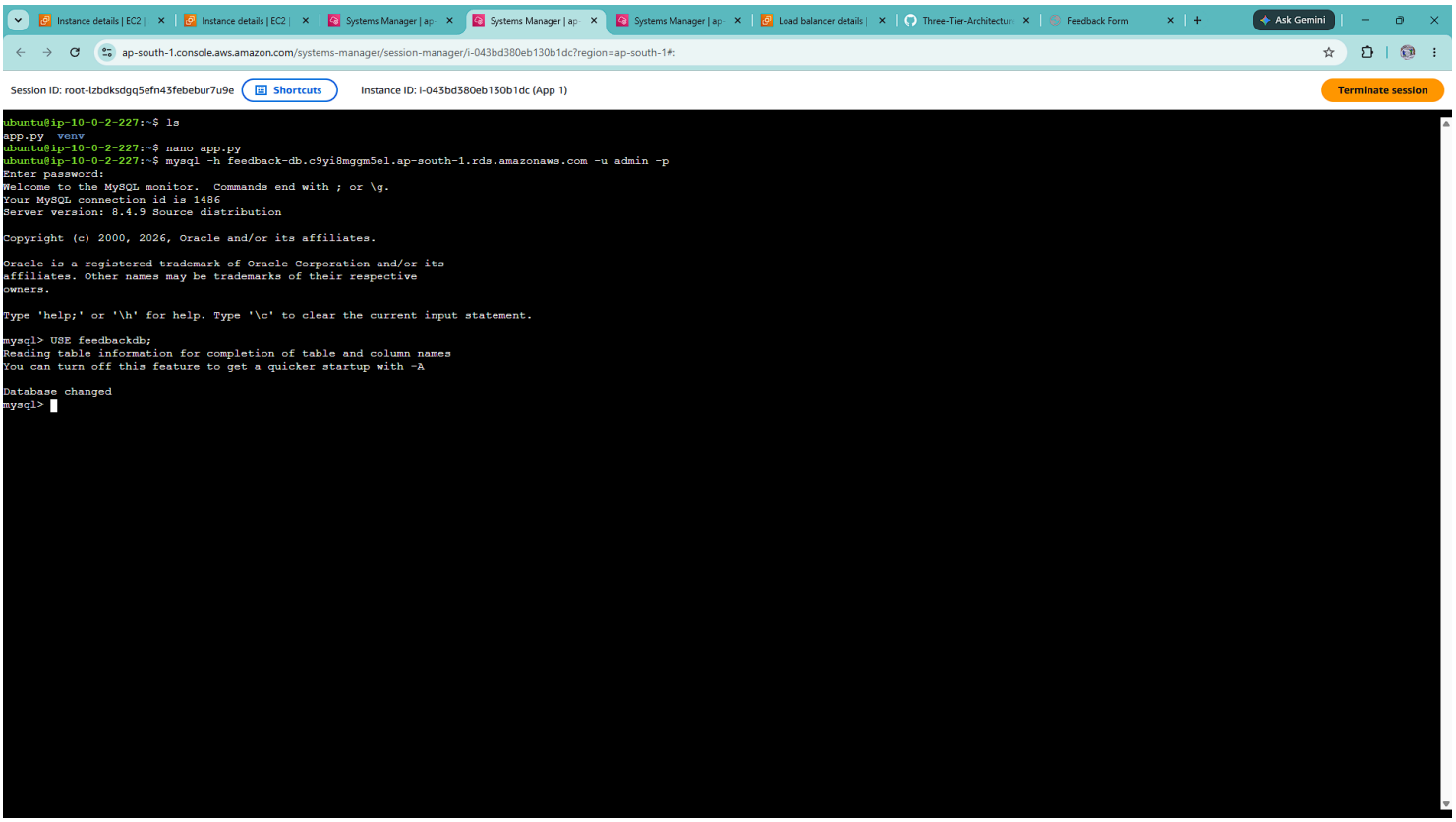


Task 9 - Integrate the Flask Application with Amazon RDS

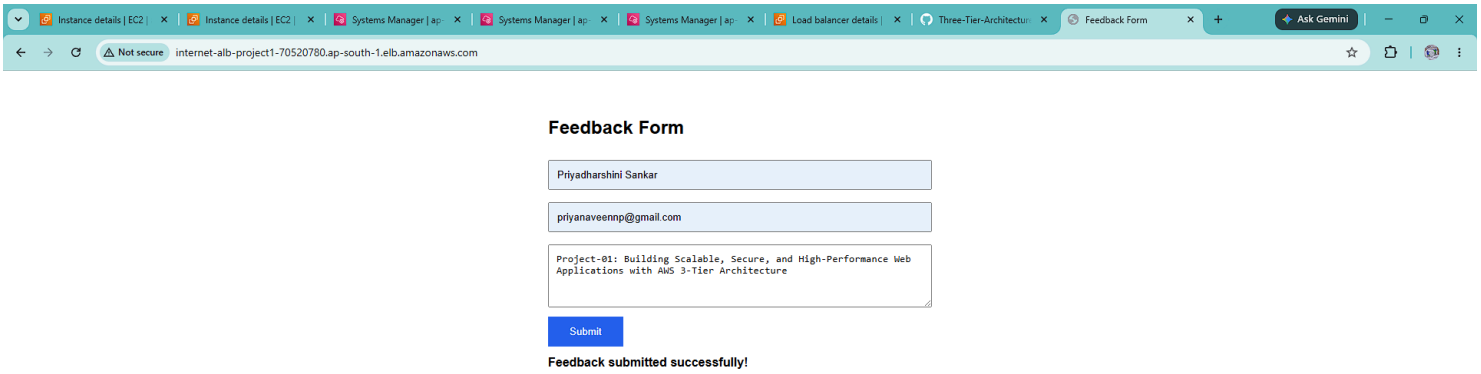
Database Configuration Verification : Verified the DB_CONFIG section in the app.py file on the Application EC2 instance. Confirmed that the Amazon RDS endpoint, database name, username and password were correctly configured, enabling the Flask application to connect to the Amazon RDS MySQL database.



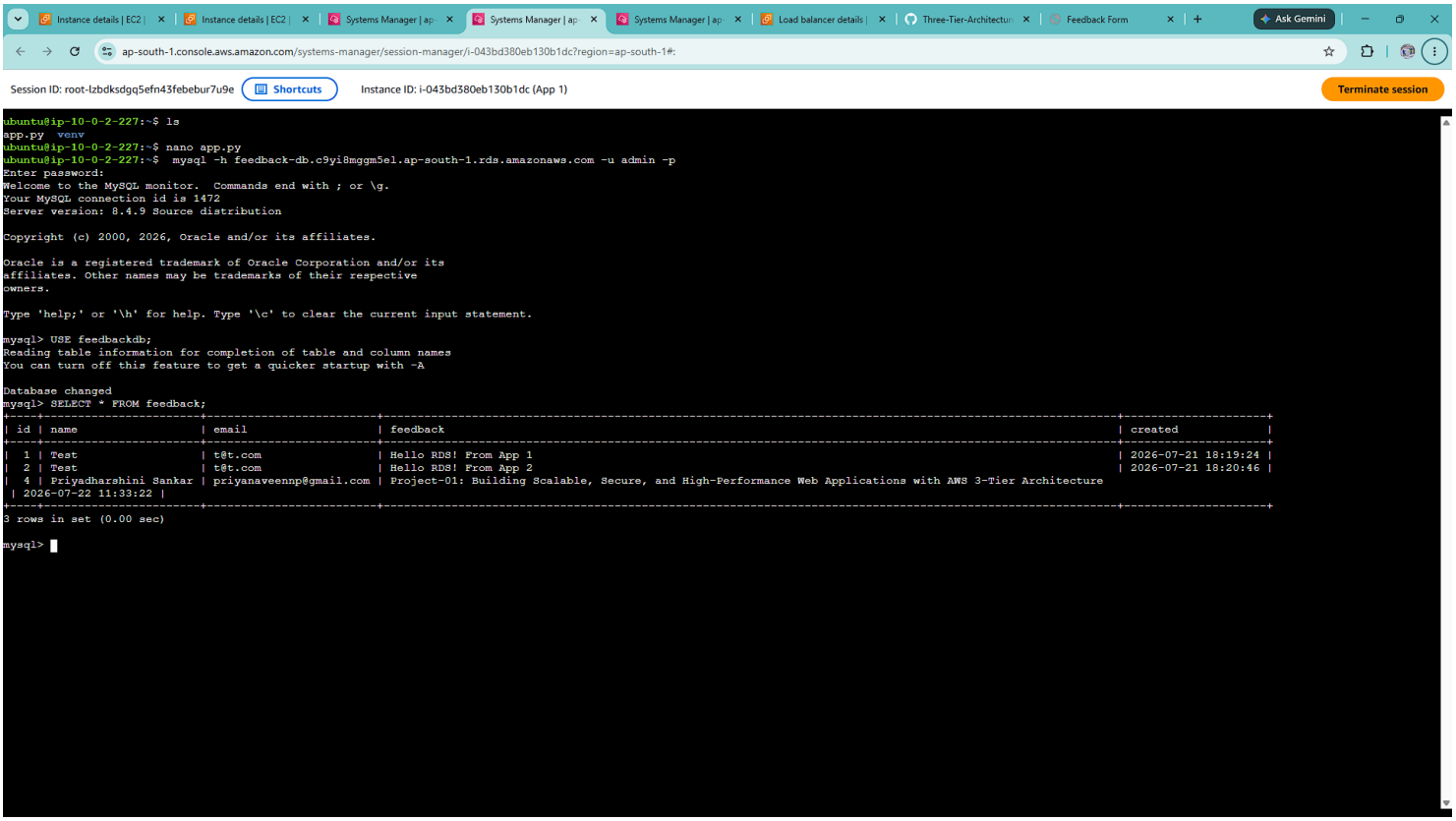
RDS Connectivity Verification : Connected to the App 1 instance using AWS Systems Manager and logged in to the Amazon RDS MySQL database using the configured endpoint and credentials. Successfully accessed the feedbackdb database, confirming that the Application Tier can communicate with the Amazon RDS instance.



Sample Feedback Submission : Accessed the Feedback Form through the Internet-facing Application Load Balancer (ALB) DNS. Entered the required details (name, email and feedback) and clicked Submit. The application displayed the message **"Feedback submitted successfully!"** confirming that the Flask application successfully processed the request and stored the feedback in the database.

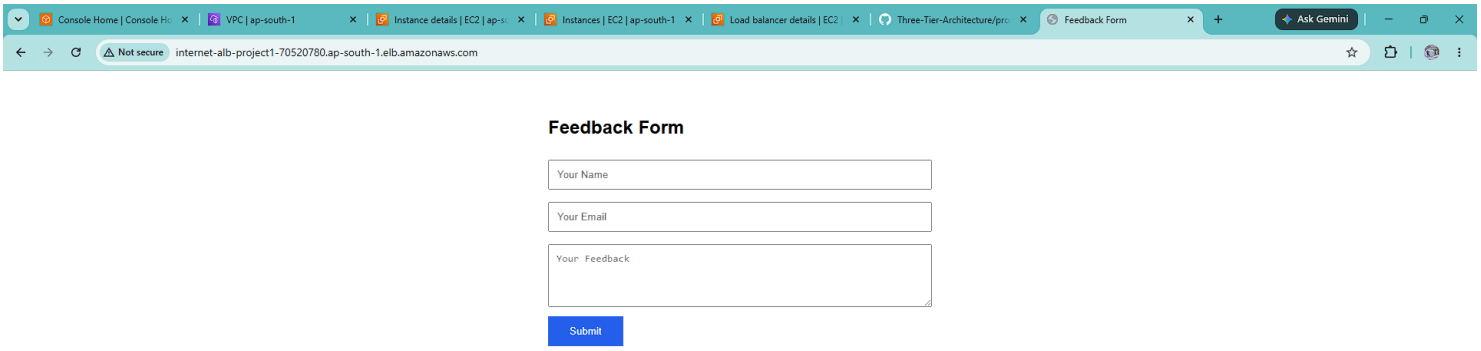


Data Confirmation in RDS : Executed a query on the feedback table in the feedbackdb database. The query returned all stored records, including the newly submitted entry from **Priyadharshini Sankar**, confirming that the feedback data was successfully stored in the Amazon RDS MySQL database.

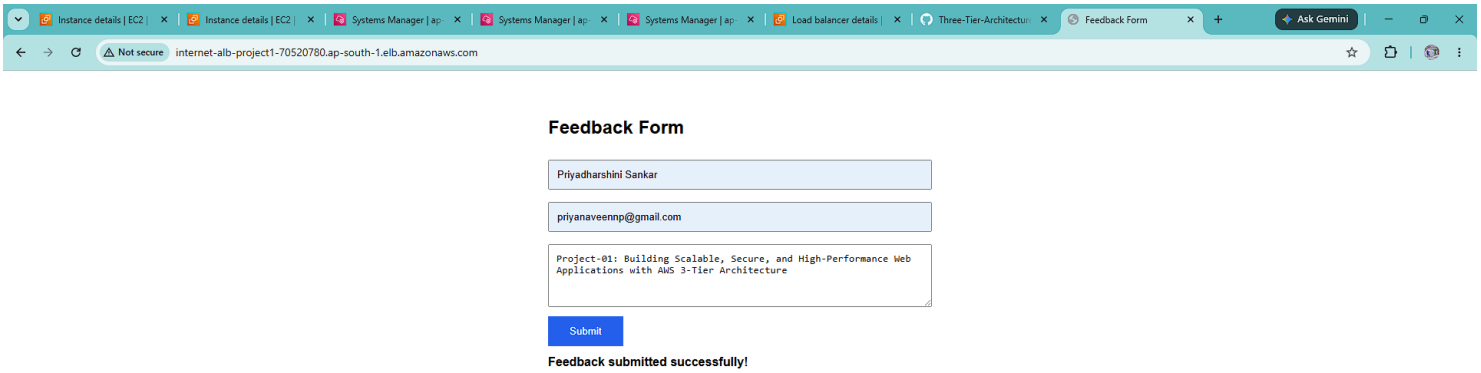


Task 10 - Validate the Complete 3-Tier Architecture

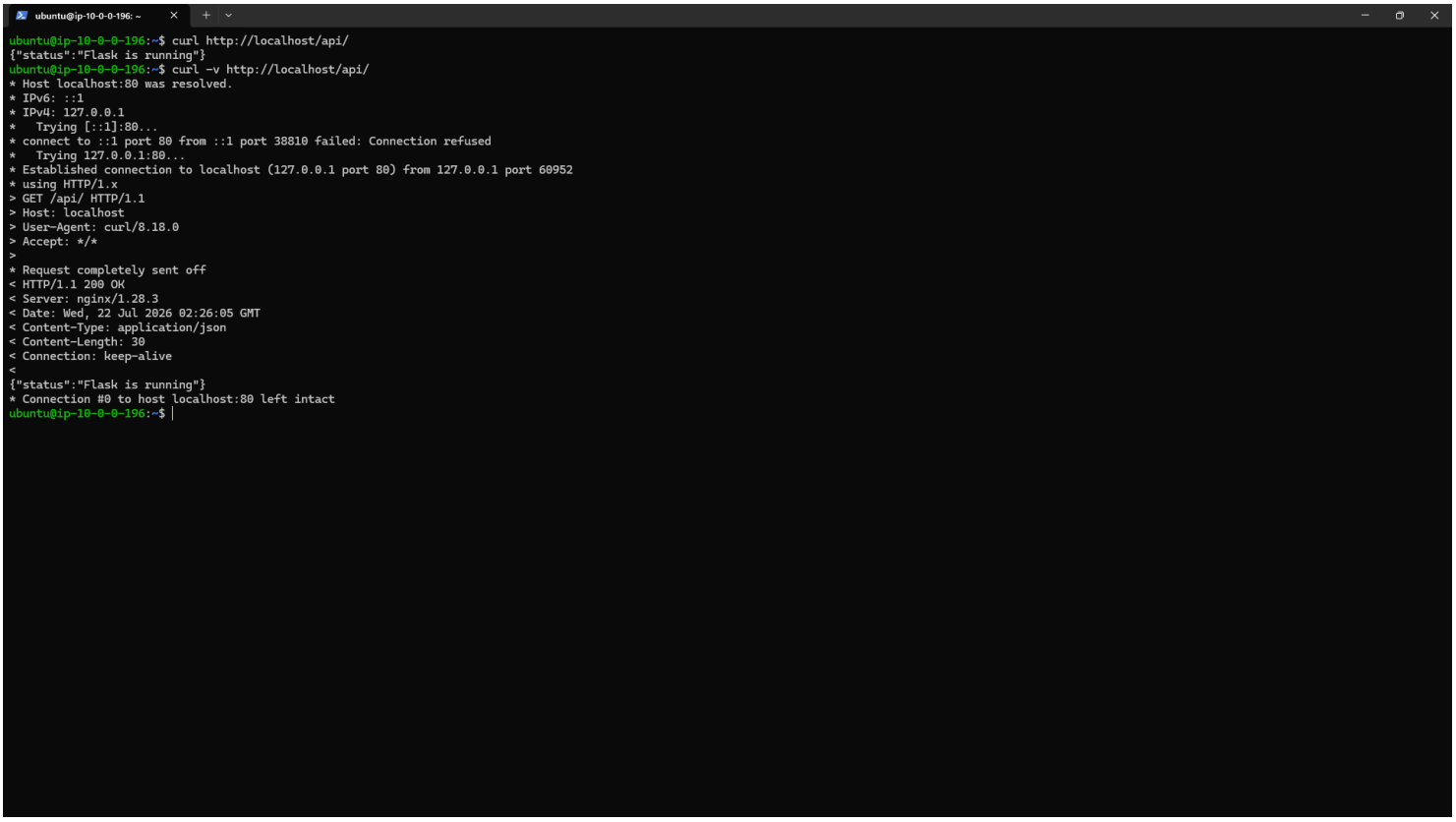
Web Application Accessibility : Accessed the Feedback Form through the Internet-facing ALB DNS in the browser. The application loaded successfully, confirming the Web Tier is publicly accessible through the Public Application Load Balancer.



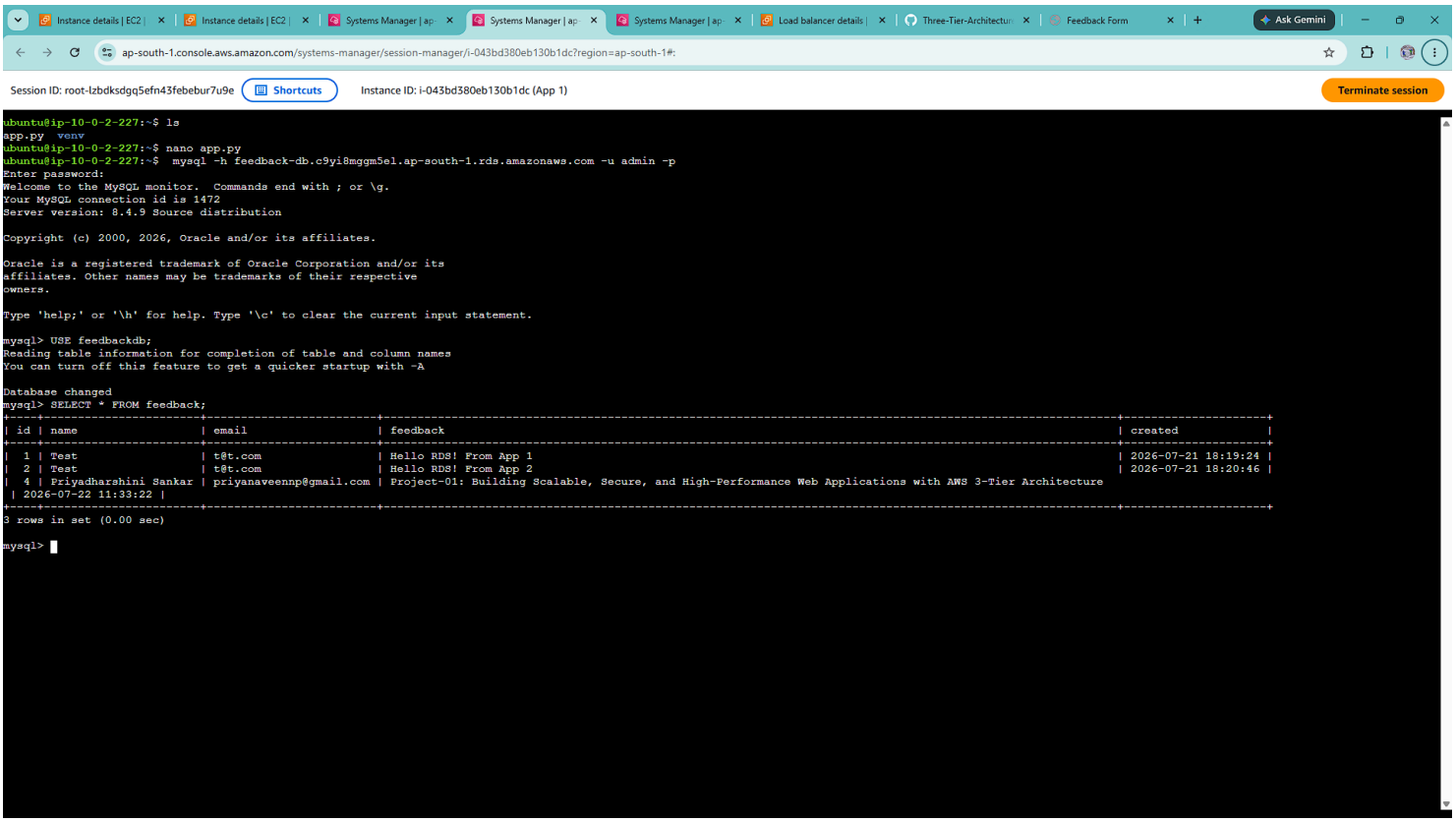
Feedback Form Submission : Filled in the form with name, email, and feedback details and clicked Submit. The application displayed **"Feedback submitted successfully!"**, confirming the form accepts user input and the Flask application processed the request correctly.



Nginx Reverse Proxy Validation : From the Web Tier instance (10.0.0.196), a request to localhost/api/ returned HTTP 200 OK with {"status":"Flask is running"} via Nginx, confirming Nginx is successfully forwarding API requests to the Internal Load Balancer.

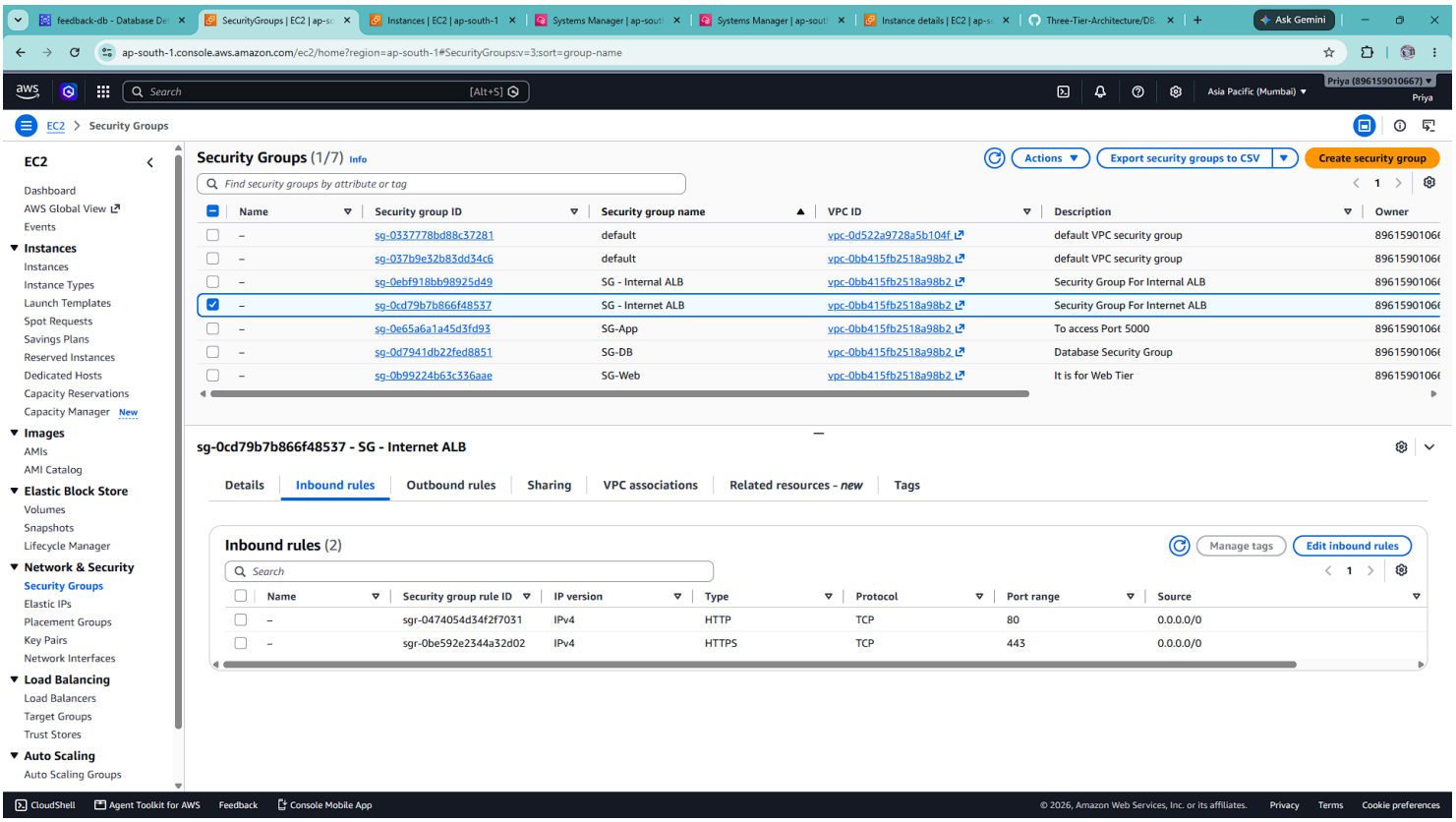


Flask Application & RDS Validation : Connected to App 1 via AWS Systems Manager and retrieved all records from the feedback table in feedbackdb. The result returned 3 rows including the newly submitted entry, confirming the Flask application is processing requests and storing data successfully in Amazon RDS MySQL.

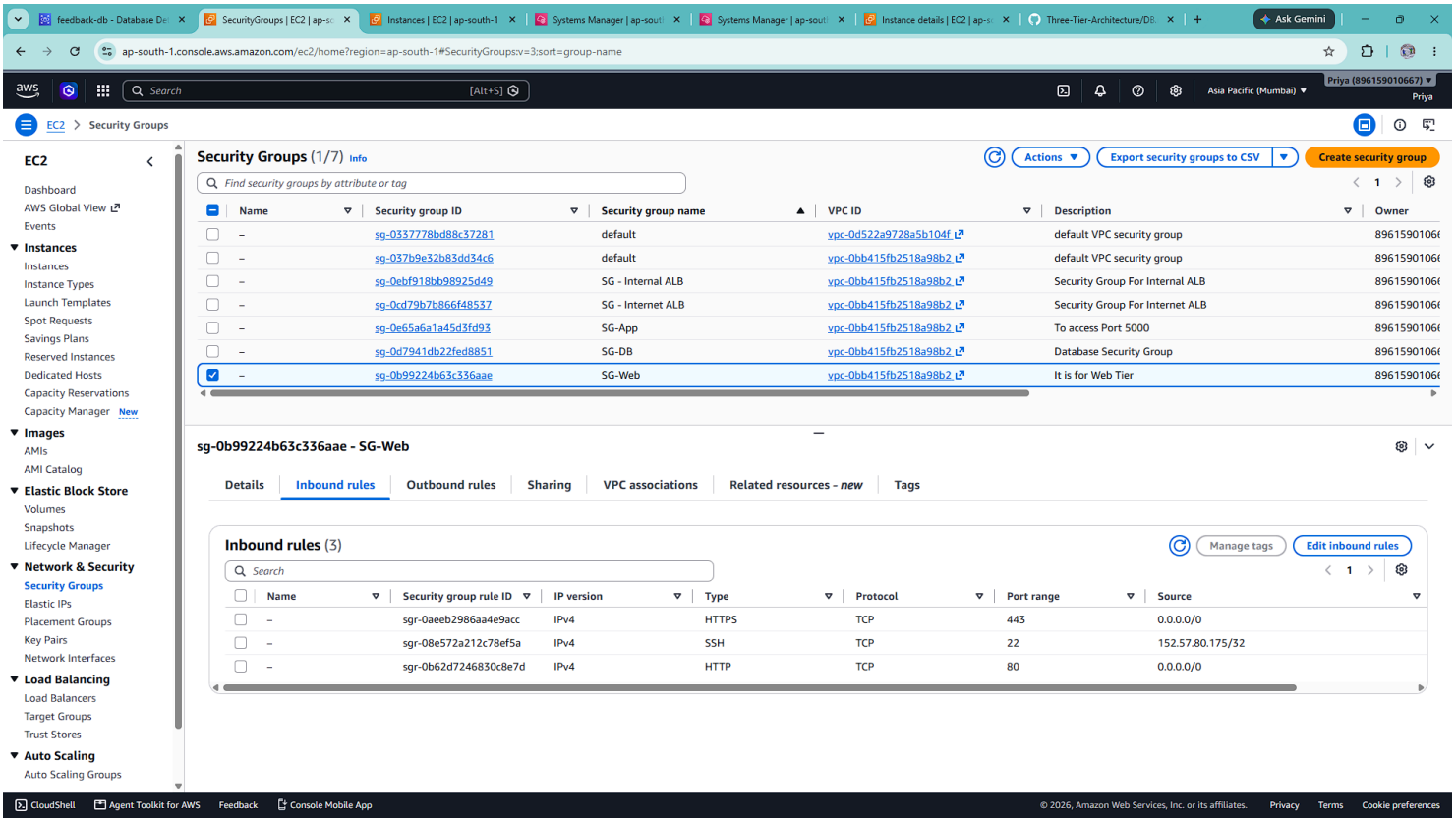


Security Groups Validation : Verified all 5 security groups. This confirms only the required traffic is allowed between each application tier.

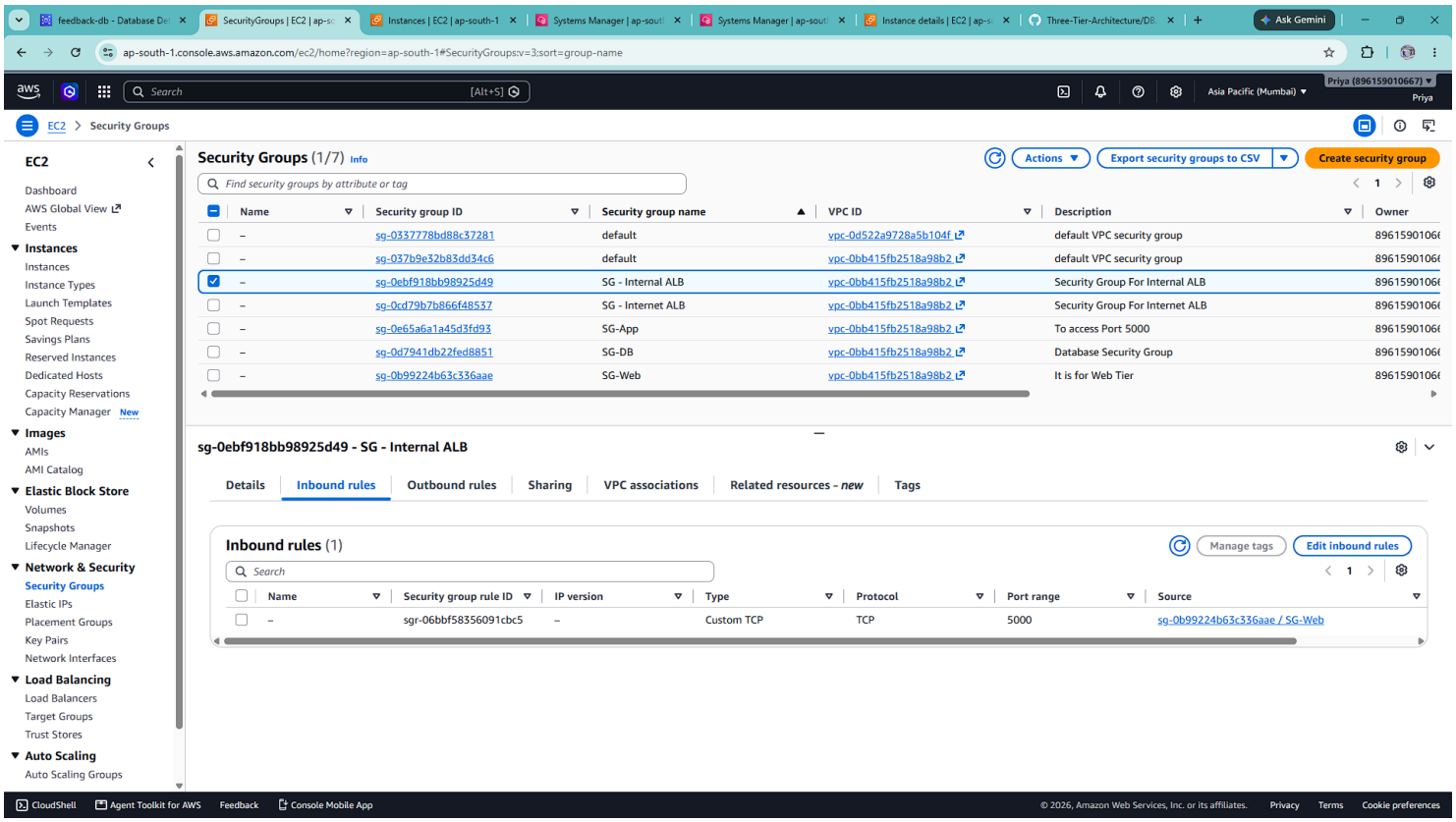
SG-Internet ALB (HTTP 80, HTTPS 443 from 0.0.0.0/0)



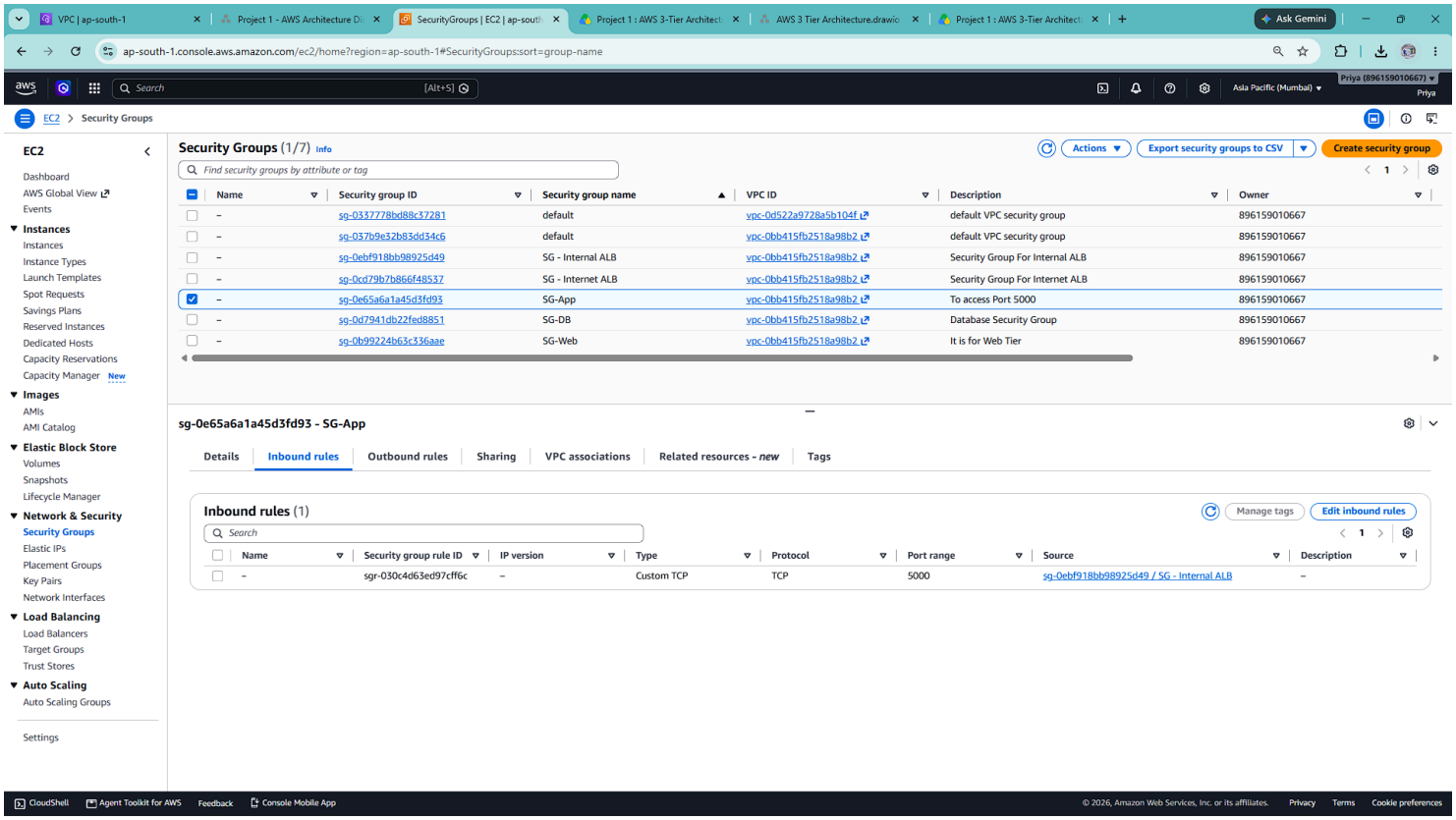
SG-Web (HTTP 80, HTTPS 443, SSH 22)



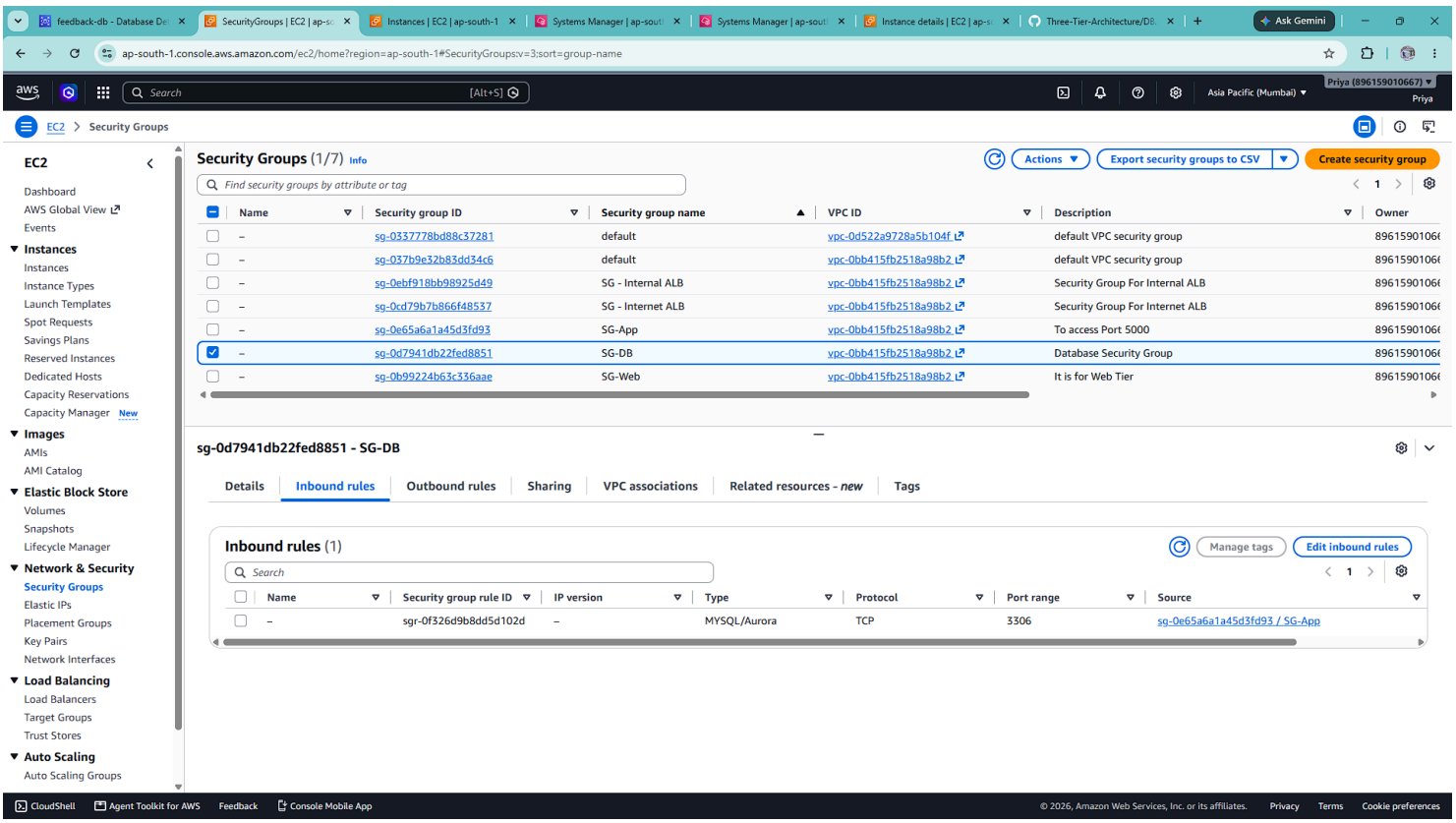
SG-Internal ALB (port 5000 from SG-Web)



SG-App (port 5000 from SG-Internal ALB)

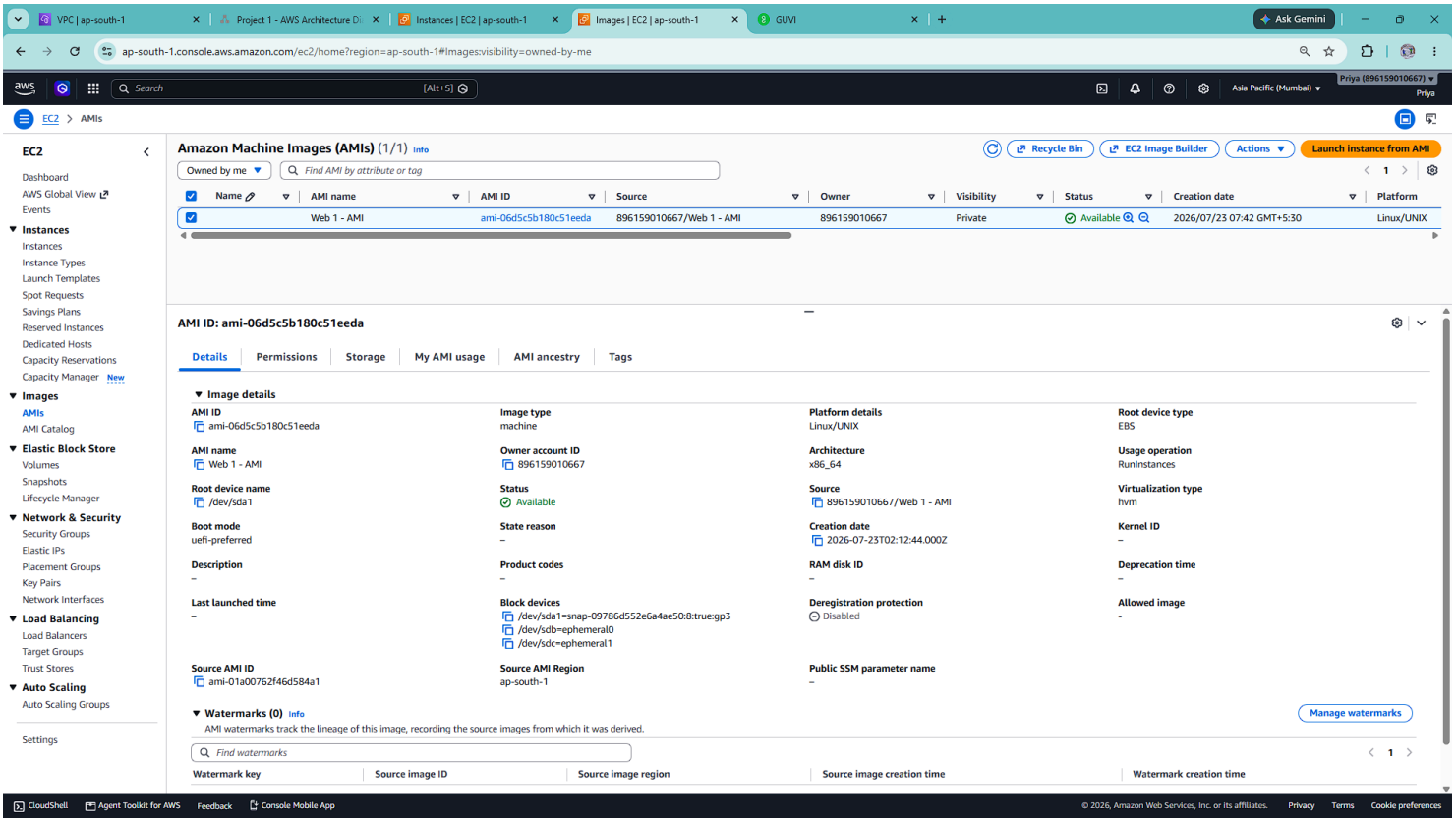


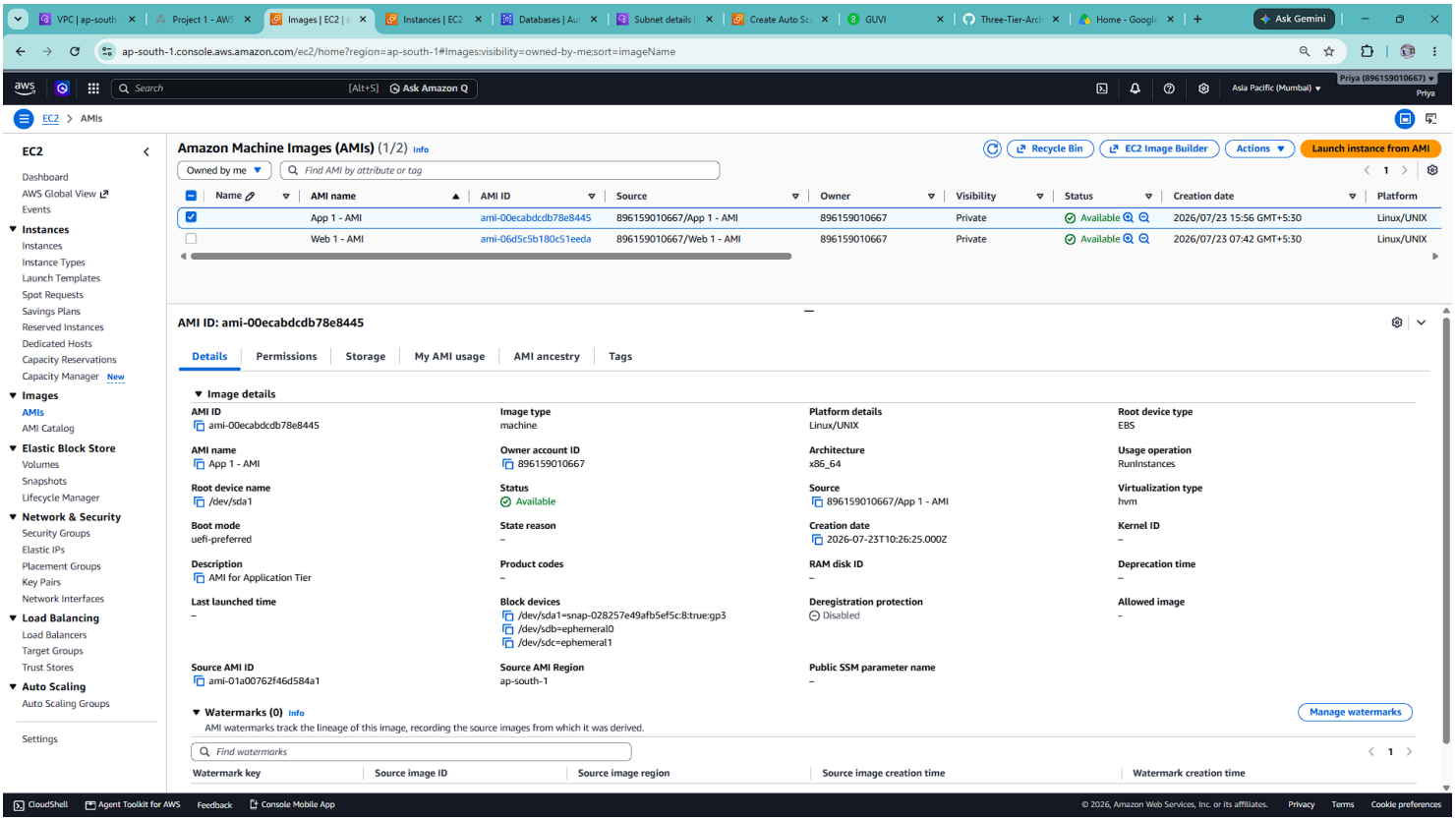
SG-DB (MySQL 3306 from SG-App only)



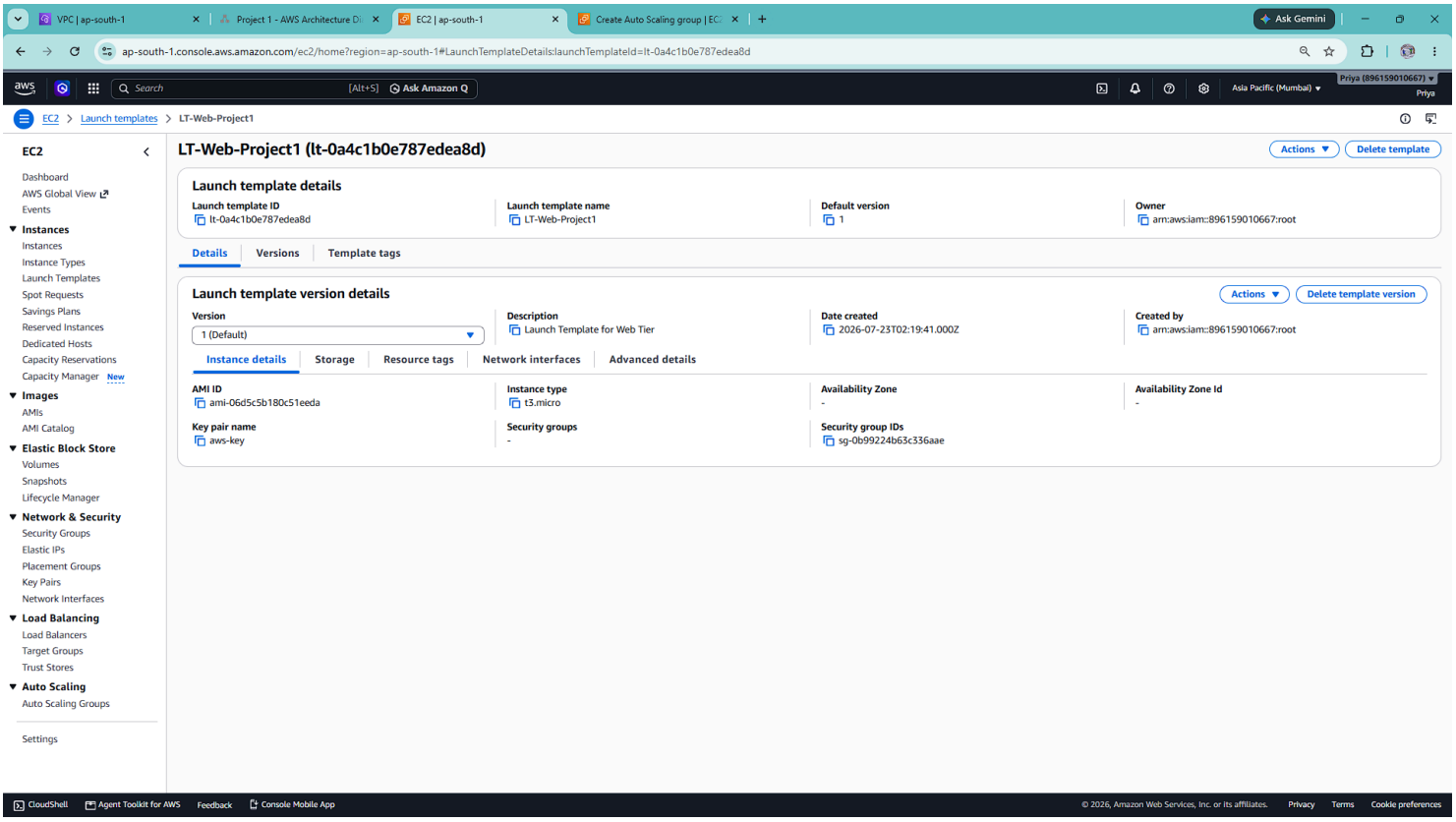
Additional Task (Bonus) : Configure Auto Scaling

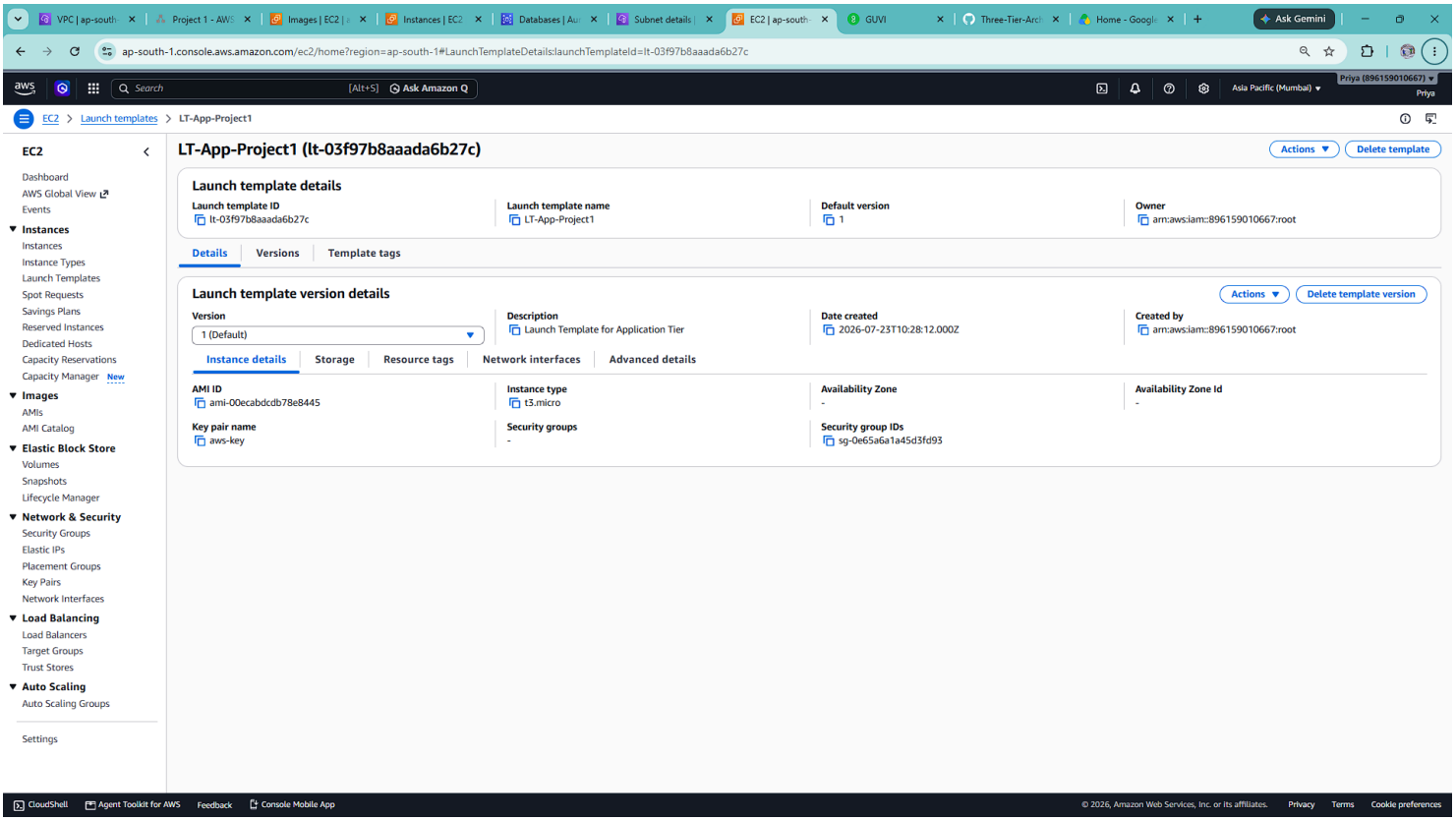
Created Custom AMIs : Created custom Amazon Machine Images "Web 1 - AMI" and "App 1 - AMI" from the fully configured Web and Application EC2 instances. These AMIs contain the operating system, installed software, application files, and all required configurations, ensuring that any new EC2 instance launched by Auto Scaling has the same configuration as the original instances.



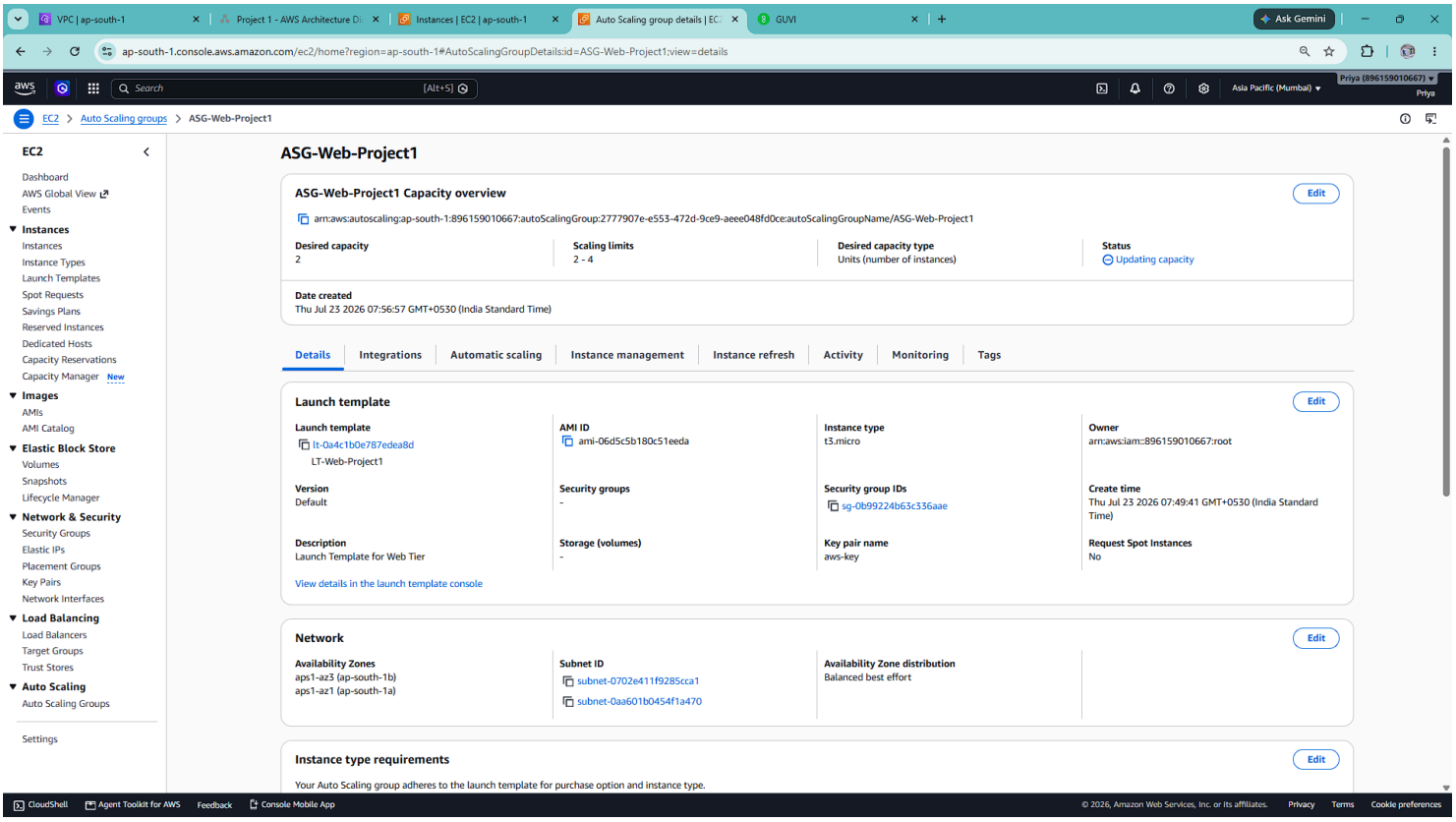


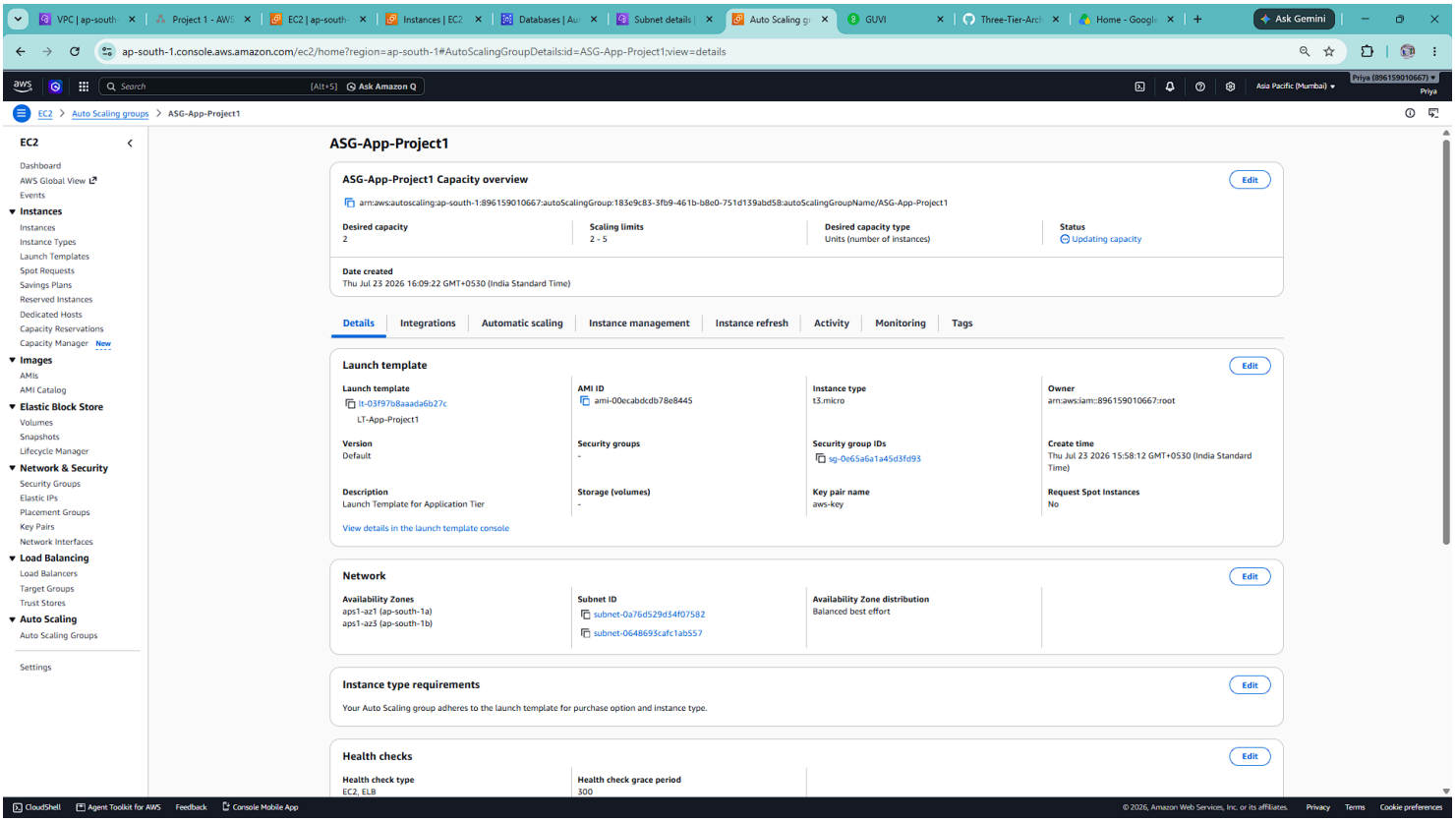
Launch Templates : Created Launch Templates "LT-Web-Project1" and "LT-App-Project1" using the respective custom AMIs. The Launch Template defines the EC2 instance configuration, including the AMI, instance type, security group, key pair, and storage settings.



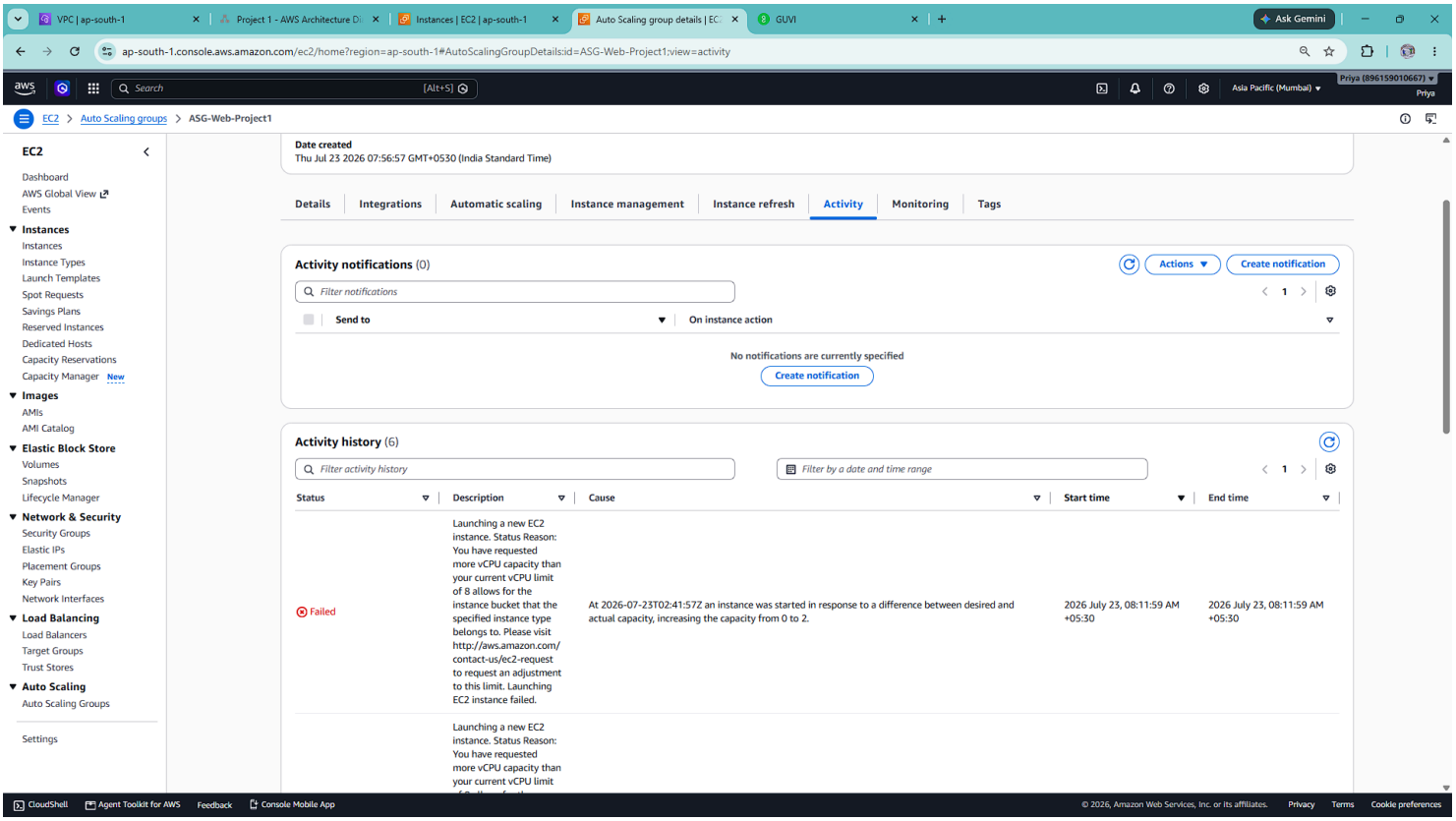


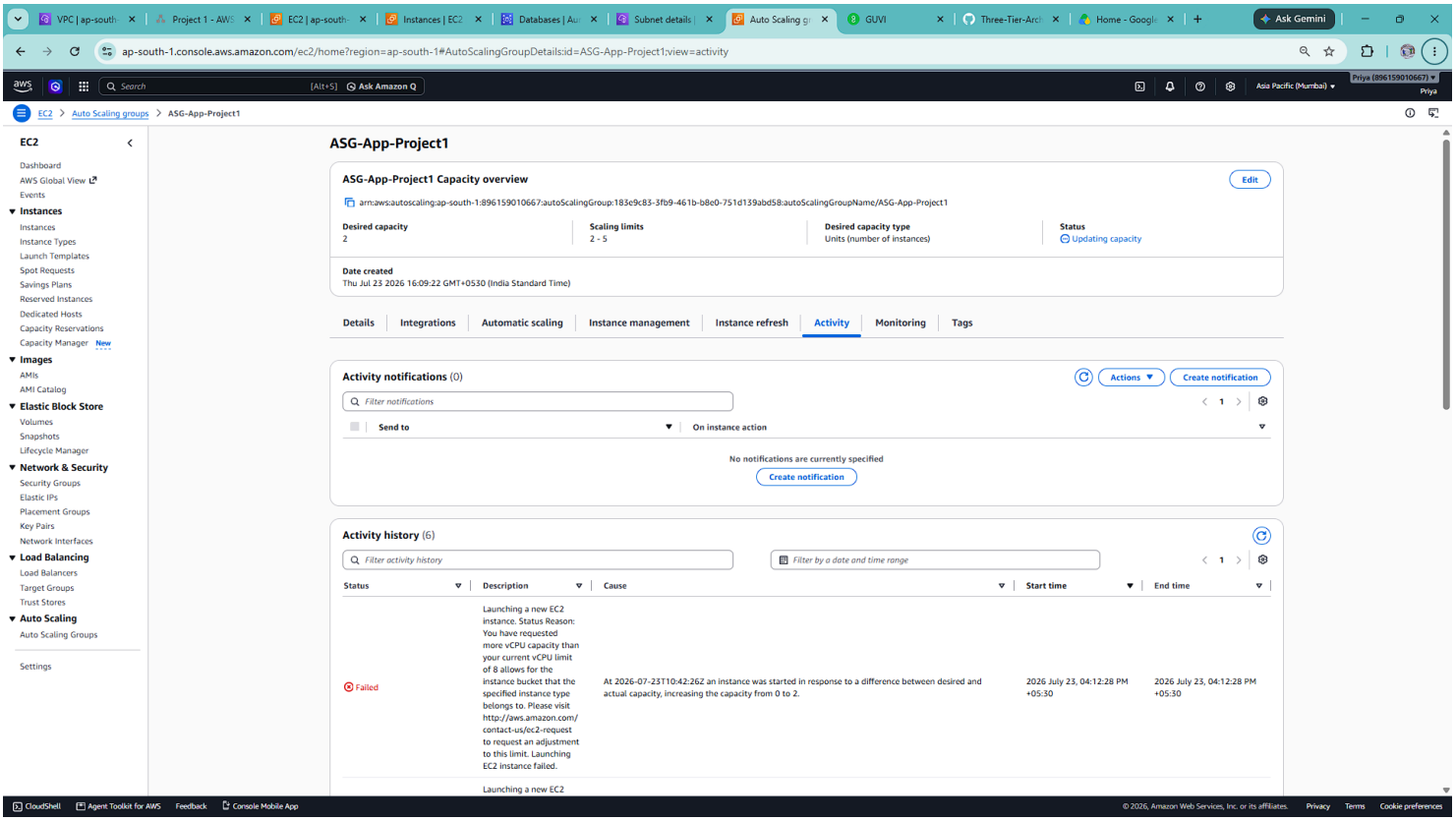
Auto Scaling Group Configuration : Created “ASG-Web-Project1” and “ASG-App-Project1” using their respective Launch Templates. The Web ASG was associated with the target group of the Internet-facing ALB, and the Application ASG was associated with the target group of the Internal ALB. The scaling configuration was set to Minimum: 2, Desired: 2, Maximum: 4 (Web) and 5 (Application). A 70% CPU Target Tracking Scaling Policy was configured for both Auto Scaling Groups.





Auto Scaling Activity : The Auto Scaling Group was configured successfully. During validation, the launch of additional EC2 instances was unsuccessful because the AWS account had reached its EC2 vCPU service quota. The configuration was correct, and the issue was caused by an AWS account quota limitation.





Submission Deliverables

Working Application Link (Public ALB DNS) :
Internet-ALB-Project1-70520780.ap-south-1.elb.amazonaws.com

Project Documentation Link :
https://docs.google.com/document/d/176NRslBQLbqE-NnM1lu4c2PSYtiQX5PlNuXBf5DuciU/edit?usp=sharing

AWS Architecture Diagram :
https://drive.google.com/file/d/1iovR_P8pSY9xsgfaLyy4a0-MkIFRHgCl/view?usp=sharing

SQL query output showing stored feedback records :

